МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Практическое занятие № 10
по дисциплине «Программирование корпоративных систем»

Тема работы: ПРОДВИНУТЫЙ УРОВЕНЬ ВЕБ-АВТОМАТИЗАЦИИ: СЛОЖНЫЕ СЕЛЕКТОРЫ И ОТКАЗОУСТОЙЧИВОСТЬ ВЕБ-РОБОТОВ

Вариант: 11

Выполнил:	ЭФБО-09-23		Калинин Н.В.
	(группа)	*(подпись)*	*(Фамилия И.О.)*
Принял:	доц.		Громов Е.В.
	(должность)	*(подпись)*	*(Фамилия И.О.)*

Москва

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

В девятой практической работе мы освоили базовые операции веб-автоматизации: открытие браузера, навигацию по URL, поиск элементов по ID и классам, клики, ввод текста. Это фундамент, на котором строятся любые RPA-решения для веб-интерфейсов. Однако реальные бизнес-задачи редко ограничиваются простыми линейными сценариями.

В современных RPA-проектах робот сталкивается с более сложными вызовами:

- Нестабильные селекторы – элементы интерфейса меняют ID, классы и расположение при обновлениях CRM

- Динамический контент – элементы загружаются асинхронно (AJAX), появляются с задержкой, лоадеры скрывают кнопки
- Несколько вкладок – робот открывает ссылки в новых вкладках, теряет управление при переключении
- Табличные данные – извлечение информации из отчетов, прайс-листов, реестров на веб-страницах
- Нештатные ситуации – ошибки, сбои, таймауты, требующие интеллектуальной обработки

Эти вызовы и составляют содержание нашей десятой практической работы. Мы переходим от уровня «базового пользователя» к уровню «профессионального разработчика», способного создавать надежные, отказоустойчивые и масштабируемые RPA-решения для веб-интерфейсов.

Мы изучим продвинутые методы поиска элементов – освоим Редактор XPATH и научимся создавать гибкие запросы с функциями contains, starts-with, text-is и логическими операторами. Рассмотрим продвинутые типы поиска – CSS-селекторы, поиск по tag_name, class_name, атрибуту name. Научимся работать с несколькими вкладками браузера – переключаться между вкладками, ждать загрузки контента в новой вкладке. Освоим извлечение табличных данных – массовый сбор структурированной информации с веб-страниц. Погрузимся в отказоустойчивость веб-роботов – разберем явные ожидания, обработку ошибок, повторные попытки (retries) и логирование. И наконец, детально рассмотрим продвинутые блоки веб-автоматизации в Puzzle RPA, которые не входили в девятую практику.

В результате вы будете не просто использовать базовые блоки веб-автоматизации, а осознанно проектировать сложные RPA-решения, способные:

- Устойчиво работать даже при изменениях интерфейса благодаря гибким XPATH-запросам
- Адаптироваться к динамическому контенту через явные ожидания и проверку состояния элементов
- Корректно обрабатывать несколько вкладок без потери управления
- Извлекать табличные данные любой сложности с веб-страниц
- Восстанавливаться после сбоев с помощью механизмов повторных попыток и логирования

Вы перейдете от уровня «базового пользователя» к уровню «профессионального разработчика», способного создавать надежные, масштабируемые и отказоустойчивые RPA-решения для любых веб-задач.

Редактор XPATH и сложные селекторы

При автоматизации веб-интерфейсов главная задача робота – найти нужный элемент на странице: кнопку, поле ввода, ссылку, таблицу. В первой практической работе мы использовали простые способы поиска – по идентификатору (`id`) или по классу (`class`). Однако в реальных системах эти атрибуты могут меняться

при каждом обновлении страницы. Разработчики CRM часто переименовывают классы, меняют структуру HTML-кода, и робот, который вчера работал, сегодня перестает находить кнопку.

XPATH (XML Path Language) – это язык запросов, который позволяет находить элементы на веб-странице не по фиксированным именам, а по их расположению, содержимому, части атрибутов или комбинации условий. Это делает поиск гибким и устойчивым к изменениям интерфейса.

Простыми словами: если простой поиск говорит роботу «найди кнопку с именем `submit_btn_12345`», то XPATH позволяет сказать «найди любую кнопку, на которой написано слово «Сохранить», независимо от того, как разработчик её назвал в коде».

XPATH рассматривает HTML-страницу как дерево элементов. Каждый тег – это узел, у которого есть родитель, дети и соседи. XPATH-запрос – это путь к нужному узлу.

В веб-автоматизации XPATH позволяет учитывать:

- расположение элемента на странице
- текст, который написан на элементе
- значения атрибутов (`id`, `class`, `name`, `type` и другие)
- частичное совпадение текста или атрибутов
- логические связи между условиями

Существует два основных типа XPATH-запросов: абсолютные и относительные.

Абсолютный XPATH – это полный путь от корня страницы до элемента. Такой запрос начинается с одного слеша и выглядит примерно так: `/html/body/div[2]/div[3]/form/div[1]/button`. Это самый ненадежный способ поиска. Если разработчик добавит новый блок на страницу или изменит порядок элементов, путь нарушится, и робот не найдет кнопку. Абсолютные XPATH лучше не использовать в промышленных роботах.

Относительный XPATH – это запрос, который ищет элемент в любом месте страницы, независимо от его положения в структуре. Такой запрос начинается с двух слешей и выглядит так: `//button[contains(text(), "Сохранить")]`. Робот найдет кнопку, даже если разработчик переместит её в другую часть страницы. Относительные XPATH – основа устойчивой веб-автоматизации.

Основные функции XPATH для гибкого поиска. В реальных сценариях мы редко знаем точное значение атрибута. Чаще мы знаем, что кнопка должна содержать слово «Сохранить» или что поле ввода начинается с «`email`». Для таких случаев в XPATH есть специальные функции.

- Функция `contains` позволяет искать элемент по частичному совпадению текста или атрибута. Это самая часто используемая функция. Например, если на странице есть кнопка с текстом «Сохранить изменения», мы можем найти её по слову «Сохранить». Если класс элемента содержит слово «`primary`»,

мы можем найти его по части этого слова. `contains` работает с любыми атрибутами и текстом.

- Функция `starts-with` ищет элементы, у которых значение атрибута начинается с определенной строки. Это удобно, когда идентификаторы элементов формируются динамически, но имеют постоянный префикс. Например, если поле ввода для `email` всегда имеет идентификатор, начинающийся с «`email_input_`», мы можем найти его по этому префиксу, игнорируя меняющуюся часть.
- Функция `text` позволяет искать элемент по точному тексту, который видит пользователь. Это самый понятный способ для поиска кнопок, ссылок, заголовков. Например, мы можем найти кнопку с текстом «Войти» или ссылку с текстом «Забыли пароль?».

Комбинирование условий позволяет создавать более точные запросы. Мы можем объединять несколько условий с помощью операторов and и or. Например, мы можем искать кнопку, которая одновременно содержит слово «Сохранить» и имеет класс «primary». Или искать поле ввода, которое является либо текстовым, либо email-полем. Комбинирование условий делает запрос максимально точным и устойчивым к изменениям.

Устойчивый XPATH – это запрос, который продолжает работать даже после изменений интерфейса. При написании запроса нужно следовать нескольким принципам:

- Используйте относительные пути. Всегда начинайте запрос с двойного слеша. Это говорит роботу: «ищи этот элемент в любом месте страницы, не привязывайся к структуре».
- Опирайтесь на текст, который видит пользователь. Если на кнопке написано «Отправить», используйте это слово в запросе. Текст меняется реже, чем идентификаторы и классы. Пользовательский интерфейс ориентирован на людей, и текст на кнопках остается стабильным.
- Используйте частичное совпадение. Вместо точного значения атрибута используйте `contains` или `starts-with`. Это защищает от ситуации, когда разработчик добавил суффикс или префикс к идентификатору.
- Избегайте привязки к порядковому номеру. Запросы типа `//div[3]` или `//tr[5]` ненадежны, потому что порядок элементов может измениться при добавлении или удалении блоков. Если нужно найти конкретную строку в таблице, лучше использовать её содержимое или комбинацию условий.
- Проверяйте уникальность запроса. Хороший XPATH должен находить ровно один элемент. Если запрос находит несколько элементов, робот не поймет, с каким из них взаимодействовать. Если запрос не находит ни одного элемента – он бесполезен. В Puzzle RPA есть функция проверки, которая показывает, сколько элементов найдено по вашему запросу.

Рассмотрим несколько типовых ситуаций, с которыми сталкивается разработчик RPA, и способы их решения с помощью XPATH.

Ситуация: кнопка, текст которой может меняться. На странице есть кнопка, которая в зависимости от контекста называется «Сохранить» или «Сохранить изменения». Нужно найти кнопку в любом случае. Решение: используем contains для поиска по части текста. Запрос `//button[contains(text(), "Сохранить")]` найдет кнопку в обоих вариантах.

Ситуация: элемент с динамическим идентификатором. Поле ввода email имеет идентификатор, который каждый раз меняется: `email_input_12345`, `email_input_67890`. Постоянной остается только часть «`email_input`». Решение: используем starts-with для поиска по началу атрибута. Запрос `//input[starts-with(@id, "email_input")]` найдет поле независимо от меняющейся части.

Ситуация: кнопка с несколькими признаками. Нужно нажать кнопку «Отправить», но на странице есть несколько кнопок с таким текстом. Нужная кнопка находится внутри формы с классом «main-form». Решение: комбинируем условия. Запрос `//form[@class="main-form"]//button[contains(text(), "Отправить")]` найдет кнопку только внутри нужной формы.

Ситуация: последняя строка таблицы. Нужно извлечь данные из последней добавленной строки динамической таблицы. Решение: используем функцию last(). Запрос `//table//tr[last()]` найдет последнюю строку таблицы, независимо от того, сколько строк в ней сейчас.

Ситуация: элемент, который может отсутствовать. Нужно проверить, появилось ли на странице сообщение об успешной отправке формы. Сообщение может иметь разные классы, но содержит слово «успешно». Решение: используем contains для поиска по тексту. Запрос `//div[contains(text(), "успешно")]` найдет сообщение, даже если его класс изменится.

Начинающие разработчики часто допускают ошибки, которые делают роботов ненадежными. Знание этих ошибок поможет их избежать.

- Ошибка: использование абсолютных путей. Новички часто нажимают «скопировать XPATH» в инструментах разработчика браузера и получают длинный абсолютный путь. Такой путь сломается при любом изменении структуры страницы. Всегда переделывайте абсолютные пути в относительные.
- Ошибка: слишком широкий запрос. Запрос `//button` найдет все кнопки на странице. Если попросить робота нажать на кнопку по такому запросу, он не поймет, какую именно кнопку нажимать. Всегда уточняйте запрос до уникального элемента.
- Ошибка: жесткая привязка к тексту. Запрос `//button[text()="Сохранить"]` найдет кнопку только если текст точно совпадает. Если разработчик добавит пробел в конце или изменит регистр, запрос перестанет работать. Используйте `contains` для гибкости.
- Ошибка: игнорирование проверки. Разработчик написал запрос, не проверил его на странице, и робот упал в прогоне. Всегда проверяйте запрос в редакторе Puzzle RPA, чтобы убедиться, что он находит нужный элемент.

Итак, XPATH – это инструмент, который делает веб-автоматизацию устойчивой к изменениям интерфейса. Вместо того чтобы привязываться к конкретным идентификаторам, вы учите робота понимать структуру страницы и находить элементы по их свойствам, тексту и расположению.

Ключевые принципы, которые нужно усвоить:

- Используйте относительные XPATH (начинаются с `//`)
- Применяйте функции `contains`, `starts-with` и `text` для гибкого поиска
- Комбинируйте условия с помощью `and` и `or` для точности
- Проверяйте уникальность запроса перед использованием
- Избегайте абсолютных путей и привязки к порядковым номерам

Редактор XPATH в Puzzle RPA позволяет создавать такие запросы визуально, без ручного написания кода. Используйте его возможности: визуальный выбор элементов, настройка условий, проверка результата. Это сэкономит время и избавит от ошибок.

Освоив XPATH, вы перейдете от хрупких роботов, которые ломаются при каждом обновлении, к устойчивым решениям, способным работать в реальных условиях, где интерфейсы меняются, а данные загружаются динамически.

Продвинутые типы поиска элементов

В девятой практической работе мы познакомились с базовым способом поиска элементов – по идентификатору (`id`) и по классу (`class`). Этого достаточно для простых сценариев, но в реальных веб-интерфейсах разработчики используют множество разных атрибутов для описания элементов. Более того, один и тот же элемент можно найти разными способами, и умение выбирать наиболее подходящий – важный навык профессионального разработчика RPA.

Puzzle RPA предоставляет шесть основных типов поиска элементов . Каждый тип имеет свои особенности, преимущества и сценарии применения. Все эти типы поиска доступны в выпадающем меню практически всех блоков веб-автоматизации – будь то клик по элементу, ввод текста, чтение свойства или поиск коллекции элементов .

1. Поиск по идентификатору (`id`)

Идентификатор – это уникальный атрибут, который разработчик присваивает элементу. По спецификации HTML, на странице не может быть двух элементов с одинаковым id. Это делает поиск по идентификатору самым быстрым и надежным способом.

Когда стоит использовать поиск по `id`: если у элемента есть идентификатор, это почти всегда лучший выбор. Робот найдет элемент мгновенно и точно.

Однако у этого способа есть ограничение: современные веб-фреймворки часто генерируют идентификаторы динамически. Например, `id` может выглядеть как `input_12345_67890`, где числа меняются при каждом обновлении страницы. В таком случае поиск по id становится ненадежным, и лучше использовать другие способы.

2. Поиск по имени класса (`class_name`)

Классы используются для группировки элементов, которые должны выглядеть одинаково. В отличие от идентификатора, один класс может быть присвоен множеству элементов на странице.

Поиск по имени класса полезен, когда нужно найти все элементы определенного типа. Например, все кнопки с классом `primary-button` или все строки таблицы с классом `table-row`.

При поиске по классу важно помнить, что элемент может иметь несколько классов одновременно. Например, кнопка может иметь классы `btn primary large`. В таком случае поиск сработает, если указать любой из этих классов, но лучше использовать комбинацию условий для большей точности.

3. Поиск по имени тега (`tag_name`)

Имя тега – это тип элемента HTML: `button`, `input`, `div`, `span`, `a`, `table` и другие. Поиск по имени тега – самый широкий способ поиска, потому что на любой странице есть множество элементов одного типа.

Этот способ редко используется сам по себе, но становится очень полезным в комбинации с другими условиями. Например, можно найти все поля ввода (input) внутри конкретной формы. Или найти все ссылки (a) в определенном блоке страницы.

Поиск по имени тега также незаменим, когда нужно собрать все элементы определенного типа, например, все изображения на странице или все строки таблицы.

4. Поиск по атрибуту `name`

Атрибут `name` традиционно используется для именования полей в формах, особенно при отправке данных на сервер. В современных веб-приложениях этот атрибут встречается реже, чем раньше, но во многих системах он остается важным идентификатором.

Поиск по `name` удобен, когда нужно найти поле ввода с определенным именем, например `email` или phone. Этот способ более надежен, чем поиск по динамическому идентификатору, потому что разработчики реже меняют атрибут name.

5. Поиск по CSS-селектору (`css-selector`)

CSS-селекторы – это язык запросов, который используется в веб-разработке для стилизации элементов. В RPA CSS-селекторы позволяют находить элементы с высокой гибкостью.

CSS-селекторы поддерживают:

- Поиск по идентификатору: `#submit-btn`
- Поиск по классу: `.primary-button`
- Поиск по типу элемента: `input`
- Поиск по атрибутам: `input[type="email"]`
- Комбинации: `form#login-form input[type="text"]`
- Поиск потомков: `div.container table tr`

- Псевдоклассы: `tr:last-child`, `button:first-of-type`

CSS-селекторы часто короче и читаемее, чем XPATH. Для многих задач они являются предпочтительным выбором. Однако возможности CSS-селекторов ограничены – они не могут, например, искать элемент по тексту, который видит пользователь, или перемещаться по дереву вверх (от потомка к родителю). В таких случаях на помощь приходит XPATH.

6. Поиск по XPATH (`xpath`)

XPATH – самый мощный, но и самый сложный способ поиска. Он позволяет находить элементы не только по атрибутам и тегам, но и по тексту, по части атрибута, по расположению относительно других элементов.

Поиск по XPATH – это универсальный инструмент, который может заменить все остальные способы. Однако его мощь требует осторожности: плохо написанный XPATH может быть медленным или ненадежным.

У каждого типа поиска есть свои сильные и слабые стороны. Профессиональный разработчик RPA выбирает способ поиска в зависимости от ситуации.

Если у элемента есть стабильный идентификатор – выбирайте поиск по `id`. Это самый быстрый и надежный вариант.

Если элемент имеет уникальный класс, который не используется на других элементах – подойдет поиск по `class_name`.

Если нужно найти все элементы определенного типа – используйте `tag_name`, возможно, в комбинации с другими условиями.

Если поле ввода имеет атрибут `name` – это хороший кандидат для поиска по `name`.

Если нужно найти элемент по комбинации признаков – CSS-селектор или XPATH дадут нужную гибкость. CSS-селектор предпочтительнее, когда его возможностей достаточно, потому что он проще и быстрее.

Если нужно найти элемент по тексту, который видит пользователь, или по части атрибута – выбирайте XPATH. Это единственный способ справиться с такими задачами.

Во всех описанных типах поиска есть важное различие: ищете ли вы один элемент или несколько. Блоки веб-автоматизации в Puzzle RPA разделяют эти сценарии.

Для поиска одного элемента используются блоки, которые возвращают первый найденный элемент. Если элементов несколько, робот выберет первый из них. Если элементов нет – возникнет ошибка.

Для поиска всех элементов, подходящих под условия, используется специальный блок «Найти все элементы». Этот блок возвращает список объектов, который может быть пустым, если ничего не найдено. Затем этот список можно обрабатывать – перебирать в цикле, извлекать из каждого элемента нужные данные, выполнять действия с каждым элементом.

Это особенно полезно при работе с таблицами, списками результатов поиска, каталогами товаров – везде, где нужно обработать множество однотипных элементов.

Puzzle RPA предоставляет единый интерфейс для работы со всеми типами поиска – Мастер UI (или Редактор XPATH). Этот инструмент открывается кнопкой редактирования на любом блоке веб-автоматизации .

Мастер UI позволяет:

- Визуально выбрать элемент на странице – просто кликните по нему мышью
- Выбрать тип поиска из выпадающего списка
- Настроить параметры поиска в удобном интерфейсе
- Проверить, сколько элементов найдено по вашему запросу
- Увидеть результат проверки прямо в мастере

Этот инструмент одинаково полезен для всех типов поиска. Даже если вы предпочитаете писать CSS-селекторы вручную, мастер поможет проверить их корректность и убедиться, что они находят именно то, что нужно.

Рассмотрим несколько ситуаций, где разные типы поиска проявляют свои преимущества.

Ситуация: уникальная кнопка с идентификатором. На странице есть кнопка отправки формы с `id="submit-order"`. Это идеальный случай для поиска по id. Робот найдет кнопку мгновенно, и запрос будет максимально надежным.

Ситуация: все товары в каталоге. Нужно собрать все карточки товаров на странице интернет-магазина. Каждая карточка имеет класс `"product-item"`. Используем поиск по `class_name`, чтобы найти все элементы с этим классом. Блок «Найти все элементы» вернет список, который можно перебрать в цикле.

Ситуация: поле ввода с динамическим `id`. Поле для ввода `email` имеет `id`, который меняется при каждой загрузке: `email_field_12345`, `email_field_67890`. При этом у поля есть атрибут `name="email"`. Вместо поиска по `id` используем поиск по `name` – это надежнее.

Ситуация: кнопка, которую нужно найти внутри формы. На странице есть несколько кнопок с одинаковым текстом «Удалить». Нужная находится внутри формы с классом "user-profile". Здесь поможет CSS-селектор: `form.user-profile button`. Он найдет кнопку только внутри указанной формы.

Ситуация: последняя строка таблицы. Нужно извлечь данные из последней добавленной строки динамической таблицы. CSS-селектор `tr:last-child` справится с этой задачей. Он найдет последнюю строку таблицы независимо от того, сколько строк в таблице.

Ситуация: элемент, который нужно найти по тексту. На странице есть сообщение об успешной операции, но у него нет уникальных атрибутов. Единственный способ его найти – по тексту: `//div[contains(text(), "успешно")]`. Это задача для XPATH.

Подведём итоги:

- Поиск по идентификатору – самый быстрый и надежный, если `id` стабилен.
- Поиск по имени класса подходит для групповых операций с однотипными элементами.
- Поиск по имени тега полезен в комбинации с другими условиями.
- Поиск по атрибуту `name` хорош для полей форм.
- CSS-селекторы дают хороший баланс между простотой и гибкостью.
- XPATH – самый мощный инструмент, способный решить практически любую задачу поиска.
- Для поиска одного элемента используйте соответствующие блоки веб-автоматизации. Для поиска коллекции элементов используйте блок «Найти все элементы», который возвращает список для дальнейшей обработки.
- Мастер UI помогает создавать и проверять запросы любого типа, не требуя ручного написания кода.

Умение выбирать правильный тип поиска для каждой ситуации – признак профессионального разработчика RPA. Это знание позволяет создавать роботов, которые не только работают корректно, но и остаются понятными для поддержки и устойчивыми к изменениям интерфейса.

Работа с несколькими вкладками браузера

В реальных сценариях веб-автоматизации робот редко ограничивается работой с одной вкладкой. При переходе по ссылкам, скачивании файлов, работе с внешними сервисами браузер постоянно открывает новые вкладки. Если робот не умеет управлять ими, он может потерять фокус и начать выполнять действия не в том окне, что приведет к сбою.

Представьте, что вы сами работаете в браузере: открываете ссылку в новой вкладке, переключаетесь между ними, закрываете ненужные. Робот должен делать то же самое, но автоматически. В Puzzle RPA для этого есть специальные механизмы, которые позволяют роботу точно знать, какая вкладка сейчас активна, и переключаться между ними по номеру.

В Puzzle RPA каждая открытая вкладка имеет порядковый номер. Нумерация начинается с нуля. Первая открытая вкладка имеет номер 0, вторая – номер 1, третья – номер 2 и так далее .

Это важно понимать, потому что во многих блоках веб-автоматизации есть параметр «Номер вкладки по порядку». Указав этот номер, вы говорите роботу: «выполни действие именно в этой вкладке». Если не указывать номер, робот по умолчанию работает с текущей активной вкладкой.

Робот всегда знает, какая вкладка активна в данный момент. Активная вкладка – это та, которая видна на экране и с которой робот будет взаимодействовать, если вы не укажете другой номер вкладки.

При открытии браузера через блок «Открыть браузер» робот автоматически делает открытую страницу активной. Когда робот кликает по ссылке, которая от-

крывается в новой вкладке, активной становится именно эта новая вкладка. Однако это поведение может зависеть от настроек браузера и способа открытия ссылки.

Поэтому важно не полагаться на автоматическое переключение, а явно управлять вкладками, указывая их номера в блоках.

Выбор вкладки. Самый простой способ переключиться на нужную вкладку – указать её номер в любом блоке, который выполняет действие на странице. Например, если вы знаете, что нужная вам форма находится во второй вкладке (номер 1), вы просто указываете этот номер в параметрах блока клика или ввода текста. Робот автоматически переключится на эту вкладку перед выполнением действия.

Обновление вкладки. Иногда требуется обновить страницу в конкретной вкладке. Для этого есть блок «Обновить вкладку браузера» . В параметрах блока вы указываете сессию браузера и номер вкладки, которую нужно обновить. Это полезно, когда страница зависла или нужно загрузить актуальные данные.

Закрытие вкладки. При работе с несколькими вкладками важно уметь закрывать ненужные, чтобы не загромождать память и не запутывать робота. Хотя в документации явно не выделен отдельный блок для закрытия вкладки, это можно сделать, закрыв браузер целиком или переключившись на другую вкладку и позволив ненужной закрыться автоматически. В более сложных сценариях можно использовать выполнение JavaScript-кода для закрытия текущей вкладки.

Рассмотрим несколько ситуаций, с которыми регулярно сталкивается разработчик RPA при автоматизации веб-интерфейсов.

Сценарий 1: переход по ссылке в новой вкладке

Представьте, что на странице есть список клиентов, и для каждого клиента есть ссылка «Подробнее», которая открывает карточку клиента в новой вкладке. Роботу нужно обработать каждого клиента: перейти по ссылке, извлечь данные, вернуться обратно.

Алгоритм действий:

- Находим все ссылки «Подробнее» на странице (например, с помощью блока «Найти все элементы»)
- Для каждой ссылки кликаем по ней. При клике открывается новая вкладка с номером 1 (если исходная вкладка была под номером 0)
- Работаем с новой вкладкой: извлекаем нужные данные, заполняем формы
- Закрываем обработанную вкладку или переключаемся обратно на исходную
- Переходим к следующему клиенту

Сценарий 2: работа с внешним сервисом

Основная страница работает в первой вкладке. При нажатии кнопки «Оплатить» открывается платежный шлюз в новой вкладке. Робот должен:

- Запомнить номер основной вкладки (0)
- Нажать кнопку оплаты

- Переключиться на новую вкладку (номер 1) – либо явно указав её номер в блоках, либо зная, что робот автоматически переключился
- Заполнить данные для оплаты, подтвердить
- Вернуться на основную вкладку (номер 0) для продолжения работы

Сценарий 3: обработка нескольких вкладок одновременно

Иногда требуется обрабатывать данные из нескольких открытых вкладок, сравнивая или перенося информацию между ними. Например, робот открывает два отчета в разных вкладках, извлекает данные и объединяет их. Здесь важно уметь переключаться между вкладками, явно указывая их номера.

При работе с несколькими вкладками есть несколько важных моментов, которые нужно учитывать:

- Нумерация не всегда предсказуема. Номера вкладок присваиваются в порядке открытия. Если вы закрыли вкладку с номером 1, оставшиеся вкладки не перенумеровываются. Вкладка, которая была номер 2, остается номером 2, а новой открытой вкладке присвоится следующий номер. Поэтому важно отслеживать, какие вкладки открыты, и не полагаться на то, что нужная вам вкладка всегда будет иметь один и тот же номер.
- Активная вкладка при открытии. Поведение браузера при открытии ссылки в новой вкладке может отличаться. Обычно фокус остается на исходной вкладке, если ссылка открывается в фоне. Но некоторые веб-приложения с помощью JavaScript могут переключать фокус на новую вкладку. Лучше не полагаться на автоматическое переключение и всегда явно указывать номер вкладки в блоках.
- Проверка существования вкладки. Если вы пытаетесь выполнить действие во вкладке с номером, который не был открыт, робот выдаст ошибку. Поэтому перед переключением на вкладку стоит убедиться, что она существует. Это можно сделать, например, попробовав получить заголовок страницы или URL вкладки.

Чтобы работа с вкладками была надежной, придерживайтесь нескольких простых правил:

- Сохраняйте номер исходной вкладки в переменную. Перед тем как открыть новую вкладку, сохраните номер текущей вкладки в переменную. Тогда после обработки новой вкладки вы сможете легко вернуться на исходную, даже если за время работы открылось еще несколько вкладок.
- Явно указывайте номер вкладки. Не надейтесь на то, что робот сам переключится на нужную вкладку. В каждом блоке, который взаимодействует со страницей, указывайте параметр «Номер вкладки по порядку» . Это делает алгоритм понятным и предсказуемым.
- Закрывайте ненужные вкладки. Если вкладка больше не нужна, закройте ее. Это не только освобождает ресурсы, но и упрощает навигацию – меньше вкладок, меньше путаницы с номерами.

- Используйте осмысленные комментарии. Когда в алгоритме участвуют несколько вкладок, обязательно комментируйте, какая вкладка за что отвечает. Это поможет вам самим и тем, кто будет сопровождать вашего робота, не запутаться в номерах.

В заключении отметим, работа с несколькими вкладками браузера – обязательный навык для профессионального разработчика RPA. В реальных проектах робот почти всегда сталкивается с ситуациями, когда нужно переключаться между вкладками, обрабатывать информацию в разных окнах и возвращаться обратно.

Ключевые принципы:

- Каждая вкладка имеет порядковый номер, начиная с нуля
- В блоках веб-автоматизации можно указать номер вкладки, в которой нужно выполнить действие
- Не полагайтесь на автоматическое переключение – явно указывайте номер вкладки
- Сохраняйте номер исходной вкладки в переменную перед открытием новых
- Закрывайте ненужные вкладки, чтобы не запутаться в нумерации

Умение управлять вкладками делает робота гибким и позволяет автоматизировать сложные сценарии, где информация распределена по разным страницам и окнам браузера.

Извлечение табличных данных

В реальных бизнес-задачах веб-страницы часто содержат структурированную информацию, представленную в виде таблиц. Это могут быть списки клиентов, прайс-листы, отчеты о продажах, реестры документов, журналы операций. Робот, который автоматизирует работу с такими данными, должен уметь не просто кликать по кнопкам, но и собирать информацию из таблиц для дальнейшей обработки.

Извлечение табличных данных – это процесс, при котором робот находит на веб-странице HTML-таблицу, считывает её содержимое и сохраняет в структурированном виде, пригодном для дальнейшей обработки. В Puzzle RPA для этой задачи предназначен специальный блок «Извлечь табличные данные» .

В отличие от пошагового считывания каждой ячейки, этот блок позволяет получить всю таблицу целиком за одно действие. Это не только упрощает разработку, но и ускоряет работу робота.

Блок «Извлечь табличные данные» работает по простому принципу: вы указываете роботу, какую именно таблицу на странице нужно обработать, а он возвращает вам все данные из этой таблицы в виде списка .

Результат извлечения представляет собой список, где каждый элемент списка соответствует одной строке таблицы. Внутри каждой строки данные организованы как словарь или список значений – в зависимости от структуры таблицы. Такой формат удобен для последующей обработки: строки можно перебирать в цикле, извлекать нужные столбцы, сохранять в Excel или базу данных.

Важно понимать, что блок извлекает именно табличные данные в том виде, как они представлены на странице. Если таблица имеет сложную структуру (объединенные ячейки, вложенные таблицы), результат может потребовать дополнительной обработки. Однако для большинства стандартных HTML-таблиц блок работает корректно.

Главная задача при извлечении табличных данных – правильно указать роботу, какую именно таблицу нужно обработать. На странице может быть несколько таблиц: одна с клиентами, другая с заказами, третья с историей операций. Робот должен точно знать, какую из них извлекать.

Для этого используется Мастер UI (Редактор XPATH), который открывается кнопкой редактирования на блоке «Извлечь табличные данные». С помощью мастера вы можете:

- Нажать кнопку «Выбрать» и кликнуть на таблицу в открытом браузере
- Увидеть, какой XPATH сгенерировался для выбранной таблицы
- Проверить, сколько элементов найдено по этому запросу
- При необходимости уточнить запрос, добавив атрибуты или изменив условия поиска

После того как вы сохранили настройки, робот будет использовать этот XPATH для поиска таблицы при каждом запуске.

Результат работы блока «Извлечь табличные данные» – это список, который можно использовать в других блоках. Рассмотрим типовые сценарии обработки.

Проверка количества строк. Часто бывает нужно убедиться, что таблица не пуста. Вы можете извлечь таблицу, затем с помощью блока «Получить размер списка» узнать, сколько строк в ней содержится, и принять решение – продолжать работу или завершиться с ошибкой.

Перебор строк в цикле. Если таблица содержит список объектов для обработки (например, список заказов), вы можете поместить блок извлечения перед циклом «Для каждого элемента из списка». Внутри цикла каждая строка будет доступна как отдельный элемент, и вы сможете извлекать из неё нужные поля.

Сохранение в Excel. Часто извлеченные данные нужно сохранить для отчетности или дальнейшего анализа. Вы можете передать полученный список в блоки работы с Excel и сохранить его в файл в папке ресурсов проекта.

Вывод на экран. Для отладки или для простых сценариев можно использовать блок «Уведомление», чтобы показать извлеченные данные пользователю. Однако для больших таблиц это неудобно – лучше сохранять в файл.

Важно понимать, что не все таблицы на веб-страницах созданы одинаково. Разработчики могут использовать разные подходы к верстке, и это влияет на то, как робот извлекает данные.

Стандартные HTML-таблицы. Таблицы, созданные с помощью тега table и вложенных тегов tr (строки) и td (ячейки), обрабатываются блоком лучше всего. Робот правильно определяет структуру строк и столбцов.

Таблицы, сверстанные на div и span. Некоторые современные веб-интерфейсы используют гибкую верстку, где таблица визуально выглядит как таблица, но в HTML-коде представлена наборами div с классами. В таких случаях первичным ключом элемента может быть не table, а div с определенным классом. Блок «Извлечь табличные данные» позволяет указать любой тег, главное – правильно настроить атрибуты для поиска .

Таблицы с пагинацией. Часто таблицы на веб-страницах разбиты на страницы, чтобы не загружать все данные сразу. В этом случае извлечение одной страницы не дает всей информации. Робот должен уметь переключать страницы, последовательно извлекать данные с каждой и объединять результаты. Это более сложный сценарий, который требует использования циклов и работы с элементами навигации.

Динамически загружаемые таблицы. Некоторые таблицы подгружаются асинхронно, когда пользователь прокручивает страницу вниз. В таких случаях перед извлечением данных нужно убедиться, что все строки загружены. Это можно сделать с помощью явных ожиданий или прокрутки страницы до конца.

Начинающие разработчики часто допускают ошибки, которые приводят к тому, что робот либо не находит таблицу, либо извлекает не те данные.

Ошибка: слишком широкий поиск. Если вы указали только первичный ключ table без дополнительных атрибутов, а на странице несколько таблиц, робот может извлечь первую попавшуюся. Всегда уточняйте поиск с помощью атрибутов класса или идентификатора.

Ошибка: неправильный тег. Если таблица сверстана с помощью div, а вы ищете тег table, робот ничего не найдет. Используйте Мастер UI, чтобы точно определить, какой тег содержит табличные данные.

Ошибка: игнорирование ожидания. Если таблица загружается после выполнения какого-либо действия (например, после нажатия кнопки «Поиск»), нужно сначала дождаться её появления. Используйте блок «Ожидать появление элемента» перед извлечением данных.

Ошибка: работа с пагинированной таблицей. Извлечение только первой страницы вместо всех данных – частая ошибка. Если таблица разбита на страницы, робот должен уметь переходить по страницам и собирать данные целиком.

Рассмотрим простой сценарий: на странице CRM есть таблица с клиентами. Нужно извлечь всех клиентов и сохранить их в Excel-файл.

Алгоритм работы робота:

- Открыть браузер и перейти на страницу со списком клиентов
- Используя блок «Извлечь табличные данные», найти таблицу с классом clients-table и извлечь все строки
- Проверить, что список не пуст – если пуст, завершиться с ошибкой
- Сохранить полученный список в Excel-файл с помощью блоков работы с Excel
- Сохранить файл в папке ресурсов проекта под именем clients.xlsx

- Вывести уведомление о количестве обработанных клиентов

Этот алгоритм позволяет за несколько минут автоматизировать процесс, который вручную занимал бы десятки минут, особенно если клиентов много и таблица большая.

Подведём итоги:

- Извлечение табличных данных – ключевой навык для автоматизации бизнес-процессов, связанных с обработкой структурированной информации.
- Блок «Извлечь табличные данные» позволяет получить всю таблицу целиком за одно действие . Для его настройки нужно указать тег таблицы и атрибуты для поиска.
- Мастер UI помогает визуально выбрать таблицу на странице и проверить корректность настроек .
- Результат извлечения – список строк, который можно обрабатывать в цикле, сохранять в Excel, выводить на экран или передавать в другие блоки.
- При работе с таблицами важно учитывать их особенности: пагинацию, динамическую загрузку, нестандартную верстку. В сложных случаях может потребоваться комбинация извлечения с ожиданиями, прокруткой страницы и циклами для перебора страниц.

Умение извлекать табличные данные превращает робота из простого кликера в полноценного сборщика и обработчика информации, что открывает возможности для автоматизации широкого круга бизнес-задач – от формирования отчетов до миграции данных между системами.

Отказоустойчивость веб-роботов

В девятой практической работе мы создавали роботов, которые работают в идеальных условиях: страница загружается мгновенно, все элементы находятся на своих местах, сеть стабильна. Но в реальном мире так не бывает. Веб-страницы загружаются медленно, серверы дают сбои, интерфейсы обновляются, элементы появляются с задержкой.

Отказоустойчивость – это способность робота сохранять работоспособность и корректно завершать выполнение задач даже при возникновении нештатных ситуаций. Это не просто «обработка ошибок», а целый комплекс подходов и инструментов, которые делают робота устойчивым к реальным условиям эксплуатации.

В контексте RPA ошибка – это не просто синтаксическая ошибка в коде. Это любая ситуация во время выполнения, когда робот не может продолжить работу из-за внешних факторов: элемент не найден, страница не загрузилась, таймаут истек, всплыло неожиданное окно. Если такие ситуации не обработаны, робот аварийно завершается, и процесс, который он автоматизирует, останавливается .

Чтобы строить отказоустойчивых роботов, нужно понимать, что именно может пойти не так. Рассмотрим наиболее частые причины сбоев.

- Нестабильная загрузка страниц. Это самая распространенная проблема. Элементы страницы могут загружаться асинхронно: сначала появляется шапка,

потом меню, потом таблица с данными. Если робот попытается кликнуть по кнопке, которая еще не загрузилась, он получит ошибку «элемент не найден».

- Динамическое изменение интерфейса. Веб-приложения обновляются. Разработчики меняют идентификаторы, переименовывают классы, изменяют структуру DOM. Робот, который ищет элемент по жестко заданному id, после обновления перестает его находить.
- Сетевые задержки и таймауты. Медленный интернет, высокая нагрузка на сервер, временные сбои связи – все это приводит к тому, что страница загружается дольше обычного. Если робот не умеет ждать, он начнет выполнять действия на незагруженной странице.
- Неожиданные модальные окна. В процессе работы могут появляться диалоги подтверждения, уведомления об ошибках, запросы на сохранение данных. Если робот не ожидает такое окно, он может «зависнуть» в ожидании или выполнить действие не в том окне.
- Изменение состояния элементов. Кнопка может быть неактивна (disabled) до заполнения формы. Элемент может быть визуально присутствовать на странице, но быть скрытым или перекрытым другим элементом.
- Сбои на уровне браузера. Браузер может зависнуть, вкладка может закрыться, расширения могут конфликтовать с работой робота.

Puzzle RPA предоставляет несколько ключевых инструментов, которые позволяют строить отказоустойчивых роботов:

1. Явные ожидания (Ожидать появление элемента)

В базовом подходе часто используют фиксированные паузы (блок «Пауза»). Это плохая практика: если страница загрузилась быстрее, робот теряет время; если медленнее – робот получает ошибку. Правильный подход – использовать явные ожидания. Блок «Ожидать появление элемента» приостанавливает выполнение робота до тех пор, пока указанный элемент не появится на странице. У блока есть настраиваемый таймаут – время, в течение которого робот будет ждать. Если элемент появляется за 2 секунды, робот продолжит работу сразу; если элемент не появился за 10 секунд (таймаут по умолчанию), блок вернет значение «ложь», и вы сможете обработать эту ситуацию . Важная особенность: блок возвращает не просто факт появления, а булево значение – истина или ложь. Это позволяет строить ветвления: если элемент появился – выполняем одно действие, если нет – выполняем другое (например, перезагружаем страницу или завершаем робота с ошибкой). Блок поддерживает все типы поиска элементов, с которыми мы познакомились ранее: XPATH, CSS-селекторы, поиск по id, по классу, по имени тега, по атрибуту name. Вы можете настроить время ожидания и указать номер вкладки, на которой нужно ждать элемент .

Когда использовать явные ожидания:

- После нажатия кнопки, которая запускает загрузку данных
- Перед кликом по элементу, который может загружаться асинхронно

- Перед извлечением данных из таблицы, которая подгружается динамически
- После перехода на новую страницу или открытия новой вкладки

2. Проверка существования элемента

Иногда нужно не ждать появление элемента, а просто проверить, есть ли он на странице в данный момент. Например, перед тем как нажать кнопку «Сохранить», можно проверить, активна ли она. Или проверить, появилось ли сообщение об ошибке после отправки формы.

В Puzzle RPA есть блок «Существует ли элемент», который проверяет наличие элемента в данный момент и возвращает булево значение . Этот блок полезен для:

- Проверки, что нужная таблица загрузилась перед извлечением данных
- Проверки появления сообщения об успешной операции
- Проверки, что элемент не исчез после выполнения действия
- Построения условной логики с ветвлением

Важное отличие от «Ожидать появление элемента»: этот блок не ждет. Он проверяет элемент прямо сейчас и сразу возвращает результат. Это полезно, когда вы хотите быстро проверить состояние, не тратя время на ожидание.

3. Обработка ошибок с Try-Catch

Даже при самых лучших ожиданиях и проверках могут возникать непредвиденные ошибки. Для таких случаев нужен механизм, который перехватывает ошибку и позволяет роботу корректно на нее отреагировать, а не аварийно завершаться.

В Puzzle RPA (как и в большинстве RPA-платформ) есть конструкция Try-Catch. Эта конструкция позволяет выделить блок действий, при выполнении которых может возникнуть ошибка, и задать действия, которые нужно выполнить, если ошибка произошла .

Как работает Try-Catch:

- Внутри блока Try вы размещаете действия, которые могут вызвать ошибку. Например, клик по кнопке, которая может не загрузиться, или извлечение данных из таблицы, которой может не быть.
- Если все действия в Try выполнились успешно, робот пропускает блок Catch и идет дальше.
- Если в Try возникла ошибка, робот немедленно переходит к блоку Catch и выполняет действия, которые вы там указали. Это может быть логирование ошибки, отправка уведомления, повторная попытка выполнения или корректное завершение робота с сохранением промежуточных результатов .

Что можно добавить в Catch:

- Запись в лог информации об ошибке: какое действие не удалось, на каком элементе, в какое время
- Отправка уведомления ответственному лицу через email или мессенджер

- Повторная попытка выполнения действия (например, подождать и попробовать еще раз)
- Сохранение скриншота страницы в момент ошибки для анализа

4. Повторные попытки (Retry-механизмы)

Многие ошибки носят временный характер. Сайт может быть временно недоступен, сервер может ответить с задержкой, элемент может не загрузиться с первого раза. В таких случаях правильная стратегия – не сразу падать, а попробовать еще раз.

Повторные попытки – это частный случай использования Try-Catch. Вы помещаете действие в Try, а в Catch организуете счетчик попыток и повторяете действие, пока оно не выполнится успешно или не будет исчерпано максимальное количество попыток.

Пример стратегии повторных попыток:

- Устанавливаем счетчик попыток = 0
- Пытаемся выполнить действие
- Если действие успешно – выходим из цикла
- Если ошибка – увеличиваем счетчик, ждем (с увеличивающейся задержкой) и повторяем
- Если счетчик достиг максимума – завершаем робота с ошибкой и уведомляем администратора

Такой подход позволяет роботу самостоятельно справляться с временными сбоями без участия человека.

5. Логирование и мониторинг

Отказоустойчивый робот должен не только уметь справляться с ошибками, но и оставлять следы своей работы. Без логов невозможно понять, почему робот упал, на каком шаге произошла ошибка, какие данные были обработаны.

Что должно быть в логах:

- Время начала и окончания каждого шага
- Результат выполнения (успех или ошибка)
- При ошибке – тип ошибки и описание
- Значения ключевых переменных на момент выполнения
- URL страницы и номер вкладки
- Сделанные скриншоты при ошибках

Логирование – это не просто «хорошая практика». Это инструмент, который позволяет поддерживать робота в рабочем состоянии, быстро находить и исправлять проблемы, особенно когда роботов много и они работают в фоновом режиме.

На основе рассмотренных инструментов можно сформулировать набор рекомендаций, которые помогут создавать надежных веб-роботов.

1. Всегда используйте явные ожидания вместо фиксированных пауз. Фиксированная пауза – это всегда компромисс: либо вы ждете дольше, чем нужно (теря-

ете время), либо не ждете достаточно (получаете ошибку). Явное ожидание адаптируется к реальной скорости загрузки.

2. Проверяйте наличие элементов перед взаимодействием. Прежде чем кликнуть по кнопке или ввести текст в поле, проверьте, существует ли этот элемент и доступен ли он. Это особенно важно для динамических интерфейсов.

3. Используйте Try-Catch для критических участков. Оберните в Try-Catch те участки алгоритма, которые потенциально могут вызвать ошибку. В Catch предусмотрите логирование и корректное завершение или повторную попытку.

4. Реализуйте повторные попытки для временных сбоев. Если ошибка может быть вызвана временными проблемами (сеть, загрузка сервера), сделайте несколько попыток с увеличивающейся задержкой.

5. Проектируйте роботов с учетом поддержки. Роботы не работают по принципу «создал и забыл». Любые обновления в CRM или других веб-системах требуют проверки и, часто, доработки роботов . Закладывайте время на поддержку в план проекта.

6. Логируйте все важные шаги. Без логов вы никогда не узнаете, почему робот упал. Добавляйте информационные сообщения о начале и завершении ключевых этапов, а при ошибках – детальную информацию.

7. Используйте устойчивые селекторы. Как мы обсуждали в предыдущих разделах, XPATH с contains и starts-with делает робота устойчивым к изменениям интерфейса. Это первая линия защиты от сбоев.

Отказоустойчивость – это не одна функция, а комплекс подходов и инструментов, которые делают робота готовым к реальным условиям работы.

Продвинутые блоки веб-автоматизации в Puzzle RPA

В девятой практической работе мы освоили базовые блоки веб-автоматизации: создание сессии браузера, открытие страниц, клики, ввод текста, работу с вкладками и скачивание файлов. Это необходимый фундамент, который позволяет роботу выполнять простые линейные сценарии.

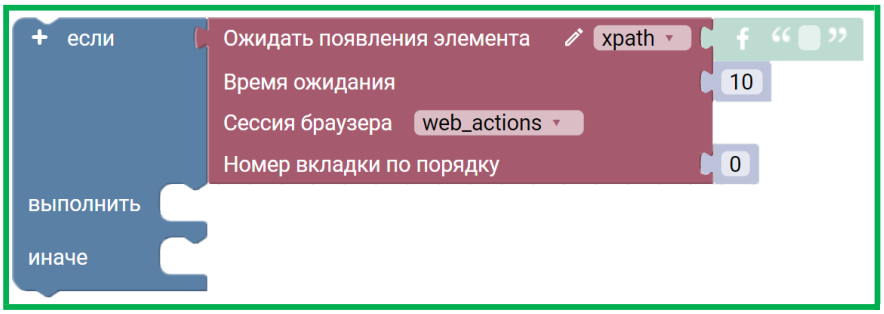
Однако реальные бизнес-задачи редко ограничиваются такими простыми сценариями. На практике робот сталкивается с динамическим контентом, нестабильной загрузкой страниц, необходимостью обрабатывать коллекции элементов и извлекать структурированные данные. Для решения этих задач требуются более сложные инструменты и подходы.

Сейчас мы переходим к изучению продвинутых блоков веб-автоматизации в Puzzle RPA. Цель этой практической работы – освоить инструменты для создания устойчивых и масштабируемых роботов.

В результате освоения блоков вы сможете создавать не просто роботов, работающих в идеальных условиях, а промышленные RPA-решения, устойчивые к изменениям интерфейса, динамическому контенту и нестабильной загрузке страниц.

Приступаем к изучению продвинутых блоков веб-автоматизации в Puzzle RPA.

Блок «Ожидать появление элемента»



Блок ожидает появления указанного элемента в сессии браузера. Он приостанавливает выполнение сценария до тех пор, пока элемент не станет доступным на странице, либо до истечения заданного времени ожидания.

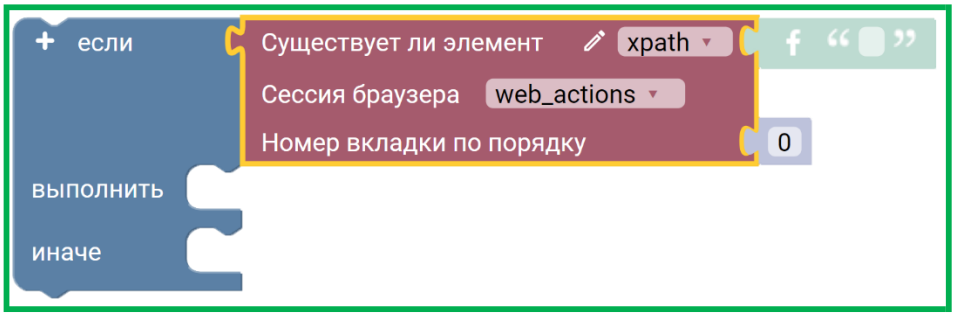
Параметры блока

- Параметры подключения – выбор способа идентификации и поиска веб-элемента на странице. Доступны следующие опции:
  - ✓ `xpath` – поиск по XPATH
  - ✓ `css-selector` – поиск по CSS-селектору
  - ✓ `tag_name` – поиск по тегу элемента
  - ✓ `name` – поиск по атрибуту `name`
  - ✓ `id` – поиск по атрибуту `id`
  - ✓ `class` – поиск по атрибуту `class`
- Время ожидания – максимальное время ожидания появления элемента в секундах.
- Сессия браузера – экземпляр запущенного браузера, управляемый через Selenium WebDriver или pyDoll. В переменной хранится ссылка на этот экземпляр, через которую можно управлять браузером.
- Номер вкладки по порядку – порядковый номер вкладки, в которой необходимо выполнить действие.

Здесь вы также можете использовать Мастер UI для веб-автоматизации, который мы уже изучили. При нажатии на кнопку редактирования рядом с параметрами поиска открывается этот инструмент.

Блок «Ждать появление элемента» незаменим при работе с динамическими веб-страницами, где элементы могут загружаться с задержкой. Он позволяет синхронизировать выполнение сценария с фактическим состоянием страницы, исключая ошибки, связанные с попыткой взаимодействия с ещё не загруженными элементами. Благодаря гибкому выбору способа идентификации и интеграции с Мастером UI вы можете быстро и точно настроить ожидание нужного элемента, делая автоматизацию более стабильной и надежной.

Блок «Существует ли элемент»



Блок проверяет наличие элемента на странице браузера. В отличие от блока «Ожидать появление элемента», данный блок не ожидает загрузки, а мгновенно проверяет, присутствует ли элемент в текущем состоянии страницы. Результатом выполнения является логическое значение (True или False).

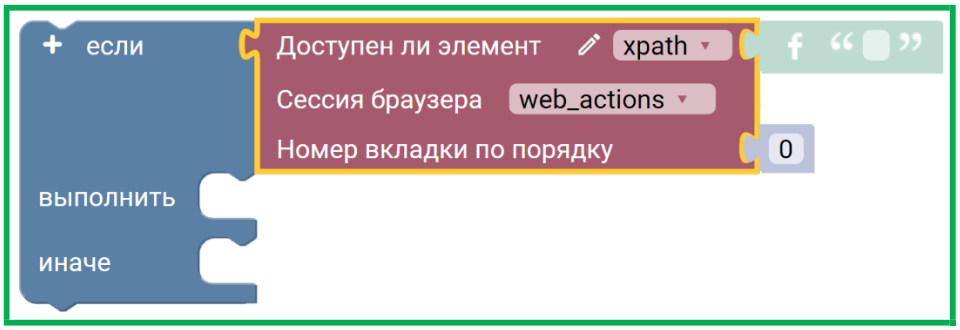
Параметры блока

- Параметры подключения – выбор способа идентификации и поиска веб-элемента на странице. Доступны следующие опции:
  - ✓ `xpath` – поиск по XPATH
  - ✓ `css-selector` – поиск по CSS-селектору
  - ✓ `tag_name` – поиск по тегу элемента
  - ✓ `name` – поиск по атрибуту name
  - ✓ `id` – поиск по атрибуту id
  - ✓ `class` – поиск по атрибуту class
- Сессия браузера – экземпляр запущенного браузера, управляемый через Selenium WebDriver или pyDoll. В переменной хранится ссылка на этот экземпляр, через которую можно управлять браузером.
- Номер вкладки по порядку – порядковый номер вкладки, в которой необходимо выполнить действие.

Здесь вы также можете использовать Мастер UI для веб-автоматизации, который мы уже изучили. При нажатии на кнопку редактирования рядом с параметрами поиска открывается этот инструмент.

Блок «Существует ли элемент» предназначен для быстрой проверки наличия элемента на странице без ожидания его загрузки. Это особенно полезно для реализации условной логики в сценарии: например, для проверки появления модального окна, отображения ошибки или наличия определенного контента. В сочетании с Мастером UI вы можете легко и точно указать нужный элемент, делая проверки наглядными и надежными. Блок позволяет строить гибкие и адаптивные сценарии автоматизации, способные реагировать на различные состояния веб-страницы.

Блок «Доступен ли элемент»



Блок проверяет, включен (доступен) или отключен элемент на веб-странице. Результатом выполнения является логическое значение (True или False). Это позволяет определить, может ли пользователь взаимодействовать с элементом – например, нажать на кнопку, ввести текст в поле или выбрать значение из списка.

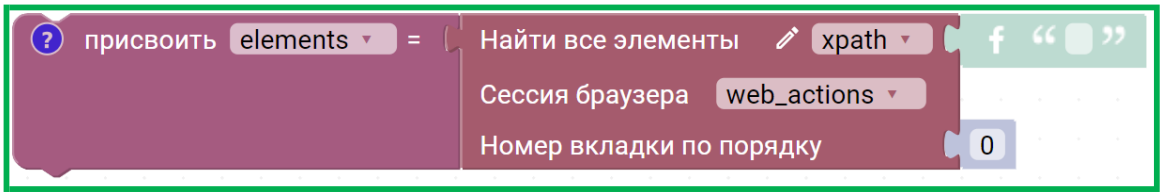
Параметры блока:

- Параметры подключения – выбор способа идентификации и поиска веб-элемента на странице. Доступны следующие опции:
  - ✓ `xpath` – поиск по XPATH
  - ✓ `css-selector` – поиск по CSS-селектору
  - ✓ `tag_name` – поиск по тегу элемента
  - ✓ `name` – поиск по атрибуту `name`
  - ✓ `id` – поиск по атрибуту `id`
  - ✓ `class` – поиск по атрибуту `class`
- Сессия браузера – экземпляр запущенного браузера, управляемый через Selenium WebDriver или pyDoll. В переменной хранится ссылка на этот экземпляр, через которую можно управлять браузером.
- Номер вкладки по порядку – порядковый номер вкладки, в которой необходимо выполнить действие.

Здесь вы также можете использовать Мастер UI для веб-автоматизации, который мы уже изучили. При нажатии на кнопку редактирования рядом с параметрами поиска открывается этот инструмент.

Блок «Доступен ли элемент» позволяет проверять состояние интерактивных элементов веб-страницы, таких как кнопки, поля ввода, чекбоксы и выпадающие списки. Это особенно важно при автоматизации сценариев, где взаимодействие с элементом возможно только после выполнения определенных условий (например, заполнения формы или загрузки данных). Блок помогает избежать ошибок, связанных с попыткой нажать на отключенную кнопку или ввести текст в заблокированное поле. В сочетании с Мастером UI вы можете быстро и точно указать нужный элемент, делая проверки его доступности наглядными и надежными.

Блок «Найти все элементы»



Блок «Найти все элементы» предназначен для поиска и получения списка всех веб-элементов на странице, которые соответствуют заданным условиям. В отличие от базовых блоков, которые находят только первый подходящий элемент, этот блок возвращает коллекцию всех элементов, что позволяет обрабатывать множественные объекты – строки таблиц, карточки товаров, элементы списков.

Блок возвращает список объектов найденных веб-элементов. Каждый элемент в этом списке представляет собой отдельный объект, с которым можно взаимодействовать: кликать, считывать текст, получать атрибуты. Если ни один элемент не найден, блок возвращает пустой список, что позволяет роботу корректно обработать ситуацию отсутствия данных.

Основные параметры

- Параметры подключения – определяет способ идентификации и поиска веб-элементов на странице. При нажатии на кнопку раскрывающегося списка открывается меню с шестью вариантами поиска:
  - ✓ `xpath` – поиск элементов с помощью XPATH-запроса. Самый гибкий способ, позволяющий искать по тексту, части атрибутов, расположению относительно других элементов.
  - ✓ `css-selector` – поиск с помощью CSS-селектора. Более простой и быстрый способ, подходящий для большинства задач, где не требуется поиск по тексту.
  - ✓ `tag_name` – поиск по имени HTML-тега (например, `button`, `input`, `div`, `table`). Используется, когда нужно найти все элементы определенного типа.
  - ✓ `name` – поиск по атрибуту `name` элемента. Часто применяется для полей ввода в формах.
  - ✓ `id` – поиск по уникальному идентификатору элемента. Самый быстрый способ, но подходит только если у элементов есть стабильные `id`.
  - ✓ `class` – поиск по атрибуту `class`. Позволяет найти все элементы с определенным классом.
- Сессия браузера – экземпляр запущенного браузера, управляемый через Selenium WebDriver. В переменной хранится ссылка на этот экземпляр, через которую блок взаимодействует с браузером. Тип данных – переменная. Пример: web_actions.
- Номер вкладки по порядку – порядковый номер вкладки браузера, в которой необходимо выполнить поиск. Нумерация начинается с нуля. Если у вас открыто несколько вкладок, вы можете указать нужную. Тип данных – число. Пример: 0 (первая вкладка).

Здесь вы также можете использовать Мастер UI для веб-автоматизации, который мы уже изучили.

При нажатии на кнопку редактирования рядом с параметрами поиска открывается этот инструмент. С его помощью вы можете:

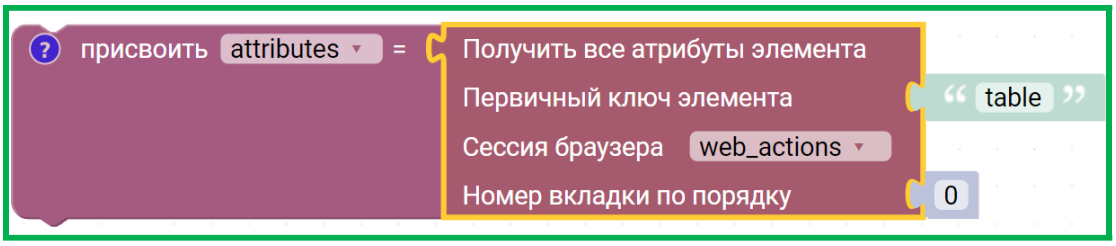
- Визуально выбрать элементы на открытой веб-странице
- Автоматически сгенерировать правильный запрос (XPATH или CSS-селектор)

- Проверить, сколько элементов найдено по текущему запросу
- Уточнить условия поиска, добавляя или изменяя атрибуты

Блок «Найти все элементы» – это ключевой инструмент для работы с коллекциями элементов на веб-странице. Он возвращает список всех элементов, соответствующих условиям поиска, что позволяет обрабатывать множественные объекты в циклах.

Умение работать с коллекциями элементов превращает робота из простого исполнителя единичных действий в инструмент для массовой обработки данных – от списков клиентов до каталогов товаров.

Блок «Получить все атрибуты элемента»



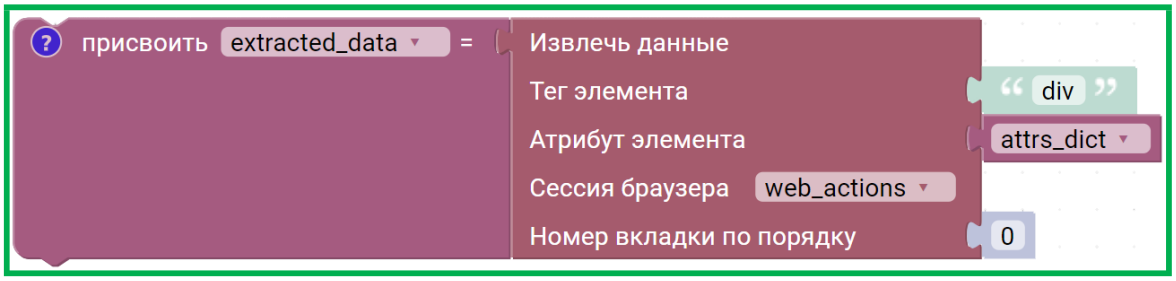
Блок извлекает все атрибуты элемента с веб-страницы. Требуется указать основной атрибут элемента (table, span, div и т.д.). Результатом будет словарь с элементами (ключами и значениями) найденных атрибутов.

Параметры блока

- Первичный ключ элемента – тег веб-элемента на HTML-странице браузера.
- Сессия браузера – экземпляр запущенного браузера, управляемый через Selenium WebDriver или pyDoll. В переменной хранится ссылка на этот экземпляр, через которую можно управлять браузером.
- Номер вкладки по порядку – порядковый номер вкладки, в которой необходимо выполнить действие.

Блок «Получить все атрибуты элемента» позволяет быстро и удобно собрать все атрибуты выбранного веб-элемента в виде словаря. Благодаря интеграции с Мастером UI для веб-автоматизации вы можете визуально указать нужный элемент на странице, не тратя время на ручное написание селекторов. Это упрощает настройку блока, снижает риск ошибок и делает процесс автоматизации более наглядным, особенно при работе со сложными или динамическими страницами.

Блок «Извлечь данные»



Блок осуществляет извлечение данных с веб-страницы. Позволяет получить текст, атрибуты или другую информацию указанного элемента. Результатом выполнения является извлеченное значение, которое можно использовать в дальнейших шагах сценария.

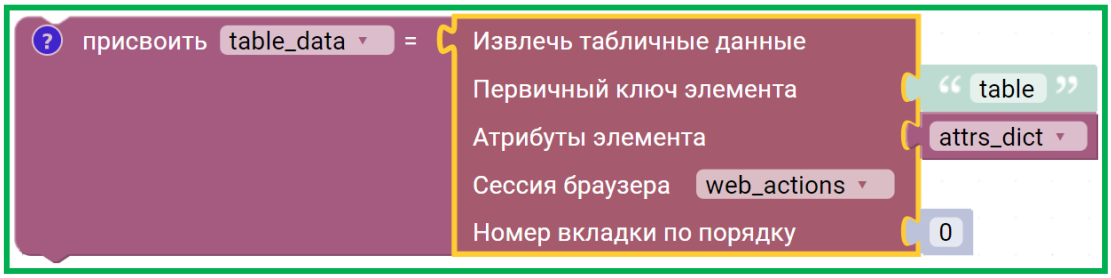
Параметры блока

- Тег элемента – тег веб-элемента на HTML-странице браузера, из которого необходимо извлечь данные.
- Атрибуты элемента – словарь атрибутов, уточняющих поиск элемента. Может включать такие атрибуты, как `class`, `id`, `name` и другие.
- Сессия браузера – экземпляр запущенного браузера, управляемый через Selenium WebDriver или pyDoll. В переменной хранится ссылка на этот экземпляр, через которую можно управлять браузером.
- Номер вкладки по порядку – порядковый номер вкладки, в которой необходимо выполнить действие.

Здесь вы также можете использовать Мастер UI для веб-автоматизации, который мы уже изучили.

Блок «Извлечь данные» является универсальным инструментом для получения информации с веб-страниц. Он позволяет извлекать текст, значения полей, атрибуты и другие данные из любого элемента. Это базовый блок для сбора информации в процессе автоматизации – от извлечения заголовков и описаний до получения значений из полей ввода и атрибутов элементов. Благодаря интеграции с Мастером UI вы можете быстро и точно указать нужный элемент, делая процесс извлечения данных наглядным и надежным, даже при работе со сложными динамическими страницами.

Блок «Извлечь табличные данные»



Блок извлекает табличные данные с веб-страницы. Требуется указать основной тег элемента (`table`, `div` и т.д.) и атрибуты элемента в виде словаря. Результатом будет список с извлеченными данными, что позволяет удобно обрабатывать и использовать табличную информацию в дальнейших шагах сценария.

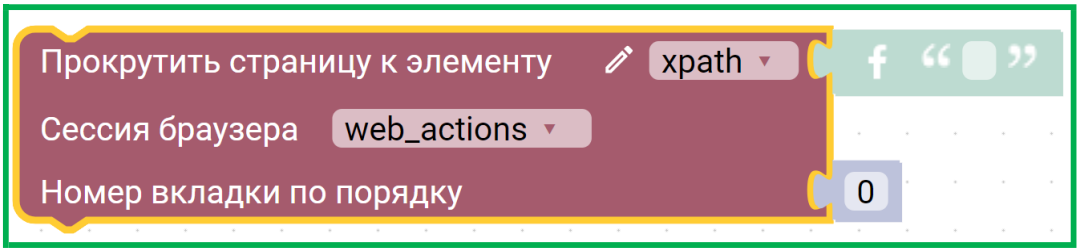
Параметры блока

- Первичный ключ элемента – тег веб-элемента на HTML-странице браузера, содержащего табличные данные.
- Атрибуты элемента – словарь атрибутов, по которым уточняется поиск таблицы. Тип данных: dict (Словарь)
- Сессия браузера – экземпляр запущенного браузера, управляемый через Selenium WebDriver или pyDoll. В переменной хранится ссылка на этот экземпляр, через которую можно управлять браузером.
- Номер вкладки по порядку – порядковый номер вкладки, в которой необходимо выполнить действие.

Здесь вы также можете использовать Мастер UI для веб-автоматизации, который мы уже изучили.

Блок «Извлечь табличные данные» предназначен для сбора структурированной информации с веб-страниц, таких как прайс-листы, отчеты, списки записей и другие табличные данные. Он автоматически преобразует HTML-таблицу в удобный для обработки формат списка, что позволяет легко интегрировать извлеченные данные в дальнейшую логику сценария – например, для анализа, записи в файл или передачи в другие системы. Благодаря интеграции с Мастером UI вы можете быстро и точно указать нужную таблицу, избегая ошибок при ручном составлении селекторов, особенно на сложных и динамических страницах.

Блок «Прокрутить страницу»



Блок позволяет прокрутить страницу (выполнить скроллинг) до указанного элемента, чтобы он оказался в зоне видимости. Это необходимо для взаимодействия с элементами, которые находятся за пределами текущей области просмотра и недоступны для нажатия или извлечения данных.

Параметры блока

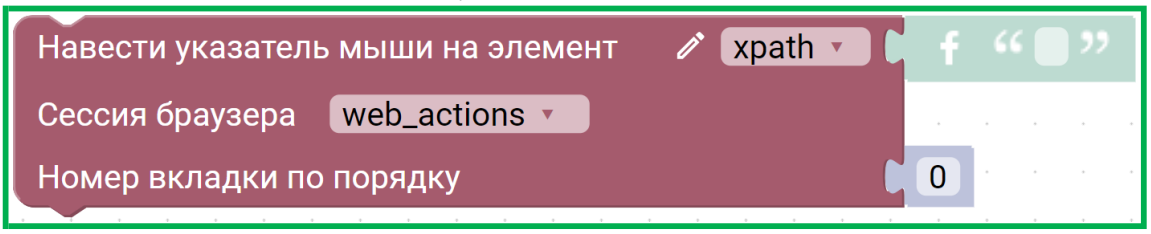
- Параметры подключения – выбор способа идентификации и поиска веб-элемента на странице, до которого необходимо прокрутить. Доступны следующие опции:
  - ✓ `xpath` – поиск по XPATH
  - ✓ `css-selector` – поиск по CSS-селектору
  - ✓ `tag_name` – поиск по тегу элемента
  - ✓ `name` – поиск по атрибуту name
  - ✓ `id` – поиск по атрибуту id
  - ✓ `class` – поиск по атрибуту class
- Сессия браузера – экземпляр запущенного браузера, управляемый через Selenium WebDriver или pyDoll. В переменной хранится ссылка на этот экземпляр, через которую можно управлять браузером.
- Номер вкладки по порядку – порядковый номер вкладки, в которой необходимо выполнить действие.

Здесь вы также можете использовать Мастер UI для веб-автоматизации, который мы уже изучили.

Блок «Прокрутить страницу» обеспечивает корректную работу с элементами, которые не видны в текущей области просмотра браузера. Он позволяет программно выполнить скроллинг до нужного элемента, делая его доступным для последующих действий – клика, ввода данных или извлечения информации. Это особенно важно при работе с длинными страницами, бесконечной лентой (infinite

scroll), модальными окнами или элементами, которые загружаются динамически при прокрутке. Благодаря интеграции с Мастером UI вы можете легко указать целевой элемент, избегая ошибок при ручном составлении селекторов и делая автоматизацию более надежной.

Блок «Навести указатель мыши на элемент»



Блок наводит указатель мыши на указанный элемент (выполняет действие hover). Это позволяет имитировать взаимодействие пользователя с элементами, которые реагируют на наведение курсора – например, отображение всплывающих подсказок, выпадающих меню или изменение стилей.

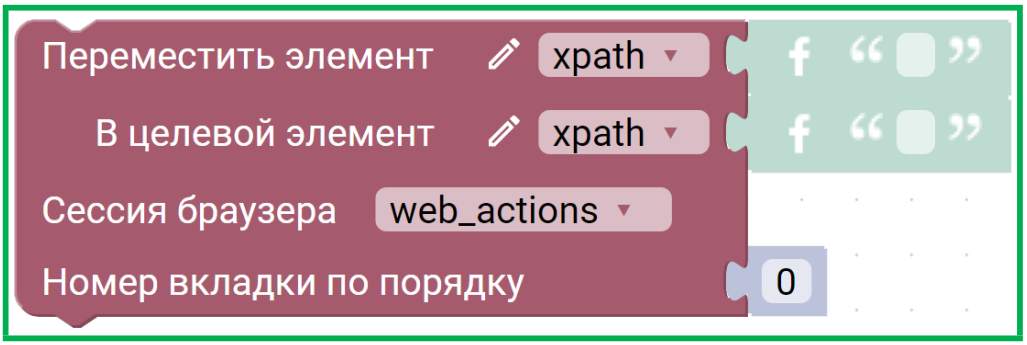
Параметры блока

- Параметры подключения – выбор способа идентификации и поиска веб-элемента на странице, на который необходимо навести указатель. Доступны следующие опции:
  - ✓ `xpath` – поиск по XPATH
  - ✓ `css-selector` – поиск по CSS-селектору
  - ✓ `tag_name` – поиск по тегу элемента
  - ✓ `name` – поиск по атрибуту `name`
  - ✓ `id` – поиск по атрибуту `id`
  - ✓ `class` – поиск по атрибуту `class`
- Сессия браузера – экземпляр запущенного браузера, управляемый через Selenium WebDriver или pyDoll. В переменной хранится ссылка на этот экземпляр, через которую можно управлять браузером.
- Номер вкладки по порядку – порядковый номер вкладки, в которой необходимо выполнить действие.

Здесь вы также можете использовать Мастер UI для веб-автоматизации, который мы уже изучили.

Блок «Навести указатель мыши на элемент» позволяет эмулировать одно из ключевых пользовательских взаимодействий – наведение курсора. Это незаменимый инструмент для работы с современными веб-интерфейсами, где многие элементы становятся доступны только после наведения (например, всплывающие меню, тултипы, дополнительные кнопки действий). Блок обеспечивает корректную активацию таких элементов, позволяя выполнять с ними последующие действия – клик, извлечение данных или проверку состояния. Благодаря интеграции с Мастером UI вы можете быстро и точно указать целевой элемент, делая автоматизацию сложных сценариев с наведением простой и надежной.

Блок «Переместить элемент»



Блок выполняет операцию drag-and-drop (перетаскивание) – сначала щелкает и удерживает исходный элемент, затем перемещает курсор к местоположению целевого элемента и отпускает мышь. Это позволяет программно имитировать перемещение объектов на веб-странице.

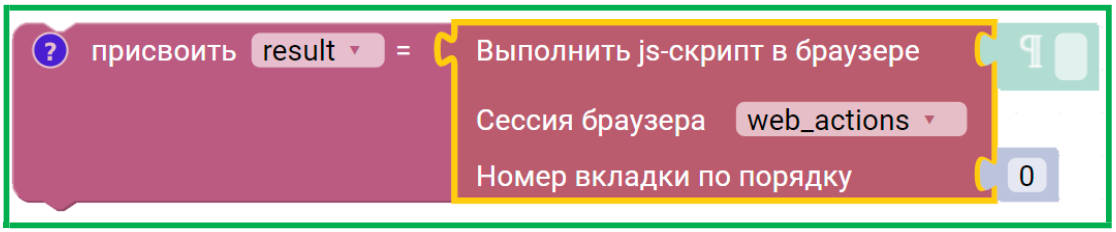
Параметры блока:

- Параметры подключения (Перемещаемый элемент) – выбор способа идентификации и поиска элемента, который необходимо перетащить. Доступны следующие опции:
  - ✓ `xpath` – поиск по XPATH
  - ✓ `css-selector` – поиск по CSS-селектору
  - ✓ `tag_name` – поиск по тегу элемента
  - ✓ `name` – поиск по атрибуту `name`
  - ✓ `id` – поиск по атрибуту `id`
  - ✓ `class` – поиск по атрибуту `class`
- Параметры подключения (Целевой элемент) – выбор способа идентификации и поиска элемента, на который необходимо перетащить исходный элемент. Доступны те же опции идентификации.
- Сессия браузера – экземпляр запущенного браузера, управляемый через Selenium WebDriver или pyDoll. В переменной хранится ссылка на этот экземпляр, через которую можно управлять браузером.
- Номер вкладки по порядку – порядковый номер вкладки, в которой необходимо выполнить действие.

Здесь вы также можете использовать Мастер UI для веб-автоматизации, который мы уже изучили.

Блок «Переместить элемент» обеспечивает автоматизацию сложного взаимодействия drag-and-drop, которое часто используется в современных веб-приложениях – например, в конструкторах, файловых менеджерах, календарях, интерфейсах с сортируемыми списками и канбан-досках. Блок позволяет точно имитировать действия пользователя по перетаскиванию объектов, сохраняя порядок операций: захват, перемещение и отпускание. Благодаря интеграции с Мастером UI вы можете наглядно указать оба элемента – исходный и целевой – что значительно упрощает настройку блока и снижает риск ошибок при работе со сложными интерфейсами.

Блок «Выполнить js-скрипт в браузере»



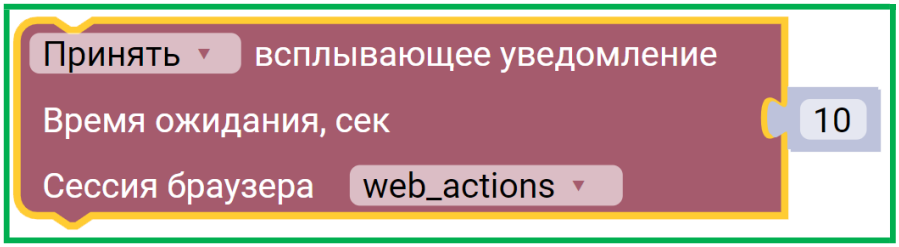
Блок позволяет выполнить произвольный JavaScript-код в контексте текущей страницы браузера. Это дает возможность расширить функциональность автоматизации, выполняя действия, которые могут быть сложны или невозможны с использованием стандартных блоков.

Параметры блока:

- Выполнить js-script в браузере – произвольный JavaScript-код, который необходимо выполнить на странице.
- Сессия браузера – экземпляр запущенного браузера, управляемый через Selenium WebDriver или pyDoll. В переменной хранится ссылка на этот экземпляр, через которую можно управлять браузером.
- Номер вкладки по порядку – порядковый номер вкладки, в которой необходимо выполнить действие.

Блок «Выполнить js-скрипт в браузере» предоставляет максимальную гибкость при автоматизации веб-интерфейсов. Он позволяет решать задачи, выходящие за рамки стандартных блоков: манипуляция DOM-элементами, изменение стилей, работа с атрибутами, выполнение сложных вычислений на стороне клиента, взаимодействие с пользовательскими скриптами и библиотеками, а также обход ограничений, возникающих при использовании только Selenium-команд. Это мощный инструмент для продвинутых сценариев автоматизации, где требуется тонкая настройка поведения страницы. При этом важно помнить, что использование JavaScript требует аккуратности, так как некорректный код может нарушить работу страницы или привести к неожиданным результатам.

Блок «Всплывающее уведомление»



Блок позволяет принять или отклонить всплывающее уведомление (alert, confirm, prompt) на странице браузера. Это необходимо для обработки стандартных диалоговых окон JavaScript, которые требуют взаимодействия пользователя для продолжения работы.

Параметры блока:

- Принять/Отклонить всплывающее уведомление – выпадающий список для выбора действия с уведомлением. Доступны следующие опции:

✓ Принять – подтвердить уведомление (нажать OK)

- ✓ Отклонить – отклонить уведомление (нажать Cancel)

- Время ожидания – максимальное время ожидания появления уведомления в секундах.
- Сессия браузера – экземпляр запущенного браузера, управляемый через Selenium WebDriver или pyDoll. В переменной хранится ссылка на этот экземпляр, через которую можно управлять браузером.

Особенность работы PyDoll

При взаимодействии с веб-браузером через протокол PyDoll необходимо размещать блок обработки всплывающего уведомления до шага, инициирующего появление этого уведомления.

Для корректной обработки всплывающего уведомления блок «Принять/отклонить всплывающее уведомление» должен быть выполнен заблаговременно (например, перед открытием страницы). При фактическом появлении уведомления будет автоматически выбран вариант, указанный в этом блоке.

Параметр «Время ожидания» при этом не будет применен. Указанное действие будет применяться на протяжении всей работы алгоритма программного робота.

Блок «Всплывающее уведомление» обеспечивает корректную обработку стандартных JavaScript-диалоговых окон, которые часто используются на веб-страницах для подтверждения действий, ввода данных или вывода предупреждений. Без обработки таких уведомлений выполнение сценария может остановиться или завершиться ошибкой. Блок позволяет гибко настроить действие (принять или отклонить) и время ожидания. Важно учитывать особенность работы с PyDoll: блок должен быть установлен до момента возникновения уведомления, что требует продуманного проектирования последовательности шагов в сценарии автоматизации.

Мы рассмотрели группу блоков, которые составляют основу для построения сложных и гибких сценариев веб-автоматизации. Каждый из них решает определенный круг задач, а вместе они позволяют автоматизировать взаимодействие с веб-страницами на профессиональном уровне.

Общие преимущества:

- Интеграция с Мастером UI – во всех блоках, где требуется указание элемента, доступен Мастер UI для веб-автоматизации. Это позволяет визуально выбирать элементы на странице, автоматически генерировать корректные XPATH или CSS-селекторы и проверять результат, что значительно упрощает настройку и снижает количество ошибок.
- Гибкость идентификации – каждый блок поддерживает различные способы поиска элементов (xpath, css-selector, id, class, name, tag_name), что позволяет выбирать наиболее удобный и надежный вариант в зависимости от структуры страницы.

- Управление вкладками – возможность указывать номер вкладки по порядку обеспечивает корректную работу в многоконтекстных сценариях, когда автоматизация взаимодействует с несколькими открытыми страницами.
- Обработка динамического контента – блоки ожидания, проверки доступности и прокрутки страницы позволяют строить устойчивые сценарии, которые корректно работают даже на сложных современных веб-сайтах с активным использованием JavaScript и асинхронной загрузкой.

Рекомендации по использованию:

- Для надежности сценариев всегда используйте блоки ожидания перед взаимодействием с динамически загружаемыми элементами.
- При работе с формами и интерактивными элементами предварительно проверяйте их доступность.
- Используйте Мастер UI для сложных селекторов – это ускоряет разработку и минимизирует ошибки.
- При работе с PyDoll учитывайте особенности обработки всплывающих уведомлений – блок должен быть размещен до момента их возникновения.
- Для нестандартных сценариев используйте выполнение произвольного JavaScript-кода, но с осторожностью, чтобы не нарушить работу страницы.

В совокупности представленные блоки формируют полноценный инструментарий для автоматизации самых разнообразных веб-интерфейсов – от простого извлечения данных до сложного взаимодействия с динамическими элементами, таблицами, drag-and-drop интерфейсами и диалоговыми окнами. Освоение этих блоков позволяет разработчику создавать устойчивые, гибкие и масштабируемые сценарии RPA для работы с веб-приложениями любой сложности.

Подведение итогов практической работы

Мы завершили изучение продвинутого курса по веб-автоматизации в Puzzle RPA. За время освоения этого раздела мы углубили свои знания и освоили сложные инструменты, которые позволяют создавать надёжные, отказоустойчивые и эффективные сценарии для работы с веб-интерфейсами любой сложности.

Мы освоили визуальный редактор XPath, который стал нашим главным инструментом для создания точных запросов к элементам страницы. Мы научились строить сложные XPath-выражения с использованием осей (ancestor, following-sibling, parent), логических операторов (and, or), функций (contains, starts-with, text-is) и группировки условий с помощью скобок.

Особую ценность представляет возможность использования переменных в XPath-запросах – как в качестве индекса узла, так и в качестве значения атрибута или даже целого узла. Это позволяет создавать динамические селекторы, которые адаптируются к изменяющимся условиям выполнения сценария.

Мы расширили набор способов поиска элементов за пределы стандартных селекторов. Научились комбинировать различные методы идентификации, использовать относительные пути, строить запросы на основе текстового содержимого элементов и их атрибутов. Особое внимание уделили поиску элементов в

сложных вложенных структурах, где прямые пути ненадёжны из-за динамически генерируемых идентификаторов или частых обновлений вёрстки.

Мы освоили управление многокладочными сценариями. Научились открывать новые вкладки, переключаться между ними по порядковому номеру, закрывать ненужные вкладки и контролировать, какая вкладка в данный момент активна. Понимание принципов нумерации вкладок (начиная с индекса 0) и динамического изменения номеров при закрытии позволяет строить корректные сценарии даже при сложной навигации.

Мы изучили блок «Получить табличные данные», который позволяет автоматически извлекать структурированную информацию из HTML-таблиц. Научились настраивать параметры извлечения, обрабатывать объединённые ячейки, сохранять данные в переменные для последующей обработки или экспорта. Это значительно расширяет возможности парсинга веб-страниц и автоматизации сбора отчётности.

Критически важной темой стало построение надёжных сценариев, устойчивых к сбоям. Мы освоили стратегии обработки исключительных ситуаций: ожидание появления элементов (явные и неявные ожидания), проверку свойств элементов (IsEnabled, Visibility), обработку таймаутов и повторные попытки выполнения действий. Научились проектировать сценарии с учётом возможных задержек загрузки, динамического появления контента и непредвиденных изменений на странице.

Мы детально разобрали расширенный набор блоков для веб-автоматизации.. Изучили работу с горячими клавишами, выпадающими списками, а также продвинутые настройки ввода текста с имитацией задержек между нажатиями.

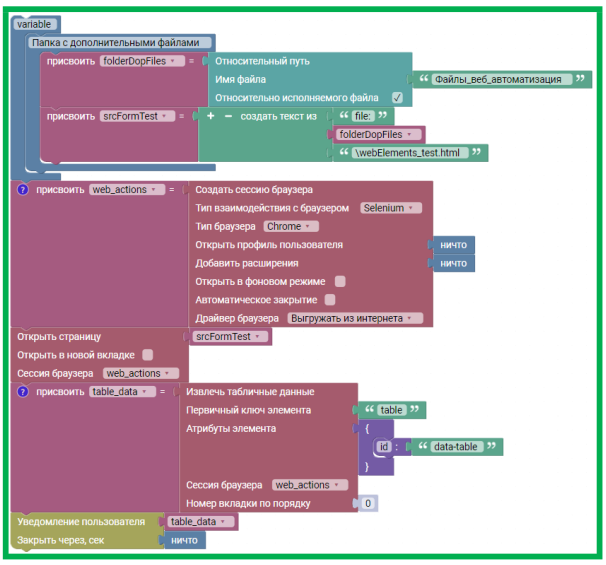
В ходе практической работы мы сформировали целостную систему знаний и навыков, необходимых для профессиональной веб-автоматизации. От простых сценариев кликов и ввода текста мы перешли к сложным, отказоустойчивым решениям, способным работать с динамическими интерфейсами, многокладочными сценариями, табличными данными и защитными механизмами в виде капч.

Главный результат – мы научились не просто использовать готовые блоки, но и понимать внутреннюю логику работы веб-приложений, что позволяет прогнозировать поведение элементов, выбирать оптимальные стратегии поиска и выстраивать архитектуру сценария, устойчивую к внешним изменениям. Полученные компетенции являются фундаментом для реализации проектов любой сложности в области автоматизации веб-систем.

ПРИМЕРЫ ЗАДАЧ

Пример 1 – Извлечь табличные данные

В данном примере происходит проверка на наличие веб-элемента на html странице:



Алгоритм ожидания веб-элемента:

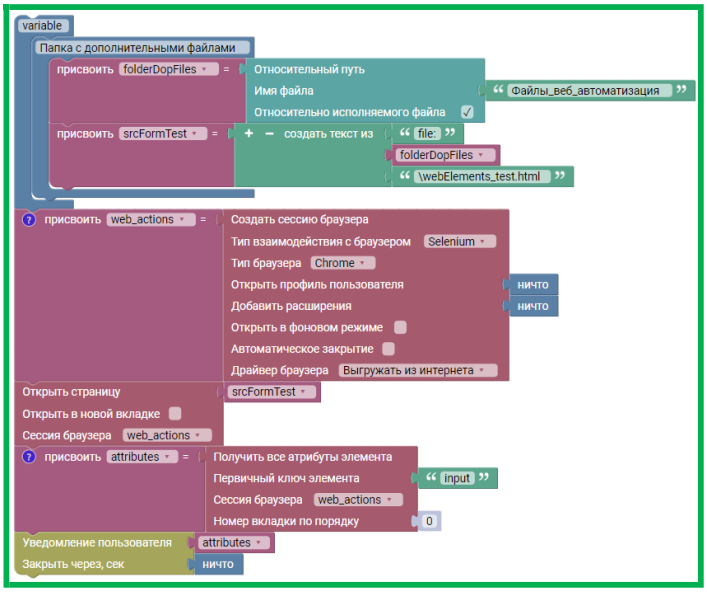
- Открывается html страница из папки **resources** проекта;
- Извлекаются данные веб-элемента по тегу h1 и классу header-class;

Вывод результата на экран:



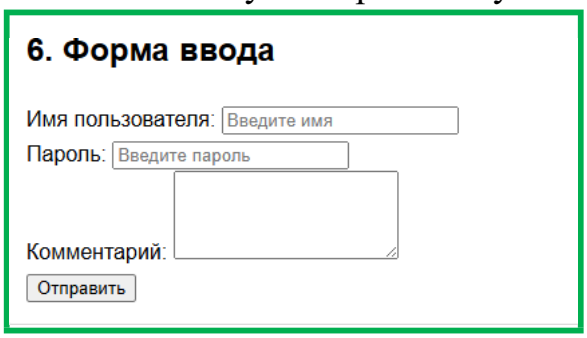
Пример 2 – Получить все атрибуты элемента

В данном примере происходит проверка на наличие веб-элемента на html странице:

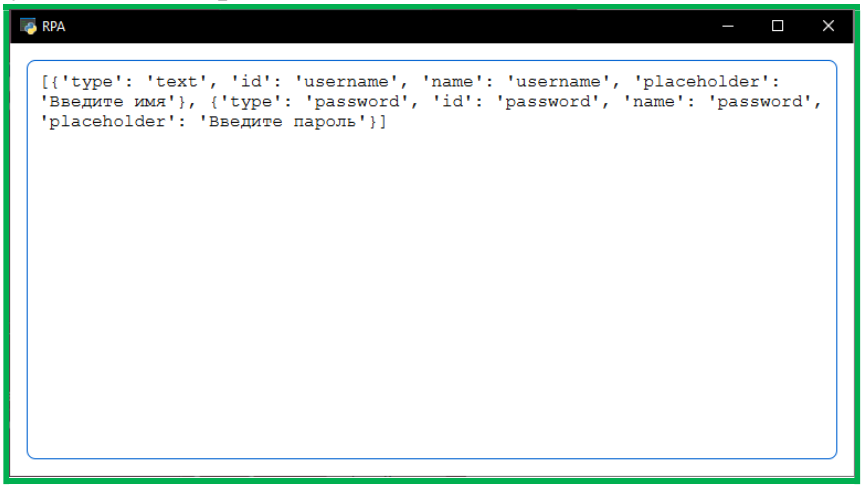


Алгоритм ожидания веб-элемента:

- Открывается html страница из папки **resources** проекта;
- Извлекаются данные веб-элементу по первичному ключу input;

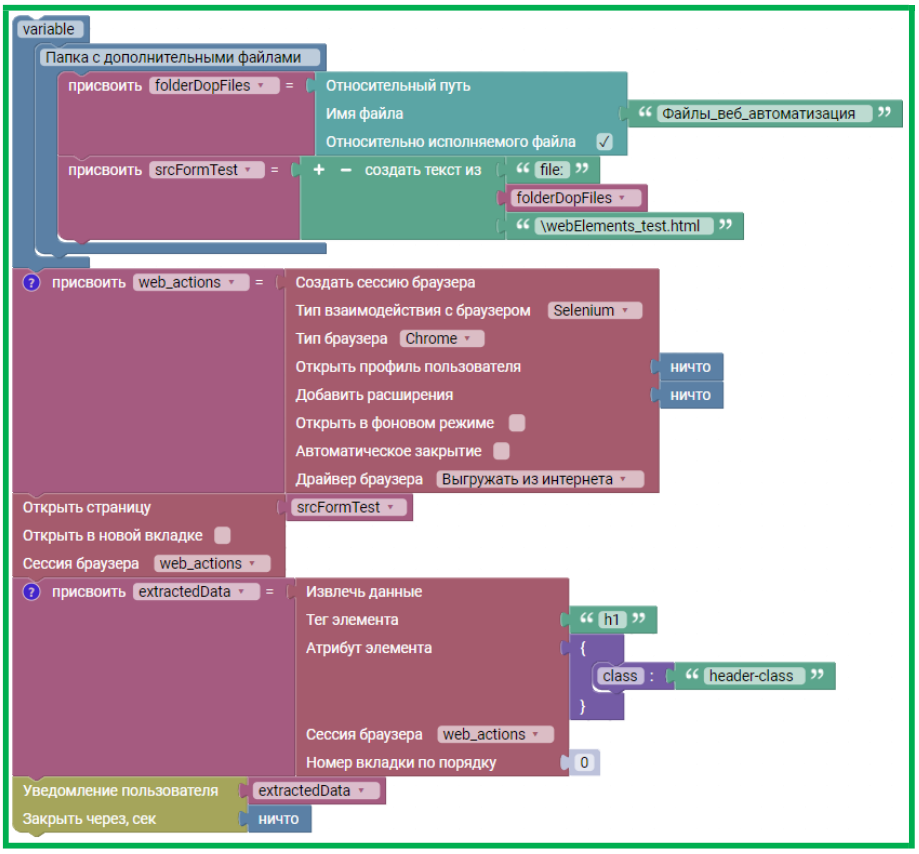


Вывод результата на экран:



Пример 3 – Извлечь данные

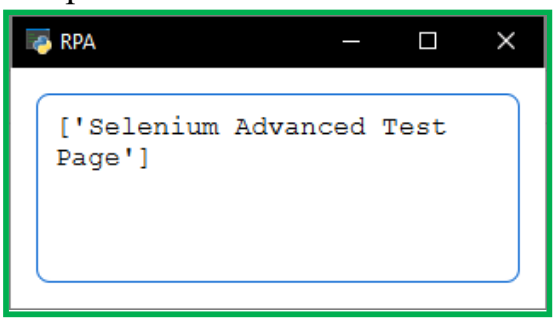
В данном примере происходит проверка на наличие веб-элемента на html странице:



Алгоритм ожидания веб-элемента:

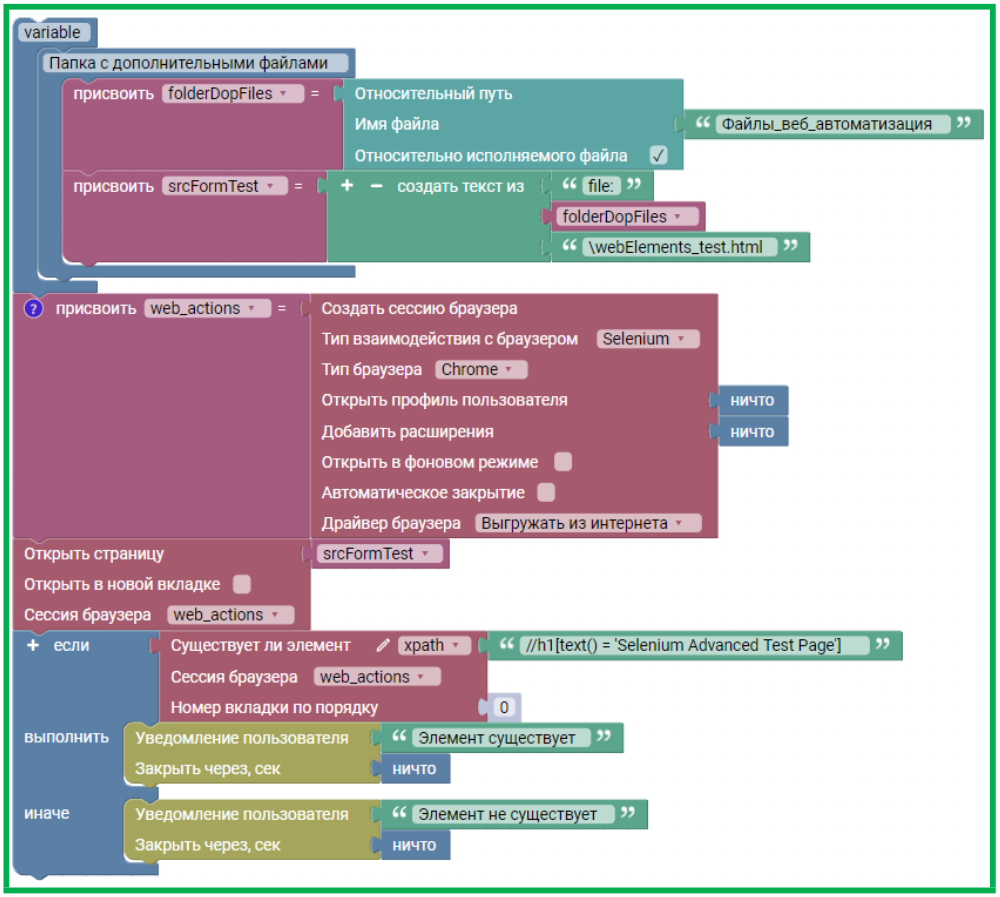
- Открывается html страница из папки **resources** проекта;
- Извлекаются данные веб-элементу по тегу h1 и классу header-class;

Вывод результата на экран:



Пример 4 – Существует ли элемент

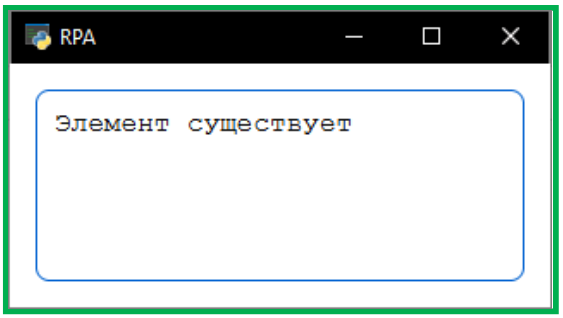
В данном примере происходит проверка на наличие веб-элемента на html странице:



Алгоритм ожидания веб-элемента:

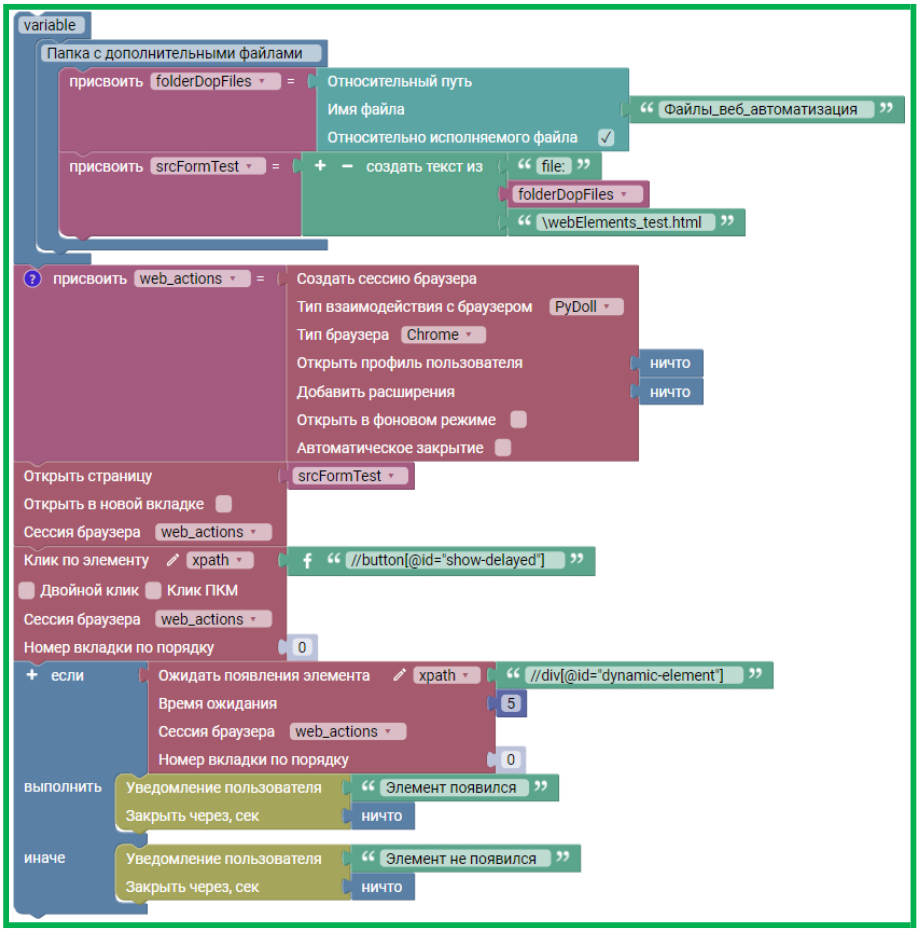
- Открывается html страница из папки **resources** проекта;
- Проверка на наличие указанного веб-элемента `//h1[text() = 'Selenium Advanced Test Page']`;
- Уведомление о результате наличия элемента;

Вывод результата на экран:

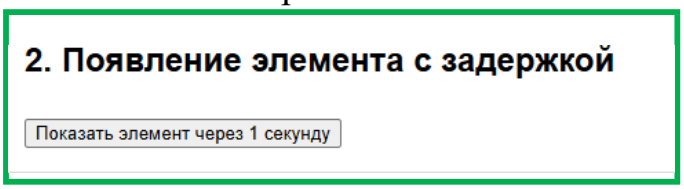


Пример 5 –Ожидать появление элемента

В данном примере происходит ожидание повления веб-элемента после клика по кнопке на html странице:



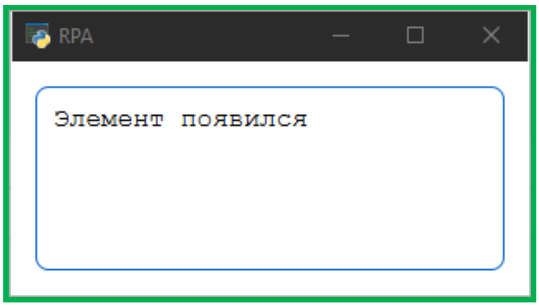
Веб-элемент до выполнения алгоритма:



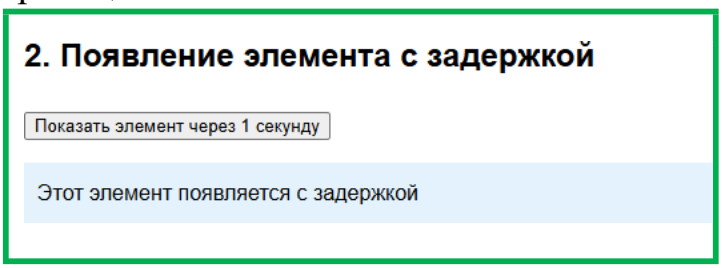
Алгоритм ожидания веб-элемента:

- Открывается html страница из папки **resources** проекта;
- Клик по кнопке для появления элемента;

Вывод результата на экран:

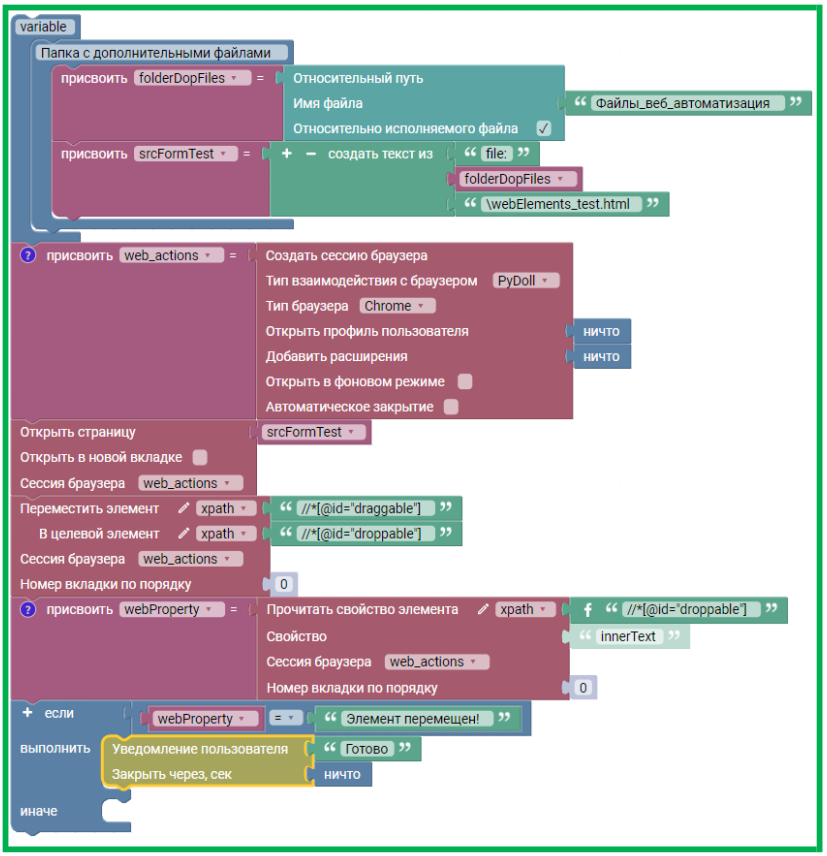


Загруженная страница:



Пример 6 – Переместить элемент

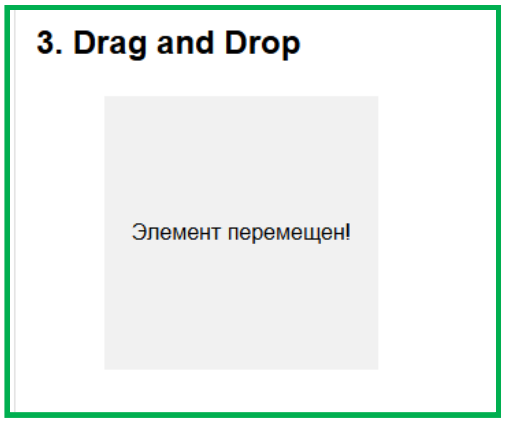
В данном примере происходит ввод текста в форму на html странице:



Алгоритм ожидания веб-элемента:

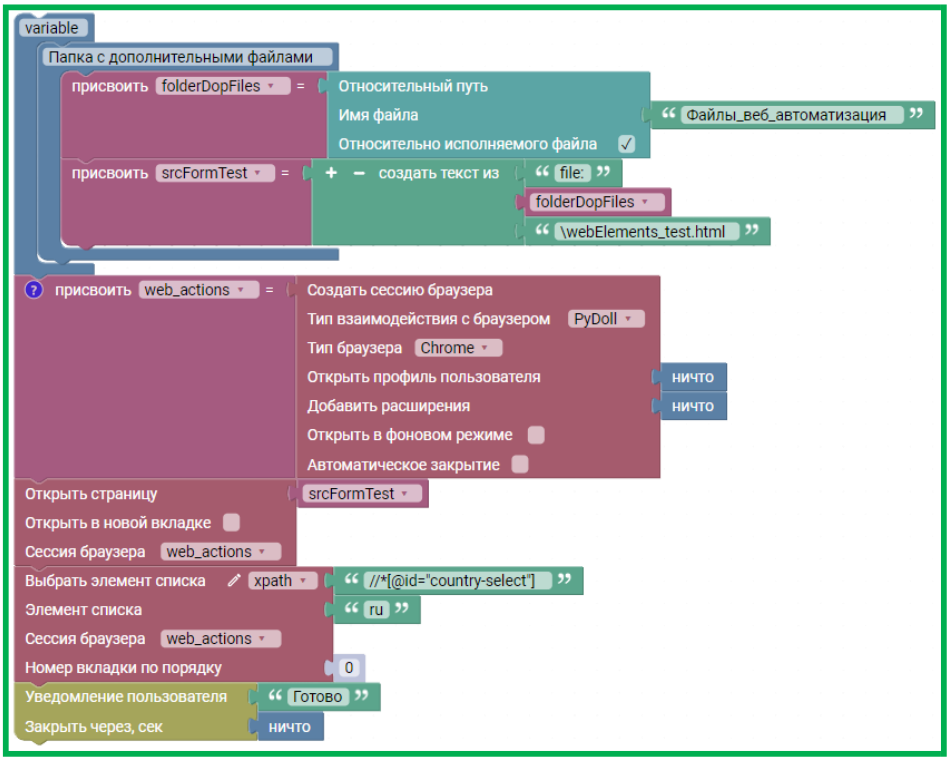
- Открывается html страница из папки **resources** проекта;
- Клик по кнопке **Нажми меня**;
- Выводится текстовое поле **Кнопка была нажата!**;

Вывод результата на экран:



Пример 7 – Выбрать элемент списка

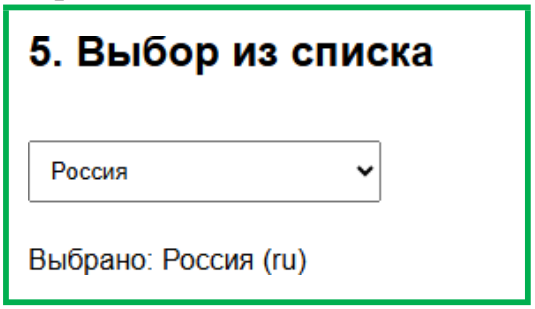
В данном примере происходит ввод текста в форму на html странице:



Алгоритм ожидания веб-элемента:

- Открывается html страница из папки **resources** проекта;
- Выбирается элемент со значение **value** – ru;

Вывод результата на экран:



Кейс 2. «Автоматизация обработки заявок в корпоративной системе»

Компания EduStreamLab разрабатывает онлайн-образовательные программы и интенсивы для специалистов в сфере IT.

После запуска веб-портала регистрации (разработанного на предыдущей практике) система начала активно использоваться, однако возникли новые задачи:

- необходимо управлять статусами заявок участников;
- удалять ошибочные или дублирующиеся заявки;
- выполнять массовую обработку данных;
- анализировать заявки по заданным критериям.

В связи с этим было принято решение:

- расширить функциональность веб-приложения;
- реализовать более сложный сценарий автоматизации с использованием Puzzle RPA.

Задание 1. Расширение функциональности веб-приложения

Необходимо доработать существующее веб-приложение, добавив новые элементы управления и логику обработки заявок.

Важно: Каждый элемент интерфейса должен иметь уникальный `id`

1.1 Дополнительные поля в таблице

В таблицу необходимо добавить два новых поля:

Кнопка	ID (рекомендуемый)	Логика
Mark as Approved	markApprovedBtn	Меняет статус на Approved
Mark as Rejected	markRejectedBtn	Меняет статус на Rejected
Delete Application	deleteAppBtn	Удаляет выбранную запись из таблицы

1.2 Новые кнопки управления

Поле	Назначение	Требования
Created At	Дата и время создания заявки	Автоматически устанавливается при добавлении записи. Формат: YYYY-MM-DD HH:mm:ss
Processed By	Имя пользователя, обработавшего заявку	Заполняется только при изменении статуса. По умолчанию: null

1.3 Управление состоянием элементов (disabled)

Общее правило для всех трёх кнопок:

Условие	Состояние
Ни одна строка не выбрана	`disabled = true`
Выбрана строка таблицы	`disabled = false`

Дополнительные условия для Mark as Approved и Mark as Rejected:

Условие	Состояние
Статус выбранной заявки = Approved	`disabled = true`
Статус выбранной заявки = Rejected	`disabled = true`
Статус выбранной заявки = Pending	`disabled = false`

1.4 Детализация кнопок управления

Кнопка Mark as Approved предназначена для подтверждения заявки и перевода её в состояние одобренной. Бизнес-смысл этой операции заключается в том, что заявка прошла проверку, и участник допускается к мероприятию.

Данная кнопка используется в следующих ситуациях: когда данные заявки заполнены корректно, когда заявка соответствует всем требованиям мероприятия.

Алгоритм работы кнопки `Mark as Approved` выглядит следующим образом. Пользователь сначала выбирает строку в таблице заявок. После выбора система проверяет текущий статус заявки. Если статус равен `Pending`, то операция разрешается. Если же статус не равен `Pending`, то операция запрещается. При нажатии на кнопку происходит изменение статуса на `Approved`, одновременно в поле `Processed By` записывается имя текущего пользователя, например `admin`. После этого таблица обновляется с небольшой задержкой, которая имитирует реальную работу API.

Кнопка Mark as Rejected предназначена для отклонения заявки. Бизнес-смысл этой операции заключается в том, что участник не соответствует условиям участия либо его заявка содержит ошибки, требующие исправления.

Данная кнопка используется в следующих ситуациях: когда в заявке присутствуют некорректные данные, когда количество участников уже превышено, либо когда заявка не соответствует требованиям мероприятия по каким-либо другим причинам.

Алгоритм работы кнопки `Mark as Rejected` следующий. Пользователь выбирает строку в таблице заявок. Затем система проверяет текущий статус. Операция допускается только для заявок со статусом `Pending`. При нажатии на кнопку статус изменяется на `Rejected`, а в поле `Processed By` записывается имя текущего пользователя. После этого таблица обновляется с задержкой.

Кнопка Delete Application предназначена для удаления заявки из системы. Бизнес-смысл этой операции включает в себя удаление дублирующихся заявок, удаление тестовых записей, которые были созданы для проверки, а также очистку системы от некорректных данных.

Алгоритм работы кнопки `Delete Application` достаточно прост. Пользователь выбирает строку в таблице, которую необходимо удалить, и нажимает кнопку `Delete Application`. После этого запись удаляется из таблицы.

1.5 Фильтрация заявок

В веб-приложение необходимо добавить блок фильтрации под названием `Filter by Status`. Этот блок представляет собой выпадающий список, который позволяет пользователю отображать только те заявки, которые соответствуют определённому статусу.

Выпадающий список `Filter by Status` содержит четыре значения. Значение `All` отображает все заявки без какой-либо фильтрации. Значение `Pending` отображает только те заявки, которые имеют статус `Pending`, то есть созданы, но ещё не обработаны. Значение `Approved` отображает только заявки со статусом `Approved`, то есть одобренные заявки. Значение `Rejected` отображает только заявки со статусом `Rejected`, то есть отклонённые.

Фильтр взаимодействует с другими элементами интерфейса определённым образом. После изменения статуса заявки, например при одобрении или отклонении, эта запись может исчезнуть из текущего фильтра. К примеру, если фильтр

настроен на отображение только заявок со статусом `Pending`, и пользователь одобряет одну из них, эта заявка меняет статус на `Approved` и сразу же исчезает из отображаемого списка. После удаления записи таблица автоматически обновляется с учётом текущего фильтра.

1.6 Поиск

В веб-приложение необходимо добавить блок поиска под названием `Search by Name`. Назначение этого элемента – поиск участника по его имени. Поиск должен выполняться в реальном времени, то есть по мере ввода символов в поле поиска список заявок должен динамически обновляться. Допускается частичное совпадение введённой строки с именем участника.

1.7 Входные данные из Excel-файла

Для тестирования и работы робота предоставляется файл с именем `applications.xlsx`, который содержит восемь тестовых заявок.

Важное замечание: Запуск веб-приложения через локальный сервер

После того как мы разработали веб-приложение, главная ошибка это просто открывать HTML-файл двойным щелчком мыши в браузере. В адресной строке при этом появляется ссылка, начинающаяся с `file:///C:/Users/.` Такой способ открытия называется "протокол file" (файловый протокол).

Однако при работе с Puzzle RPA такой подход не сработает. Потому что робот, управляя браузером через Selenium WebDriver или pyDoll, не может корректно взаимодействовать с веб-страницей, открытой через файловый протокол. Возникают проблемы с поиском элементов, выполнением JavaScript и обработкой динамического содержимого страницы.

Для корректной работы робота веб-приложение должно быть доступно по протоколу `http://` или `https://`. Для этого необходимо запустить локальный веб-сервер.

`localhost` – это специальное сетевое имя, которое указывает на ваш собственный компьютер. Оно всегда соответствует IP-адресу `127.0.0.1`. Когда робот обращается к `http://localhost:8000`, он отправляет запрос на ваш же компьютер, где за-пущен сервер. Сервер обрабатывает запрос и возвращает HTML-страницу. Благо-даря этому все механизмы веб-автоматизации работают корректно.

Puzzle RPA использует технологии, которые полагаются на стандартные веб-протоколы. Протокол `http://` поддерживает все необходимые механизмы: поиск элементов по XPath и CSS-селекторам, выполнение JavaScript, обработку событий, ожидание появления динамических элементов. Протокол `file://` эти возможности либо не поддерживает вовсе, либо поддерживает с серьёзными ограничениями.

Важное напоминание – сервер должен оставаться запущенным на всё время работы робота. Если закрыть терминал с запущенным сервером, робот потеряет связь с веб-приложением.

Таким образом, перед запуском робота всегда необходимо сначала запустить локальный веб-сервер и только затем выполнять сценарий в Puzzle RPA. Это обязательное условие для корректной работы автоматизации.

Задание 2. Разработка продвинутого RPA-сценария

Разработать RPA-робота в Puzzle RPA, который автоматически:

- Авторизуется в веб-приложении
- Заполняет 8 тестовых заявок из Excel-файла
- Отправляет их
- Анализирует заявки и изменяет статусы в зависимости от формата участия (`Online` → `Approved`, `Offline` → `Rejected`)
- Применяет фильтрацию по статусу
- Выполняет поиск по имени
- Удаляет выбранную запись
- Проверяет итоговое состояние таблицы
- Ведет расширенное логирование всех этапов

Рекомендуется реализовать структуру с использованием функций/подпроцессов.

После работы робота в таблице должны быть все добавленные записи со следующими статусами: все заявки с форматом Online должны иметь статус Approved, все заявки с форматом Offline должны иметь статус Rejected. Одна запись должна быть удалена.

Теперь перейдём к реализации RPA-сценария в среде Puzzle RPA.

В рамках практической работы был разработан продвинутый RPA-робот в среде Puzzle RPA для автоматизации процесса обработки заявок на участие в образовательных программах. Робот выполняет следующие функции:

- Автоматический вход в веб-приложение
- Чтение данных из Excel-файла
- Массовое заполнение и отправка заявок
- Анализ и изменение статусов заявок
- Применение фильтрации
- Выполнение поиска
- Удаление выбранной записи
- Проверка итогового состояния
- Расширенное логирование всех этапов выполнения

Функция `TextTooLog`

Шаг 1. Блок "Получить текущую дату" – форматируем время в формат "дд.мм.гггг чч:мм:сс".

Шаг 2. Блок "Собрать строку" – склеиваем полученную дату, текст информационного сообщения и перевод строки. На вход функция принимает параметр info, который содержит текстовое сообщение для записи в лог. Это сообщение пе-

редаётся при вызове функции, например `TextTooLog`('Окно для ввода загружено'), где строка 'Окно для ввода загружено' и есть переменная info.

Шаг 3. Блок "Дописать в файл" – сохраняем сформированную строку в файл log.txt, который располагается в папке с программой. Таким образом, каждый вызов функции добавляет новую строку в лог-файл с указанием времени и переданным сообщением.

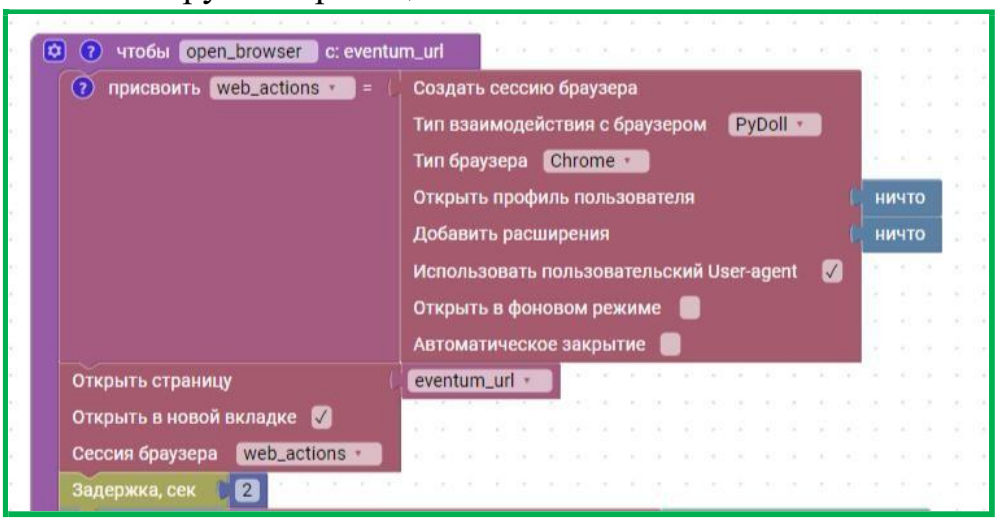


Функция `open_browser`

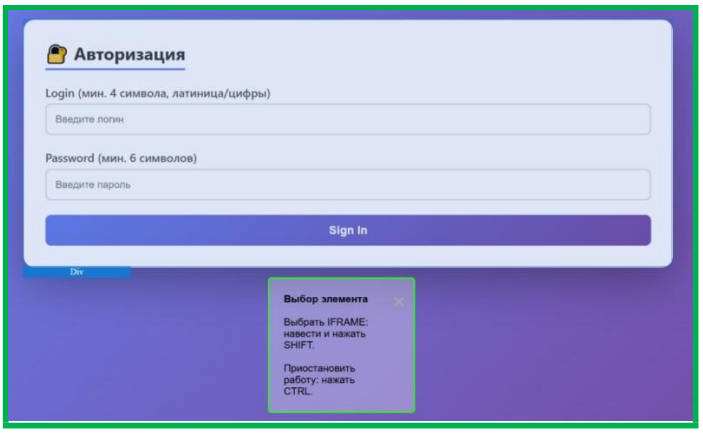
Шаг 1. Блок "Создать сессию браузера" – инициализируем браузер через функцию `init_web_automation_pydoll` с параметрами: тип браузера = Chrome. На вход функция принимает параметр eventum_url, который содержит адрес веб-страницы, которую необходимо открыть. Этот URL передаётся при вызове функции.

Шаг 2. Блок "Открыть страницу в браузере" – переходим по URL, полученному в переменной eventum_url, открывая страницу в новой вкладке браузера.

Шаг 3. Блок "Задержка" – устанавливаем паузу продолжительностью 2 секунды для полной загрузки страницы.



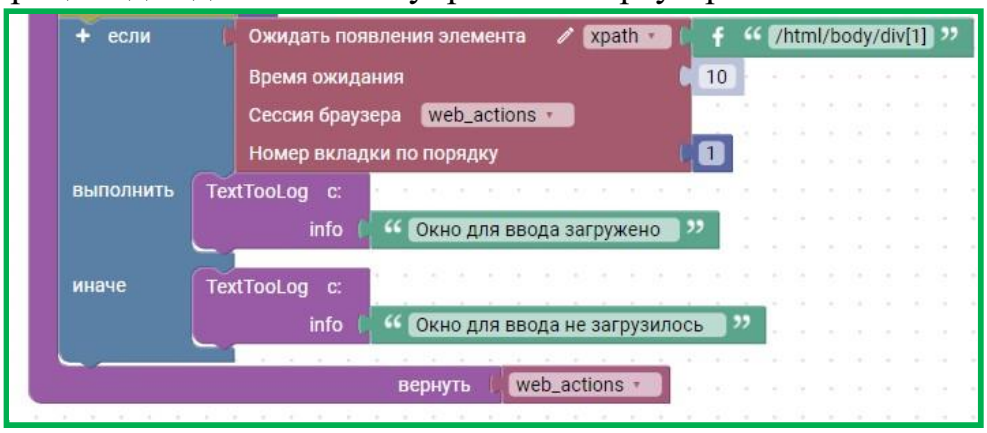
Шаг 4. Блок "Ожидать появление элемента" – ждём появления элемента с XPath `"/html/body/div[1]"` с таймаутом 10 секунд. Это позволяет убедиться, что главное окно приложения загрузилось.



Шаг 5. Если окно появилось – вызываем функцию `TextTooLog` с сообщением "Окно для ввода загружено".

Шаг 6. Если окно не появилось – вызываем функцию `TextTooLog` с сообщением "Окно для ввода не загрузилось".

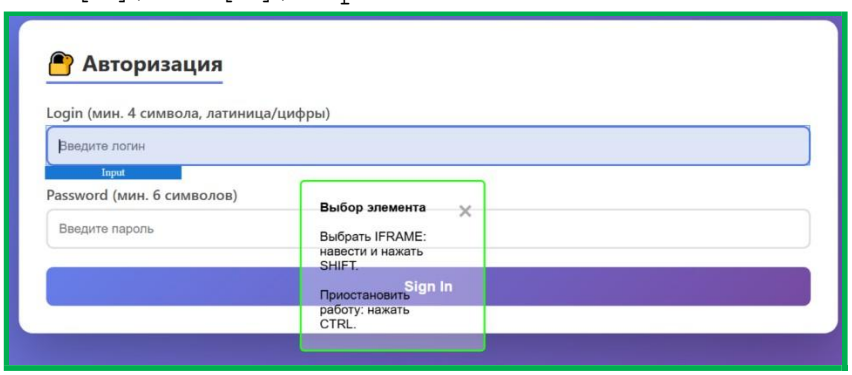
Шаг 7. Блок "Вернуть результат" – возвращаем переменную `web_actions` в основной процесс для дальнейшего управления браузером.



Функция `autorisation`

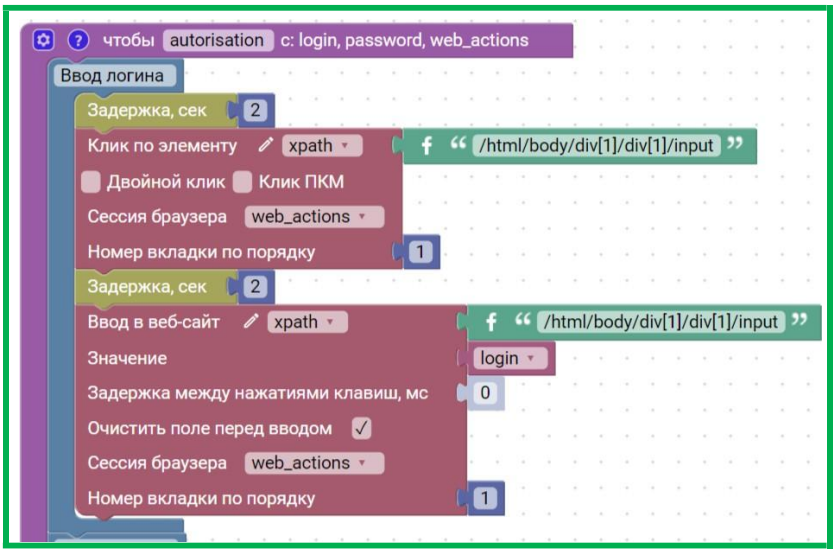
Шаг 1. Блок "Задержка" – пауза 2 секунды перед началом работы с формой авторизации. На вход функция принимает три параметра: `login` (логин пользователя), `password` (пароль пользователя) и `web_actions` (сессия браузера, через которую осуществляется управление веб-страницей). Параметр `web_actions` передаётся из функции `open_browser` и содержит все методы для взаимодействия с браузером, такие как клики, ввод текста и проверка элементов.

Шаг 2. Блок "Клик по Web-элементу" – с помощью переданной сессии `web_actions` выполняем клик по полю ввода логина с XPath `"/html/body/div[1]/div[1]/input"`.



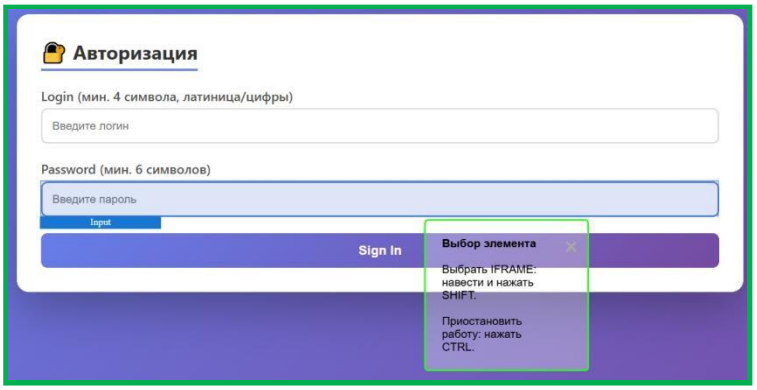
Шаг 3. Блок "Задержка" – пауза 2 секунды после клика.

Шаг 4. Блок "Ввод в веб-сайт" – используя сессию `web_actions`, вводим текст, полученный в параметре `login`, в поле ввода логина.



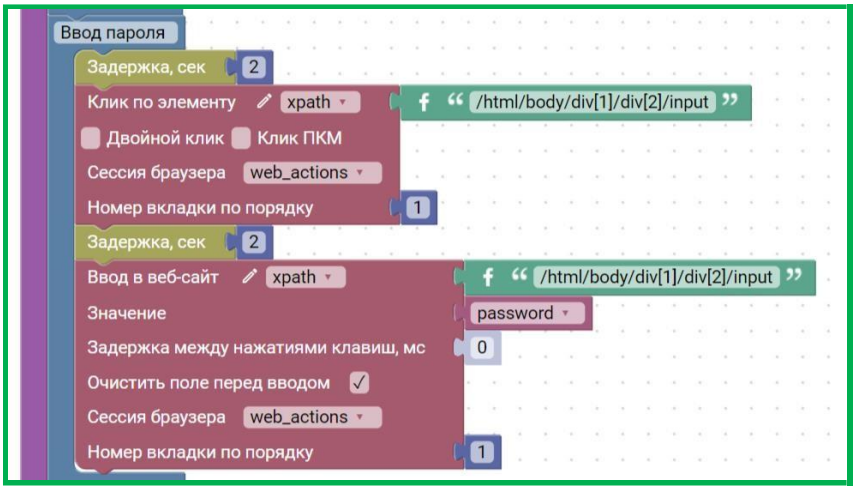
Шаг 5. Блок "Задержка" – пауза 2 секунды.

Шаг 6. Блок "Клик по Web-элементу" – с помощью сессии `web_actions` выполняем клик по полю ввода пароля с XPath `"/html/body/div[1]/div[2]/input"`.

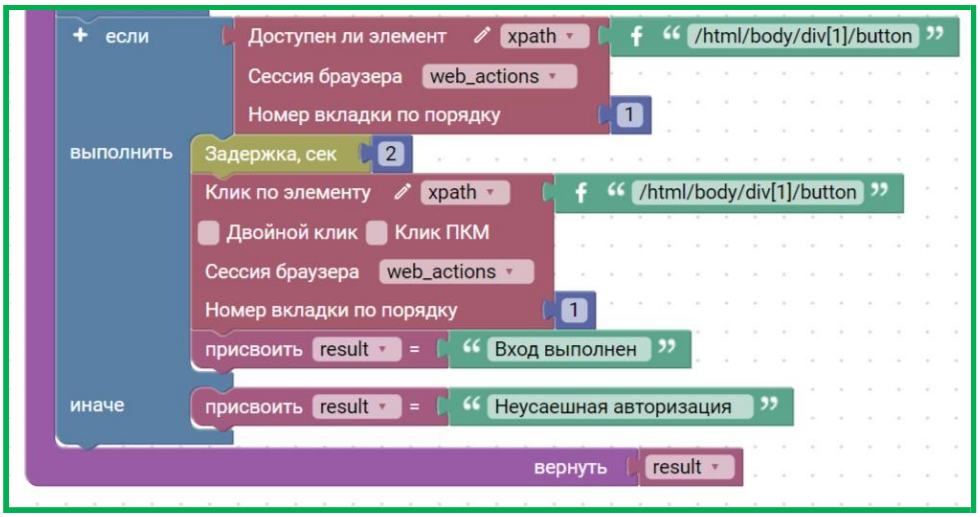


Шаг 7. Блок "Задержка" – пауза 2 секунды.

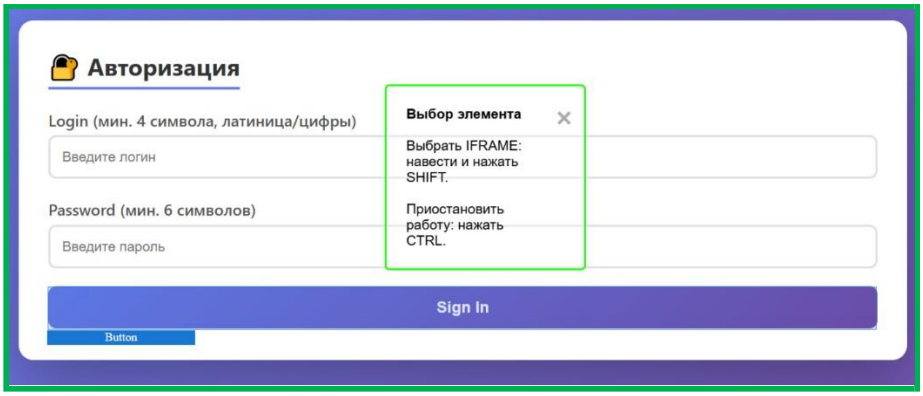
Шаг 8. Блок "Ввод в веб-сайт" – используя сессию `web_actions`, вводим текст, полученный в параметре `password`, в поле ввода пароля.



Шаг 9. Блок "Доступен ли элемент" – с помощью сессии `web_actions` проверяем, активна ли кнопка входа с XPath `"/html/body/div[1]/button"`.



Шаг 10. Если кнопка доступна – выполняем задержку 2 секунды, затем с помощью `web_actions` кликаем по кнопке и присваиваем переменной `result` значение "Вход выполнен".



Шаг 11. Если кнопка недоступна – присваиваем переменной `result` значение "Неуспешная авторизация".

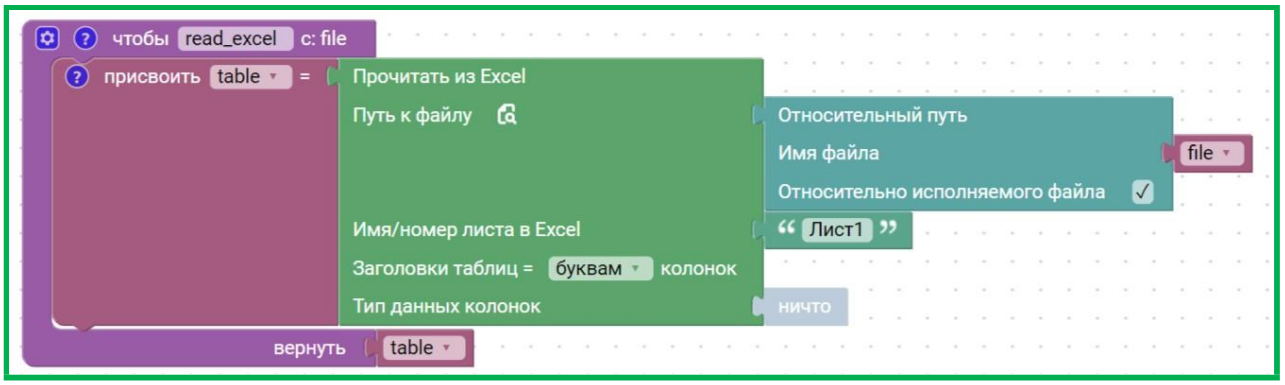
Шаг 12. Блок "Вернуть результат" – возвращаем переменную `result` в основной процесс, чтобы в дальнейшем можно было проверить, успешно ли прошла авторизация.

Функция `read_excel`

Шаг 1. Блок "Относительный путь" – формируем полный путь к файлу, имя которого передаётся в параметре `file`. На вход функция `read_excel` принимает параметр file, который содержит имя или относительный путь к Excel-файлу, 'requests.xlsx'. Этот параметр указывает, какой именно файл необходимо прочитать.

Шаг 2. Блок "Прочитать из Excel" – читаем данные из файла, путь к которому был сформирован на основе переданного параметра `file`, с листа "Лист1", сохраняя все данные в переменную `table`. Таким образом, функция может работать с любым Excel-файлом, имя которого будет передано при вызове.

Шаг 3. Блок "Вернуть результат" – возвращаем переменную `table` в основной процесс для дальнейшего использования при заполнении формы. Благодаря тому, что имя файла передаётся как параметр, функция является универсальной и может использоваться для чтения разных Excel-файлов без изменения её кода.



Функция `new_request`

Шаг 1. Блок "Цикл по (от до с шагом)" – организуем цикл для `i` от 1 до количества строк в таблице минус один, то есть повторяем действия для всех заявок из Excel-файла. Нам нужны только строки с данными (начиная с индекса 1), а не строка с заголовками (индекс 0). Поэтому цикл организуется от `i = 1` до `i_end = len(table)-1`. На вход функция принимает два параметра: `table` (таблица с данными из Excel, полученная от функции `read_excel`) и `web_actions` (сессия браузера, через которую осуществляется управление веб-страницей). Параметр `table` содержит все данные из Excel-файла, где каждая строка соответствует отдельной заявке, а колонки содержат имя участника, `email`, название мероприятия и формат участия. Параметр `web_actions` передаётся из основной программы и содержит все методы для взаимодействия с браузером, такие как клики, ввод текста и ожидание элементов.

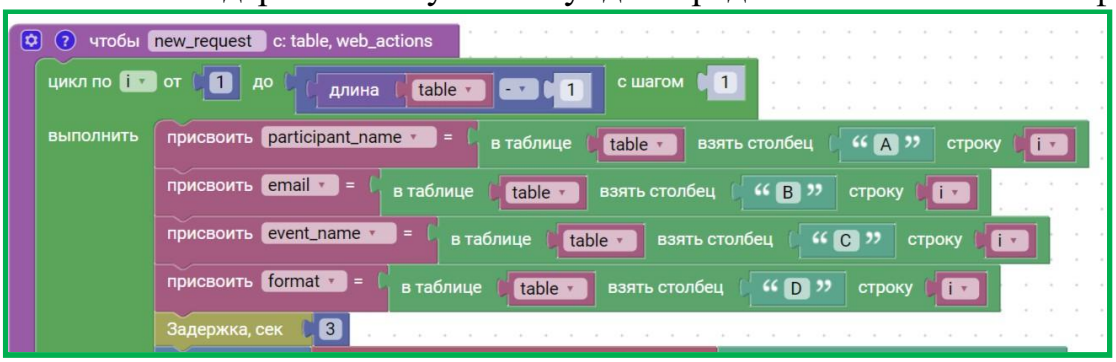
Шаг 2. Блок "Получить значение ячейки табличных данных" – из переменной table, полученной на вход, извлекаем значение из колонки A (`Participant Name`) по индексу i и сохраняем в переменную `participant_name`.

Шаг 3. Блок "Получить значение ячейки табличных данных" – из переменной `table` извлекаем значение из колонки B (`Email`) по индексу i и сохраняем в переменную `email`.

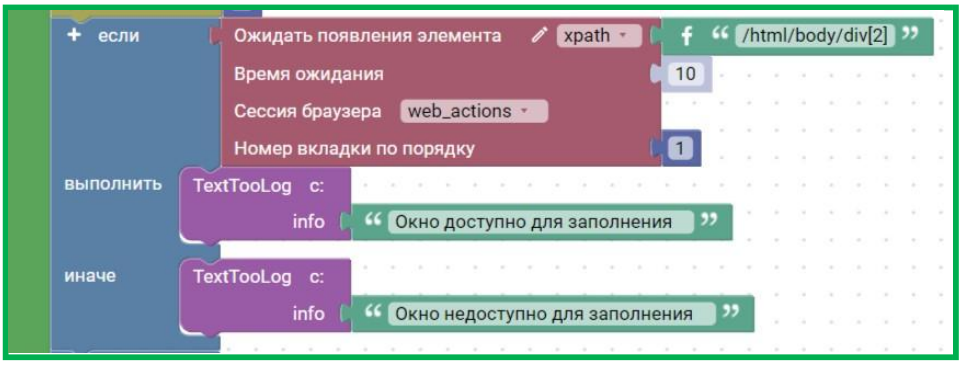
Шаг 4. Блок "Получить значение ячейки табличных данных" – из переменной `table` извлекаем значение из колонки C (`Event Name`) по индексу `i` и сохраняем в переменную `event_name`.

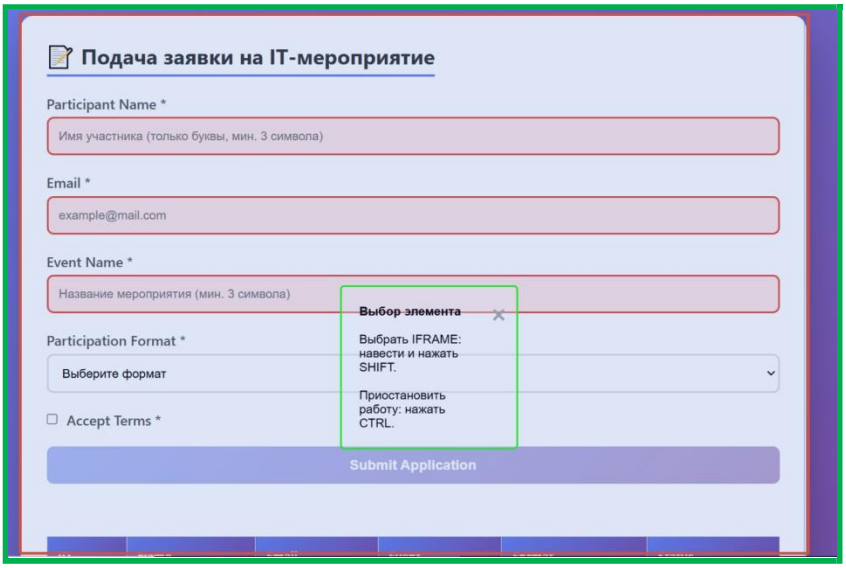
Шаг 5. Блок "Получить значение ячейки табличных данных" – из переменной `table` извлекаем значение из колонки D (`Participation Format`) по индексу i и сохраняем в переменную `format`.

Шаг 6. Блок "Задержка" – пауза 3 секунды перед началом заполнения формы.



Шаг 7. Блок "Ожидать появление элемента" – с помощью переданной сессии `web_actions` проверяем, появилось ли окно формы заявки с XPath `"/html/body/div[2]"`, с таймаутом 10 секунд.



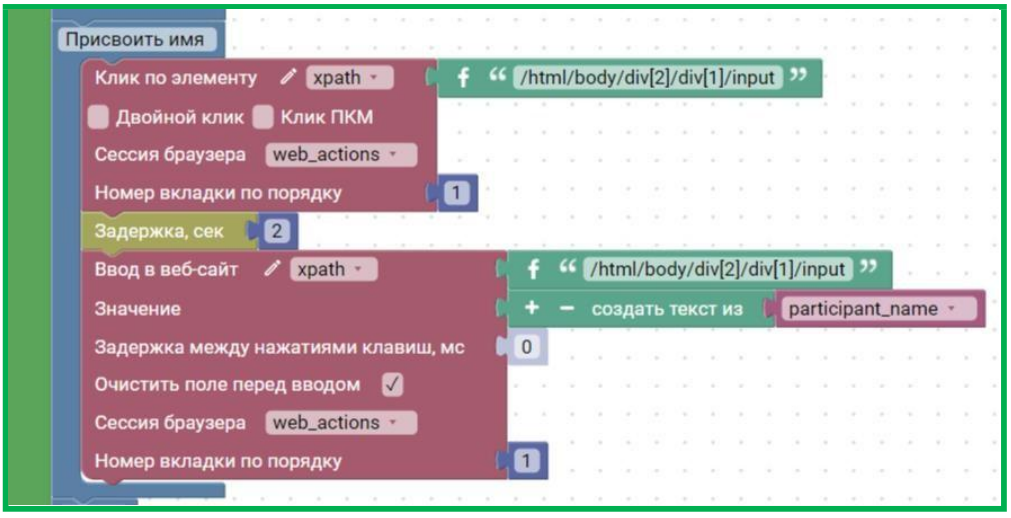


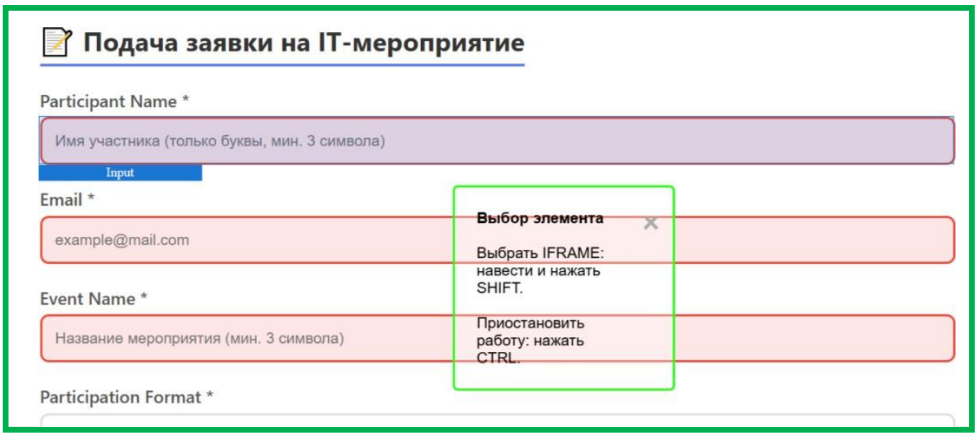
Шаг 8. Если окно доступно – вызываем функцию `TextTooLog` с сообщением "Окно доступно для заполнения". Если нет – вызываем `TextTooLog` с сообщением "Окно недоступно для заполнения".

Шаг 9. Блок "Клик по Web-элементу" – используя сессию web_actions, кликаем по полю Participant Name с XPath `"/html/body/div[2]/div[1]/input"`.

Шаг 10. Блок "Задержка" – пауза 2 секунды.

Шаг 11. Блок "Ввод в веб-сайт" – с помощью `web_actions` вводим значение переменной `participant_name` в поле `Participant Name`.

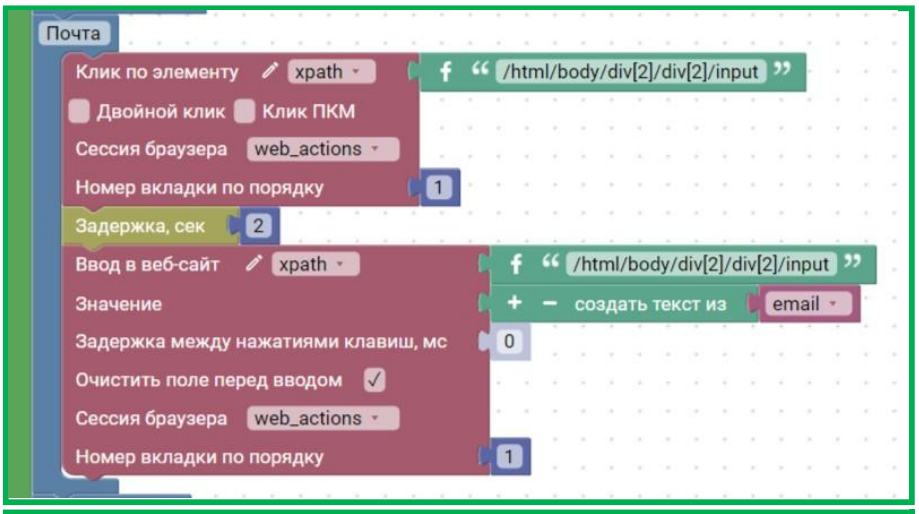


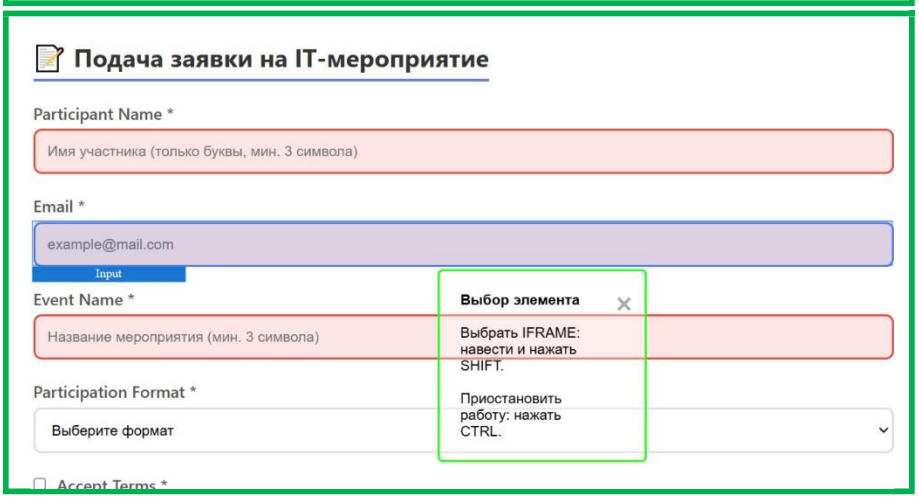


Шаг 12. Блок "Клик по Web-элементу" – используя `web_actions`, кликаем по полю Email с XPath `"/html/body/div[2]/div[2]/input"`.

Шаг 13. Блок "Задержка" – пауза 2 секунды.

Шаг 14. Блок "Ввод в веб-сайт" – с помощью `web_actions` вводим значение переменной `email` в поле `Email`.

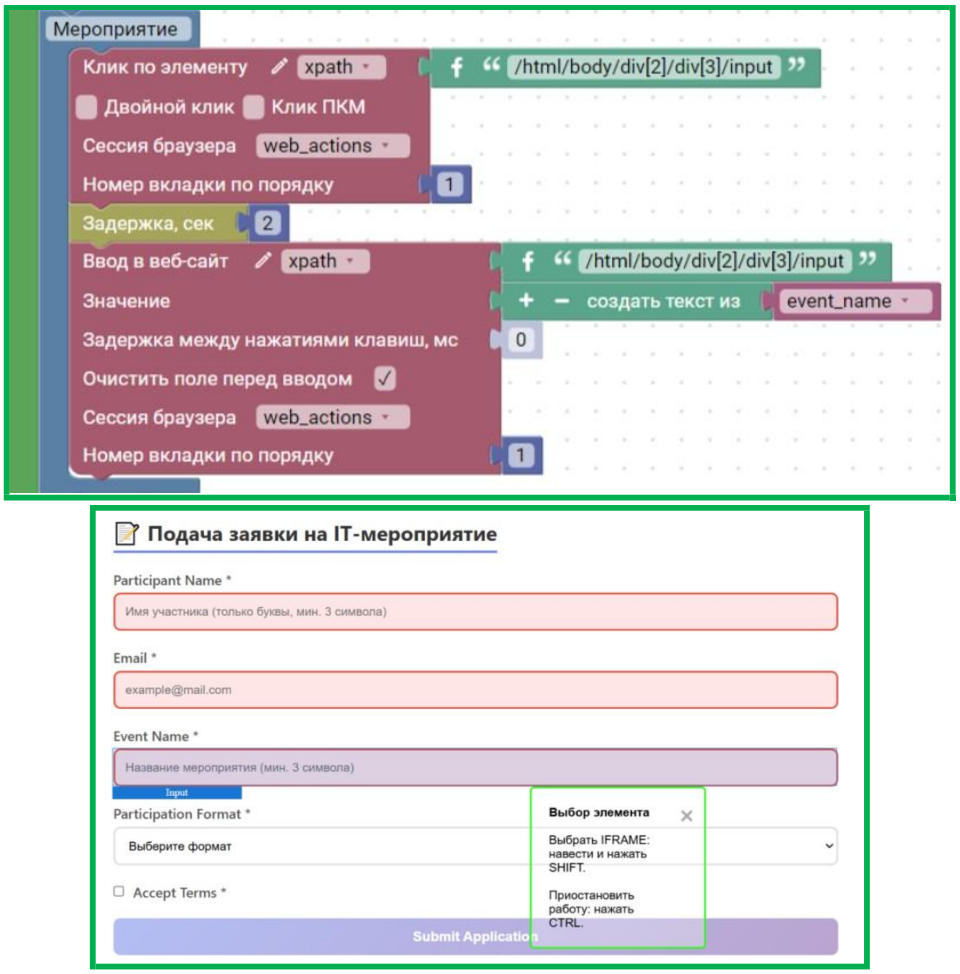




Шаг 15. Блок "Клик по Web-элементу" – используя web_actions, кликаем по полю Event Name с XPath `"/html/body/div[2]/div[3]/input"`.

Шаг 16. Блок "Задержка" – пауза 2 секунды.

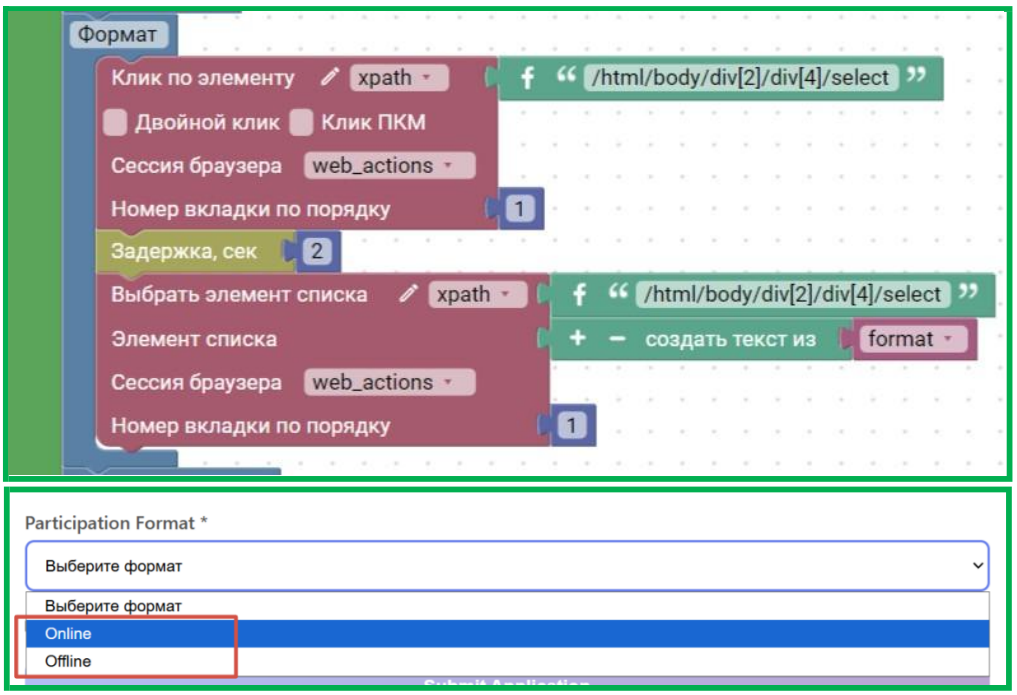
Шаг 17. Блок "Ввод в веб-сайт" – с помощью `web_actions` вводим значение переменной `event_name` в `поле Event Name`.



Шаг 18. Блок "Клик по Web-элементу" – используя web_actions, кликаем по выпадающему списку `Participation Format` с XPath `"/html/body/div[2]/div[4]/select"`.

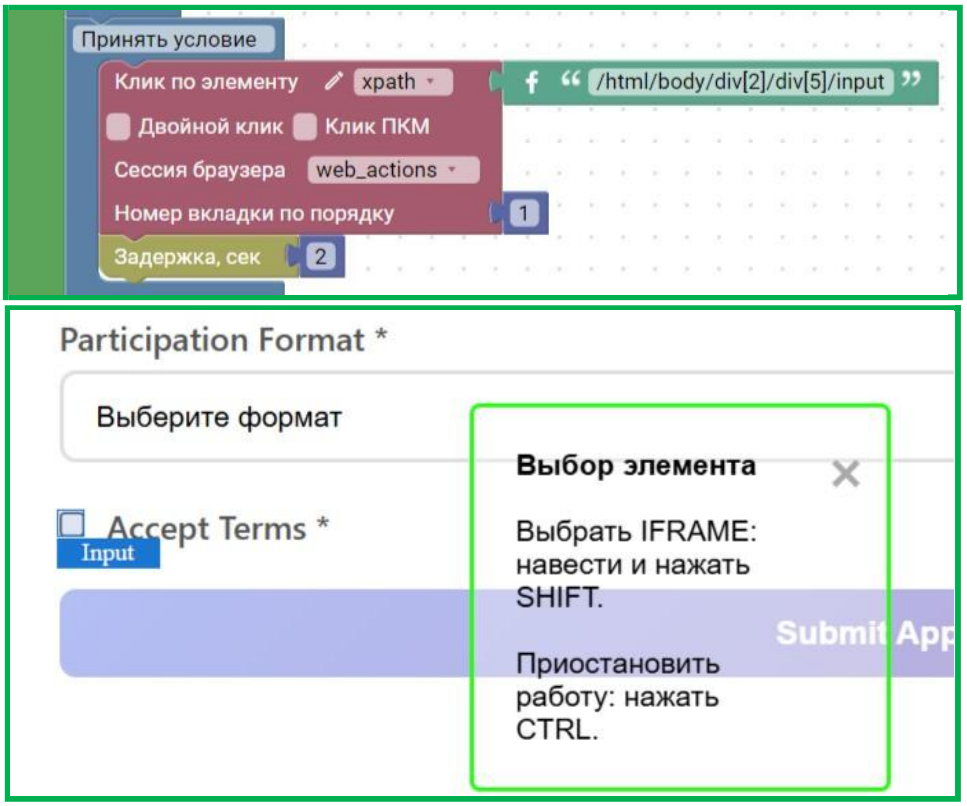
Шаг 19. Блок "Задержка" – пауза 2 секунды для раскрытия списка.

Шаг 20. Блок "Выбрать элемент списка" – с помощью `web_actions` выбираем значение из списка, соответствующее переменной `format` (`Online` или `Offline`).

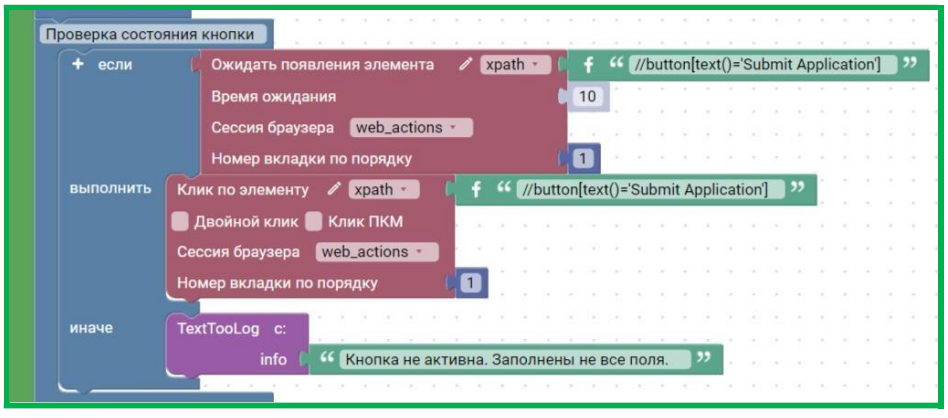


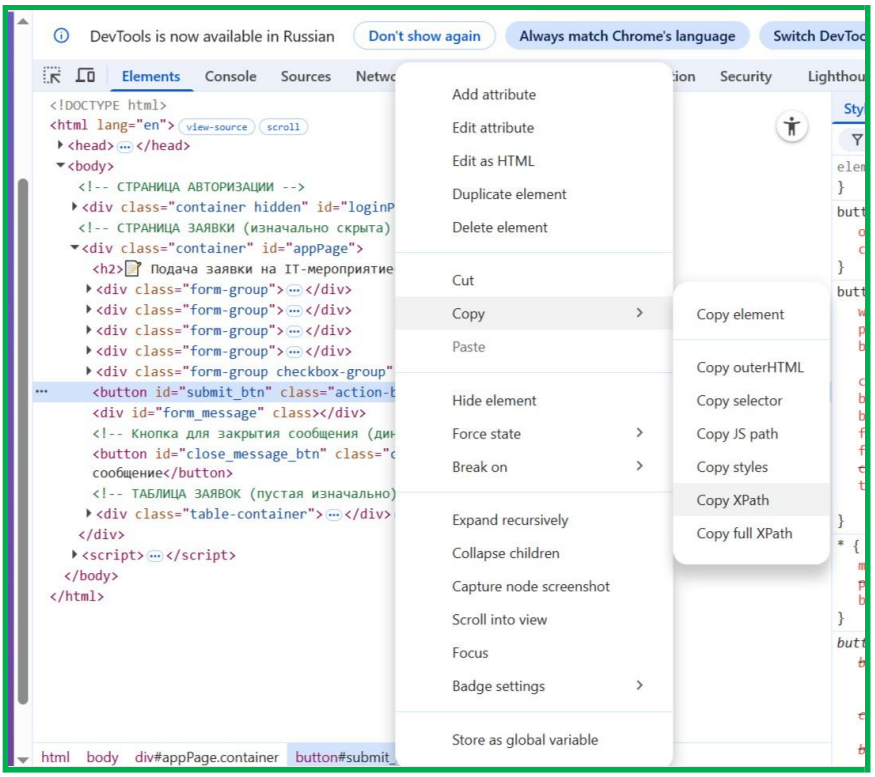
Шаг 21. Блок "Клик по Web-элементу" – используя `web_actions`, кликаем по чекбоксу `Accept Terms` с XPath `"/html/body/div[2]/div[5]/input"`.

Шаг 22. Блок "Задержка" – пауза 2 секунды.



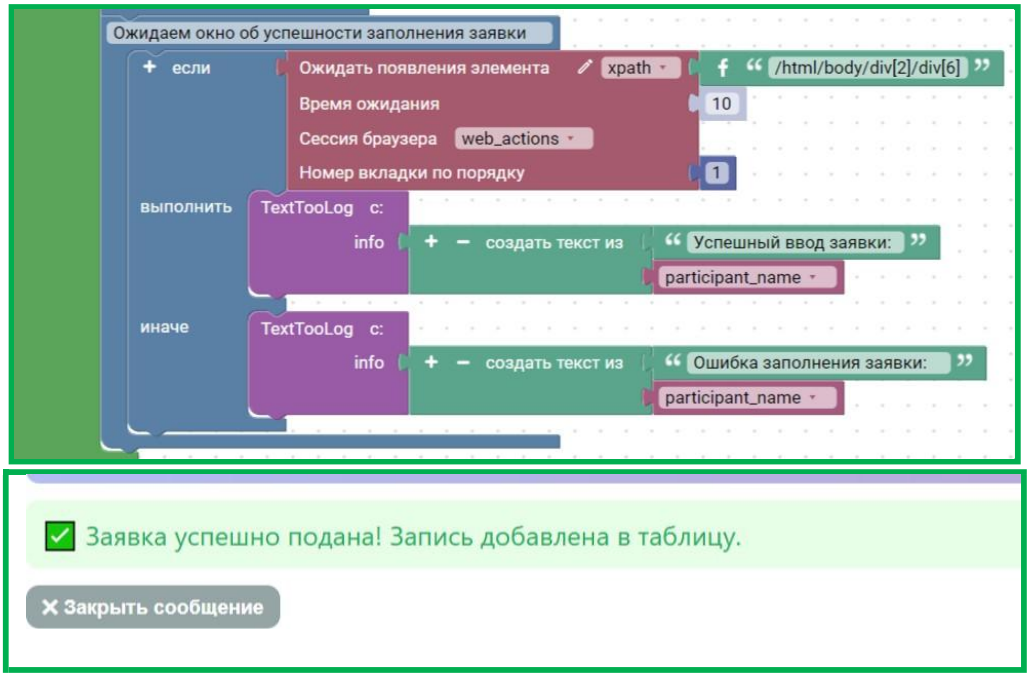
Шаг 23. Блок "Ожидать появление элемента" – с помощью `web_actions` ожидаем появления кнопки Submit Application с XPath `"//button[text()='Submit Application']"` с таймаутом 10 секунд. Чтобы выбрать XPath кнопки нам необходимо вызвать команду F12 и скопировать полный путь к элементу кнопки `Submit Application`.





Шаг 24. Если кнопка активна – выполняем клик по ней с помощью `web_actions`. Если нет – вызываем `TextTooLog` с сообщением "Кнопка не активна. Заполнены не все поля".

Шаг 25. Блок "Ожидать появление элемента" – с помощью `web_actions` ожидаем появления окна об успешной отправке заявки с XPath `"/html/body/div[2]/div[6]"` с таймаутом 10 секунд.



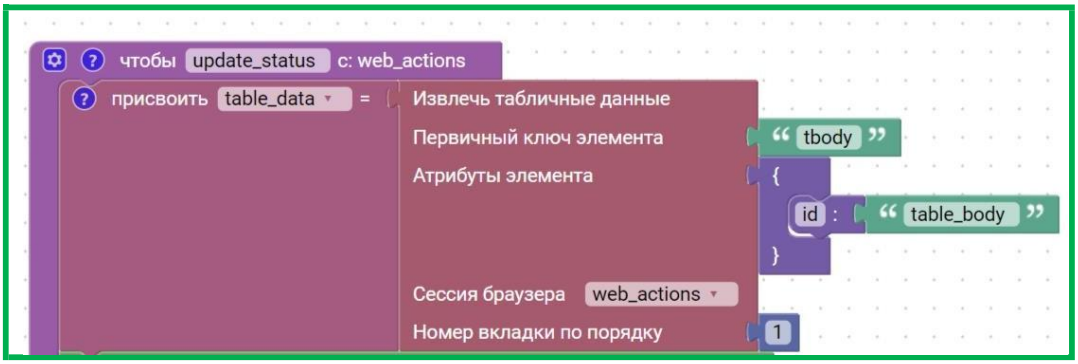
Шаг 26. Если окно появилось – вызываем `TextTooLog` с сообщением "Успешный ввод заявки: " + `participant_name`. Если нет – вызываем `TextTooLog` с сообщением "Ошибка заполнения заявки: " + `participant_name`.

Шаг 27. Блок "Задержка" – пауза 2 секунды перед переходом к следующей заявке. Таким образом, функция `new_request` использует переданные данные таблицы для последовательного заполнения пяти заявок и сессию браузера для автоматизации всех действий на веб-странице.

Функция `update_status`

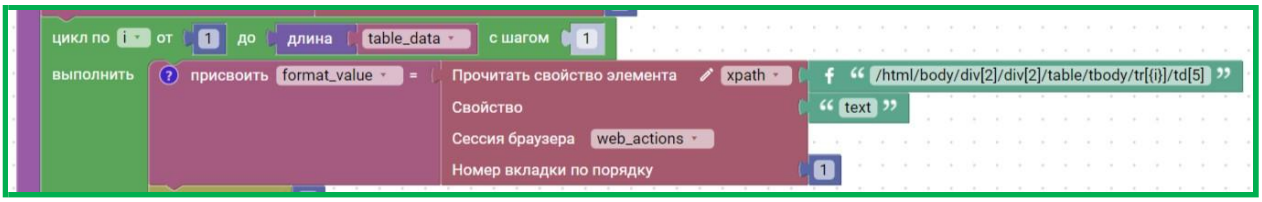
Функция `update_status` предназначена для автоматического изменения статусов уже добавленных заявок в таблице веб-приложения. Она анализирует формат участия каждой заявки (`Online` или `Offline`) и в зависимости от этого нажимает соответствующую кнопку: `Mark` as `Approved` для онлайн-заявок или `Mark` as `Rejected` для офлайн-заявок.

Шаг 1. Блок "Извлечь табличные данные" – с помощью переданной сессии `web_actions` извлекаем все данные из таблицы заявок. Для этого используется блок извлечь табличные данные, который ищет элемент `tbody` с идентификатором `table_body`. Полученные данные сохраняются в переменную `table_data`. На вход функция принимает один параметр: `web_actions` (сессия браузера, через которую осуществляется управление веб-страницей). Параметр `web_actions` передаётся из основной программы и содержит все методы для взаимодействия с браузером, такие как клики, чтение свойств элементов и ожидание элементов.



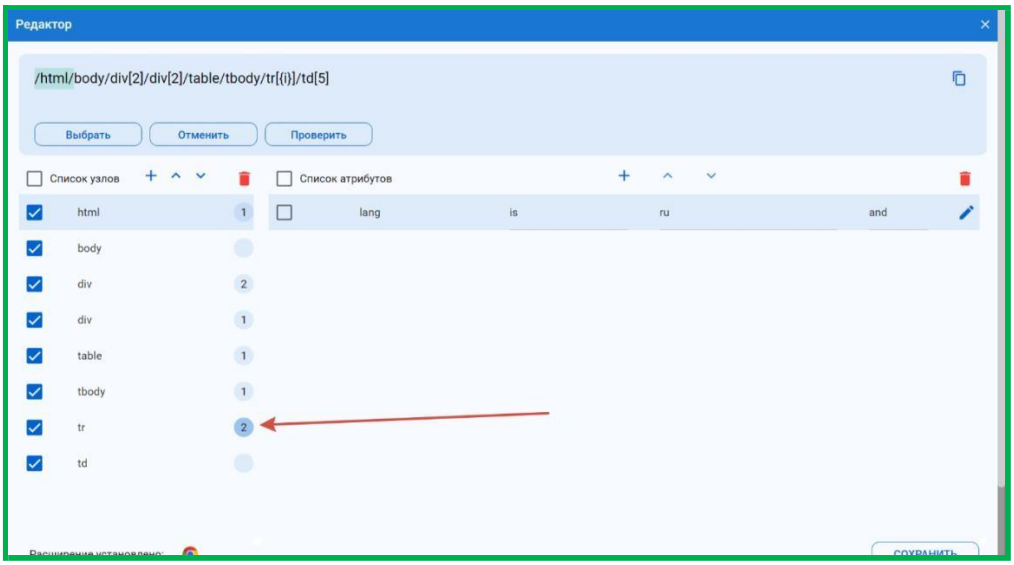
Шаг 2. Блок "Цикл по (от до с шагом)" – организуем цикл для i от 1 до количества строк в таблице включительно. В отличие от функции `new_request`, здесь мы начинаем с i = 1, потому что таблица заявок не содержит строки с заголовками – первая строка (`i = 1`) уже является первой заявкой. Конечное значение `i_end` равно общему количеству строк в таблице, полученному через `len(table_data)`. Таким образом, цикл проходит по всем заявкам, которые были добавлены в таблицу.

Шаг 3. Блок "Прочитать свойство элемента" – с помощью переданной сессии `web_actions` считываем значение формата участия для текущей строки `i`. Для этого используется метод `element_property`, который ищет элемент по XPath `"/html/body/div[2]/div[2]/table/tbody/tr[{i}]/td[5]"` и извлекает его текстовое содержимое. Пятая колонка в таблице (`td[5]`) соответствует формату участия (`Online` или `Offline`). `{i}` – это место, куда подставляется текущее значение переменной `i` из цикла. Полученное значение сохраняется в переменную `format_value`.



В редакторе Puzzle RPA при настройке XPath с переменной нужно:

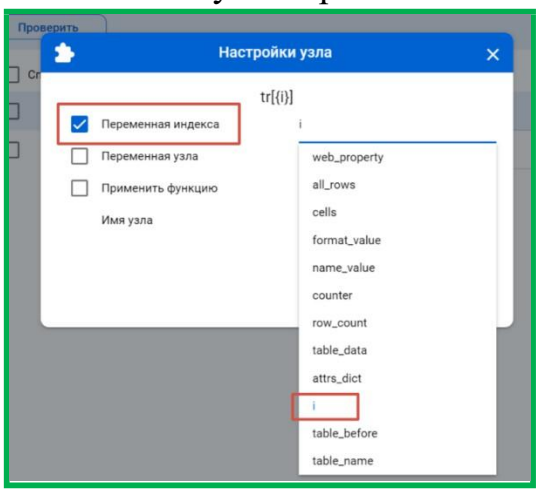
Шаг 1. Открыть форму «Настройка атрибута» для узла, где требуется подставить переменную (например, для номера строки tr).



Шаг 2. Установить галочку в чекбоксе «Переменная атрибута» – это означает, что значение атрибута будет не константой, а будет браться из переменной.

Шаг 3. Выбрать нужную переменную из выпадающего списка – в данном случае переменную i.

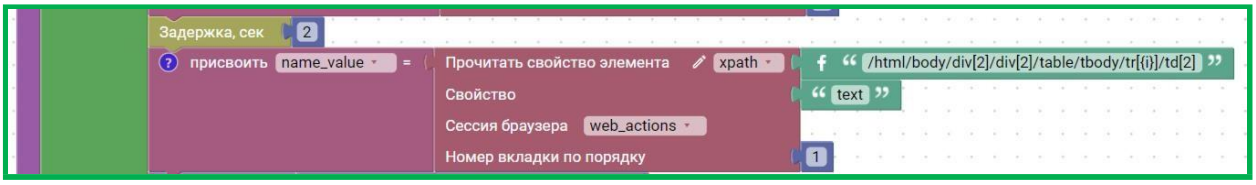
Шаг 4. Выполнить клик на кнопку «Сохранить».



Шаг 4. Блок "Задержка" – пауза продолжительностью 2 секунды. Это необходимо для того, чтобы веб-приложение успело обработать предыдущие действия и стабилизировать состояние перед чтением следующего элемента.

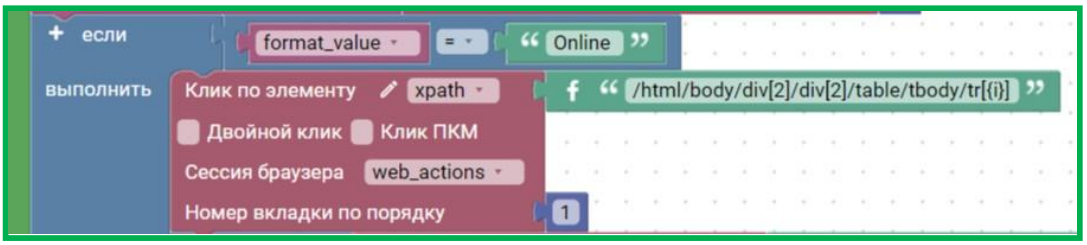
Шаг 5. Блок "Присвоить значение переменной" – с помощью переданной сессии web_actions считываем имя участника для текущей строки i. XPath для этого элемента – `"/html/body/div[2]/div[2]/table/tbody/tr[{i}]/td[2]"`. Вторая колонка таблицы (`td[2]`) содержит имя участника. `{i}` – это место, куда подставляется текущее значение переменной i из цикла. Полученное значение со-

хранится в переменную `name_value`. Это имя будет использоваться для логирования, чтобы понять, по какой именно заявке происходит изменение статуса.

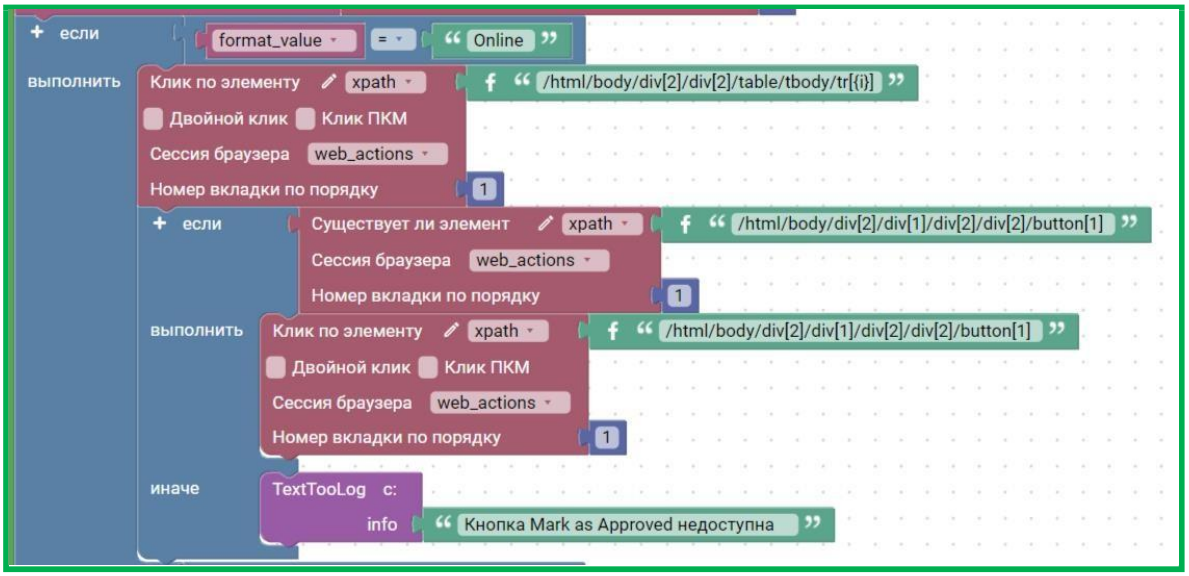


Шаг 6. Блок "Условный оператор" (Если – выполнить) – проверяем значение переменной `format_value`.

Шаг 7. Если `format_value` равно 'Online' – выполняем следующие действия. Сначала с помощью `web_actions` кликаем по строке таблицы с индексом i, чтобы выделить её. XPath для строки – `"/html/body/div[2]/div[2]/table/tbody/tr[{i}]"`. `{i}` – это место, куда подставляется текущее значение переменной i из цикла. Это необходимо, потому что кнопки управления (`Mark as Approved`, `Mark as Rejected`, `Delete Application`) активны только тогда, когда выбрана какая-либо строка.

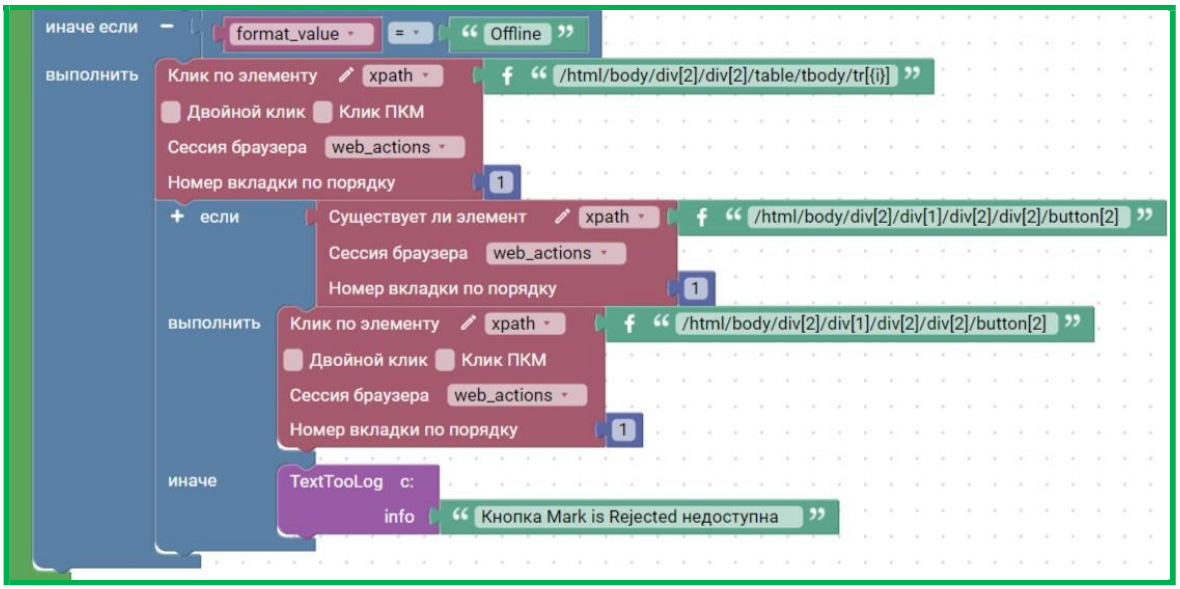


Шаг 8. Блок "Если – выполнить" (вложенный) – проверяем, существует ли кнопка Mark as Approved. Для этого используется метод `check_exists_element` с XPath `"/html/body/div[2]/div[1]/div[2]/div[2]/button[1]"`. Если кнопка существует (то есть доступна для нажатия), выполняем клик по ней. Если кнопка не существует (то есть недоступна по каким-либо причинам, например, статус заявки уже обработан), вызываем функцию `TextTooLog` с сообщением "Кнопка `Mark as Approved` недоступна" для логирования проблемы.



Шаг 9. Если `format_value` равно 'Offline' – выполняем аналогичные действия. Сначала кликаем по строке таблицы с индексом i, чтобы выделить её. За-

тем проверяем, существует ли кнопка `Mark as Rejected` с XPath `"/html/body/div[2]/div[1]/div[2]/div[2]/button[2]"`. Если кнопка существует, кликаем по ней. Если нет, вызываем `TextTooLog` с сообщением "Кнопка `Mark` is `Rejected` недоступна".



Шаг 10. Блок "Вернуть результат" – после завершения цикла по всем заявкам функция возвращает переменную `table_data` (извлечённые данные таблицы) в основной процесс. Это позволяет при необходимости проанализировать состояние таблицы после изменения статусов.



Таким образом, функция `update_status` использует переданную сессию браузера для последовательного обхода всех заявок в таблице, определения их формата участия и автоматического нажатия соответствующей кнопки для изменения статуса. Все действия логируются с помощью функции `TextTooLog`, что позволяет отслеживать успешность выполнения и выявлять проблемные ситуации, когда кнопки оказываются недоступными.

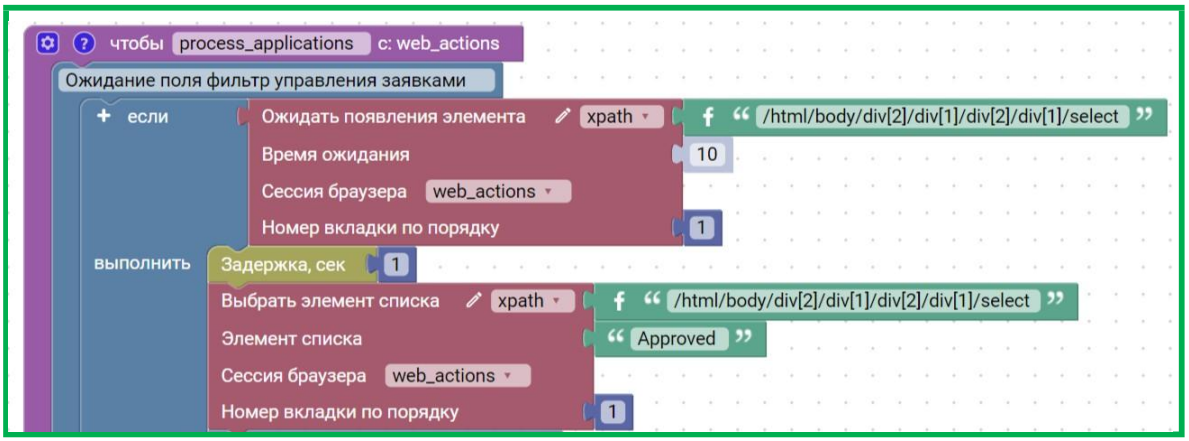
Функция `process_applications`

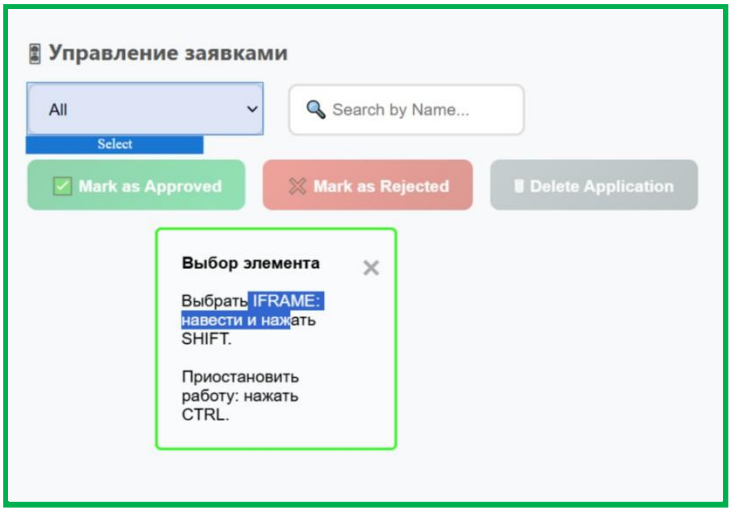
Функция `process_applications` предназначена для проверки работы фильтрации и поиска в веб-приложении. Она имитирует действия пользователя: применяет фильтр по статусу, выполняет поиск по имени, проверяет корректность результатов, а затем очищает фильтр и поле поиска. Это необходимо для тестирования функциональности веб-приложения и убеждения, что робот корректно взаимодействует с динамическими элементами.

Шаг 1. Блок "Группировка блоков" – начало группы действий по работе с фильтром и поиском. Здесь будет выполняться проверка наличия элементов фильтрации на странице.

Шаг 2. Блок "Если – выполнить" – с помощью переданной сессии `web_actions` проверяем, существует ли на странице выпадающий список филь-

тра Filter by Status. Для этого используется блок «Ожидать появление элемента» с XPath `"/html/body/div[2]/div[1]/div[2]/div[1]/select"` и таймаутом 10 секунд. Если элемент найден, робот переходит к настройке фильтра. Если нет – логируется ошибка.





Шаг 3. Если фильтр найден – выполняем задержку 1 секунду, чтобы страница стабилизировалась после ожидания.

Шаг 4. Блок "Выбрать элемент списка" – с помощью `web_actions` выбираем в выпадающем списке значение "`Approved`". Это означает, что в таблице должны остаться только заявки со статусом `Approved`.

Шаг 5. Блок "Группировка блоков" – начало группы действий по работе с полем поиска.

Шаг 6. Блок "Если – выполнить" – с помощью блока «Ожидать появление элемента» проверяем, существует ли на странице поле ввода для поиска по имени. XPath этого элемента – `"/html/body/div[2]/div[1]/div[2]/div[1]/input"`. Таймаут также составляет 10 секунд.

Шаг 7. Если поле поиска найдено – выполняем задержку 1 секунду.

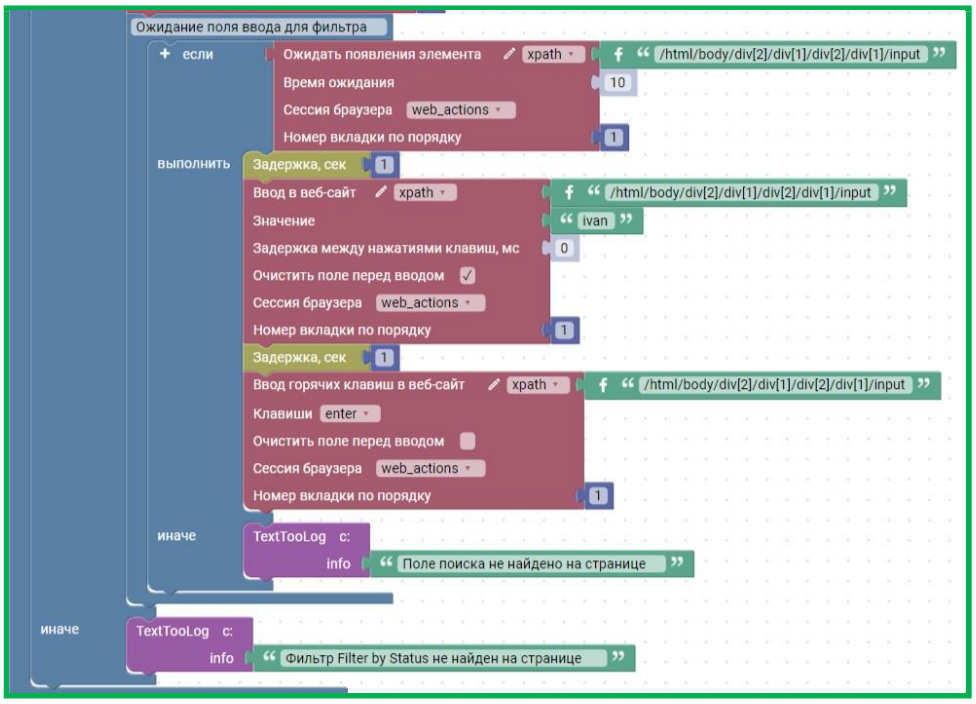
Шаг 8. Блок "Ввод в веб-сайт" – с помощью `web_actions` вводим текст `"ivan"` в поле поиска. Это имитирует действие пользователя, который ищет участника по имени.

Шаг 9. Блок "Задержка" – пауза 1 секунда после ввода текста.

Шаг 10. Блок "Ввод горячих клавиш в веб-сайт" – с помощью `web_actions` нажимаем клавишу Enter в поле поиска. Это необходимо, чтобы применить поиск и отфильтровать таблицу по введённому имени.

Шаг 11. Если поле поиска не найдено – вызываем функцию `TextTooLog` с сообщением "Поле поиска не найдено на странице" для логирования проблемы.

Шаг 12. Если фильтр не найден – вызываем функцию `TextTooLog` с сообщением "Фильтр `Filter by Status` не найден на странице".



Шаг 13. Блок "Группировка блоков" – начало группы действий по проверке корректности результатов поиска.

Шаг 14. Блок "Извлечь табличные данные" – с помощью блока извлекаем все данные из таблицы заявок после применения фильтра и поиска. Блок ищет элемент tbody с идентификатором table_body. Полученные данные сохраняются в переменную table_name.

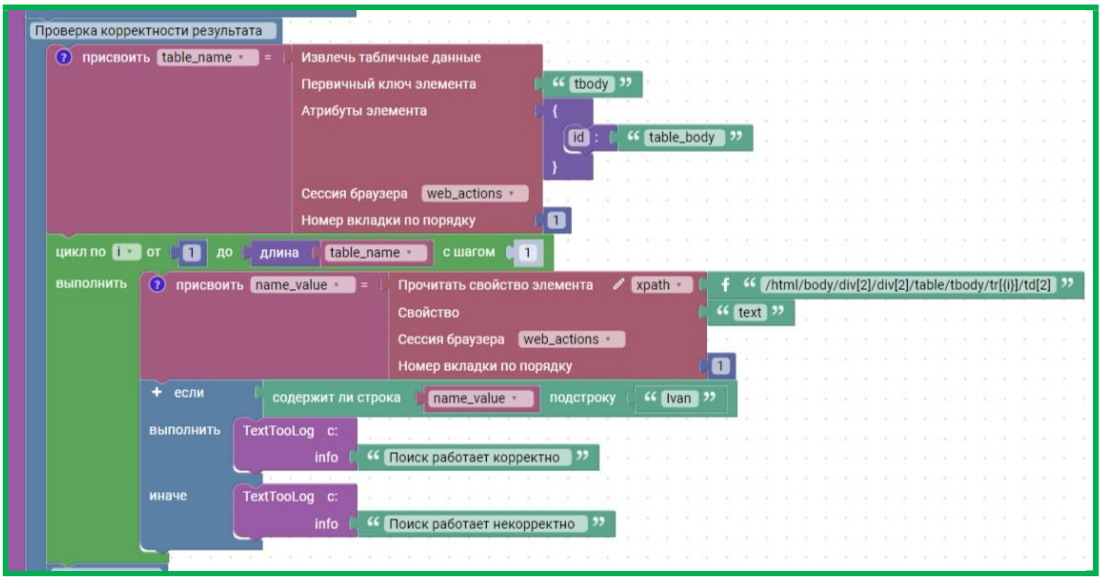
Шаг 15. Блок "Цикл по (от до с шагом)" – организуем цикл для i от 1 до количества строк в отфильтрованной таблице. Это необходимо, чтобы проверить каждую отображаемую заявку на соответствие критериям поиска.

Шаг 16. Блок "Прочитать свойство элемента" – с помощью блока считываем имя участника для текущей строки i. XPath для этого элемента – `"/html/body/div[2]/div[2]/table/tbody/tr[{i}]/td[2]"`. Вторая колонка таблицы (`td[2]`) содержит имя участника. {i} – это место, куда подставляется текущее значение переменной i из цикла. Извлекаемое свойтсво – «`text`». Полученное значение сохраняется в переменную `name_value`.

Шаг 17. Блок "Если – выполнить" – проверяем, содержит ли строка `name_value` подстроку `"Ivan"`. Для этого используем блок «содержит ли строка подстроку» которая ищет вхождение одной строки в другую.

Шаг 18. Если имя содержит `"Ivan"` – вызываем `TextTooLog` с сообщением "Поиск работает корректно". Это означает, что поиск по имени `"ivan"` нашёл соответствующего участника.

Шаг 19. Если имя не содержит `"Ivan"` – вызываем `TextTooLog` с сообщением "Поиск работает некорректно". Это означает, что поиск не дал ожидаемых результатов.

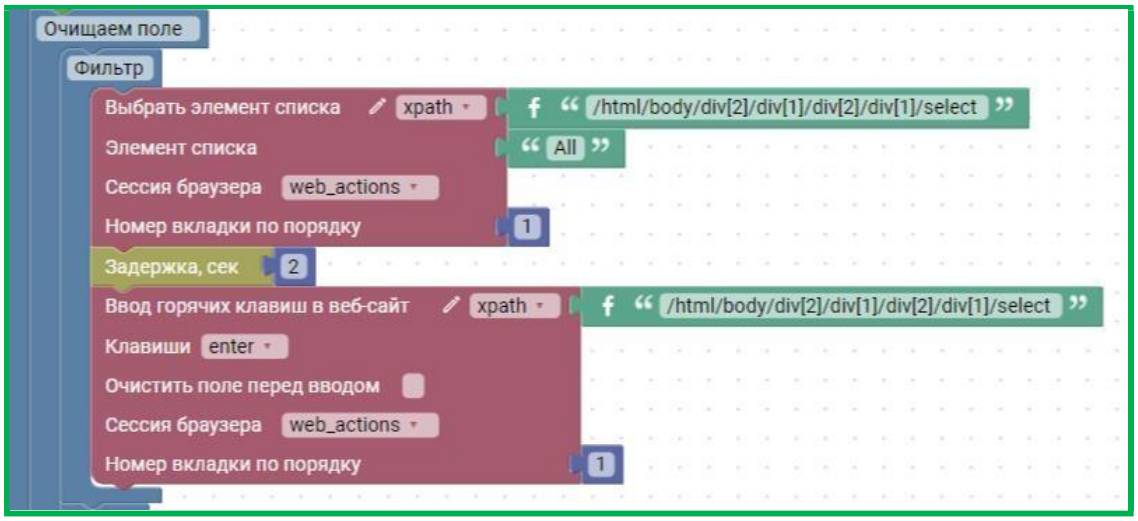


Шаг 20. Блок "Группировка блоков" – начало группы действий по очистке фильтра и поля поиска. Это необходимо, чтобы вернуть страницу в исходное состояние для последующих операций.

Шаг 21. Блок "Группировка блоков" (очистка фильтра) – с помощью блока «Выбрать элемент списка» выбираем в выпадающем списке значение `"All"`, чтобы сбросить фильтрацию по статусу и показать все заявки.

Шаг 22. Блок "Задержка" – пауза 2 секунды после выбора значения `"All"`.

Шаг 23. Блок "Ввод горячих клавиш в веб-сайт" – нажимаем клавишу Enter (код клавиши 1) в выпадающем списке, чтобы применить сброс фильтрации.



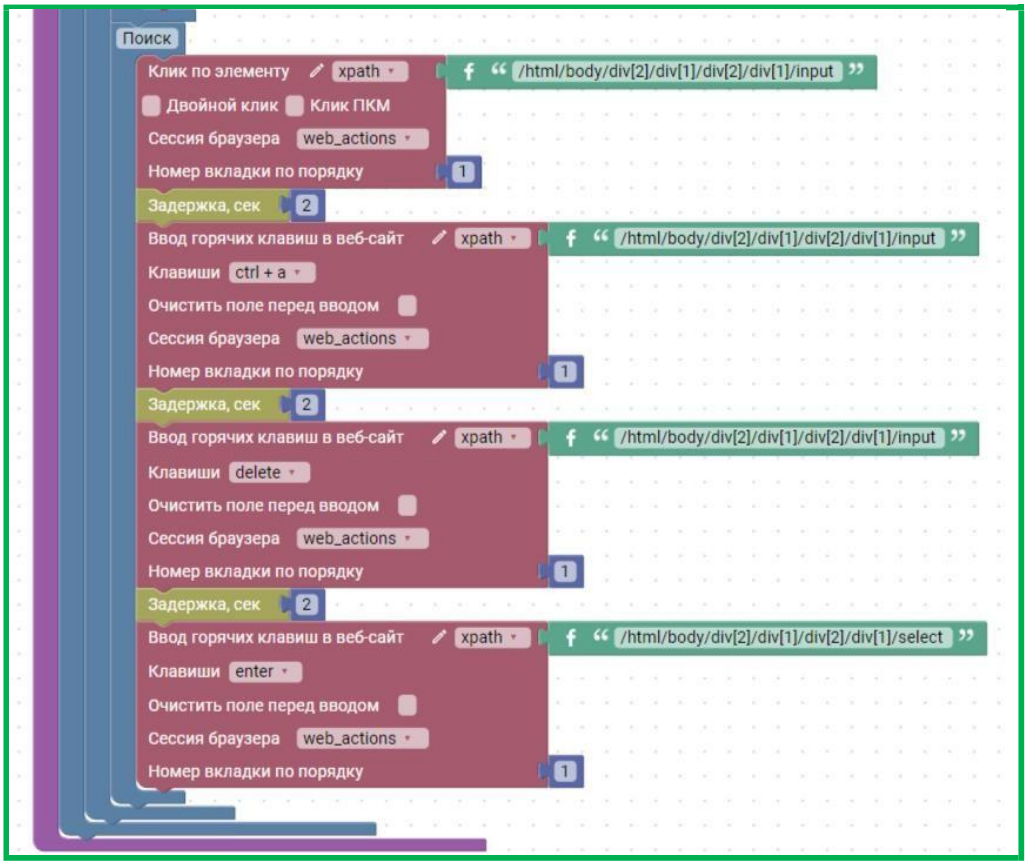
Шаг 24. Блок "Группировка блоков" (очистка поиска) – начинаем очистку поля поиска.

Шаг 25. Блок "Клик по элементу" – кликаем по полю поиска, чтобы установить в него фокус.

Шаг 26. Блок "Задержка" – пауза 2 секунды после клика.

Шаг 27. Блок "Ввод горячих клавиш в веб-сайт" – нажимаем сначала клавишу Ctr+A , затем Delete, и затем Enter, чтобы удалить введённый ранее текст.

Шаг 28. Блок "Задержка" – пауза 2 секунды после удаления текста.

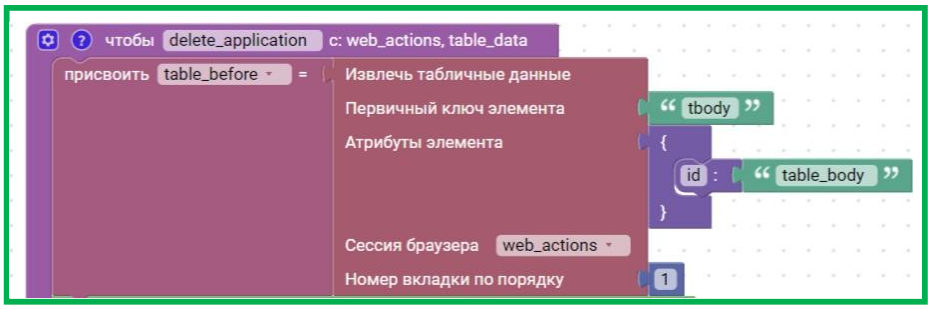


Таким образом, функция process_applications последовательно проверяет работу фильтрации по статусу и поиска по имени, логирует результаты проверки, а затем очищает все применённые фильтры и поисковые запросы, возвращая страницу в исходное состояние. Это позволяет использовать данную функцию для регулярного тестирования веб-приложения и убедиться, что его элементы управления работают корректно.

Функция `delete_application`

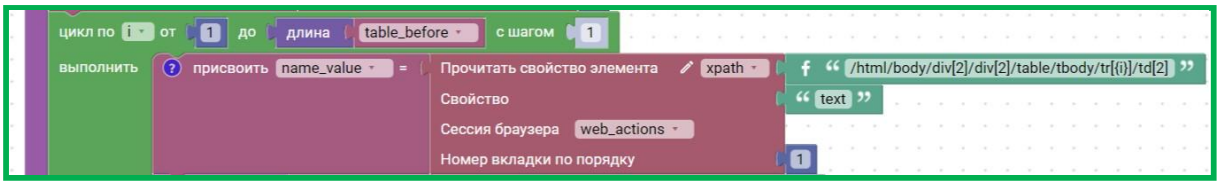
Функция `delete_application` предназначена для автоматического удаления выбранной записи из таблицы заявок. Она ищет запись с именем, содержащим подстроку `"Anna"`, выделяет её, нажимает кнопку `Delete Application`, а затем проверяет, что запись действительно исчезла из таблицы. На вход функция принимает два параметра: `web_actions` (сессия браузера, через которую осуществляется управление веб-страницей) и `table_data` (таблица с данными, полученная ранее от функции `update_status`). Параметр `table_data` используется для получения количества строк до удаления.

Шаг 1. Блок "Извлечь табличные данные" – с помощью переданной сессии `web_actions` извлекаем все данные из таблицы заявок после возможных предыдущих операций. Полученные данные сохраняются в переменную `table_before`. Это будет использоваться для сравнения количества строк до и после удаления.



Шаг 2. Блок "Цикл по (от до с шагом)" – организуем цикл для `i` от `1` до количества строк в таблице `table_after` включительно. Цикл будет проходить по всем заявкам, чтобы найти ту, которую нужно удалить.

Шаг 3. Блок "Прочитать свойство элемента" – считываем имя участника для текущей строки `i`. XPath для этого элемента – `"/html/body/div[2]/div[2]/table/tbody/tr[{i}]/td[2]"`. Вторая колонка таблицы (`td[2]`) содержит имя участника. Полученное значение сохраняется в переменную `name_value`. Обратите внимание, что здесь используется подстановка переменной `i`, что позволяет динамически обращаться к разным строкам таблицы.



Шаг 4. Блок "Условный оператор" (Если – выполнить) – проверяем, содержит ли строка `name_value` подстроку `"Anna"`. Для этого используется блок «Содержит ли строка подстроку, которая ищет вхождение одной строки в другую. Если условие истинно, значит, найдена запись, которую нужно удалить.

Шаг 5. Если имя содержит `"Anna"` – выполняем следующие действия. Сначала делаем паузу 2 секунды с помощью блока "Задержка". Это необходимо, чтобы страница стабилизировалась перед кликами.

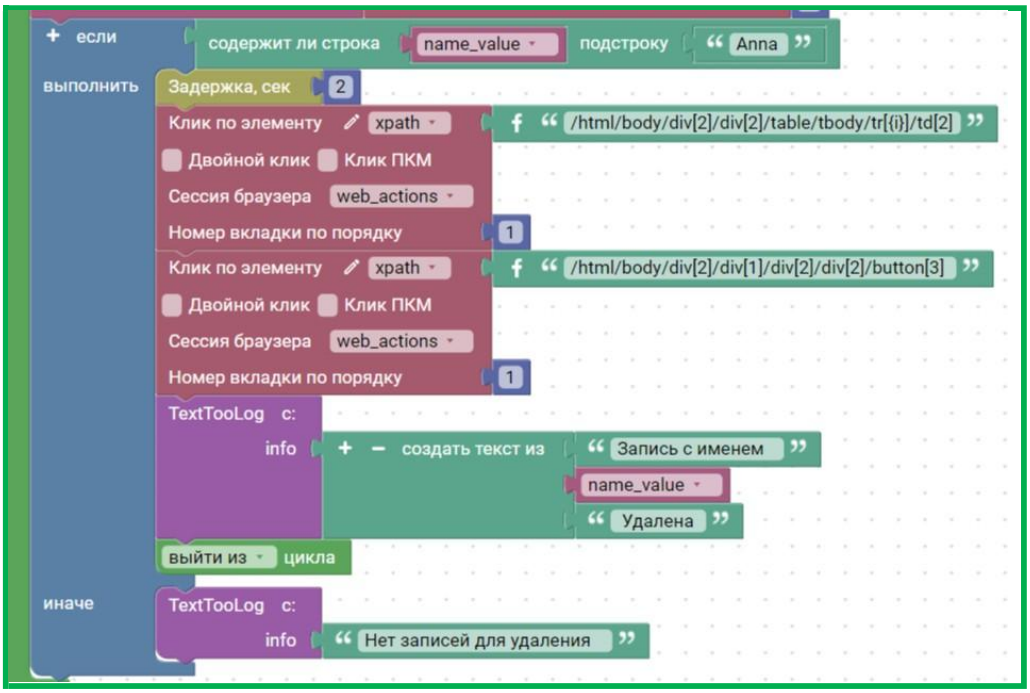
Шаг 6. Блок "Клик по элементу" – кликаем по ячейке с именем участника. XPath для этого элемента – `"/html/body/div[2]/div[2]/table/tbody/tr[{i}]/td[2]"`. Это действие выделяет строку таблицы, делая активными кнопки управления (`Mark as Approved`, `Mark as Rejected`, `Delete Application`).

Шаг 7. Блок "Клик по элементу" – кликаем по кнопке `Delete Application`. XPath для этой кнопки – `"/html/body/div[2]/div[1]/div[2]/div[2]/button[3]"`. Третья кнопка в группе управления соответствует удалению заявки.

Шаг 8. Блок "Исполнить функцию" – вызываем функцию `TextTooLog` с сообщением `"Запись с именем " + name_value + " Удалена"`. Это логирует факт успешного удаления.

Шаг 9. Блок "Выйти из цикла" – прерываем выполнение цикла с помощью оператора `break`. Нет смысла продолжать поиск, так как нужная запись уже найдена и удалена.

Шаг 10. Если имя не содержит `"Anna"` – вызываем функцию `TextTooLog` с сообщением "Нет записей для удаления". Это логирует ситуацию, когда запись с именем `Anna` не найдена в таблице.



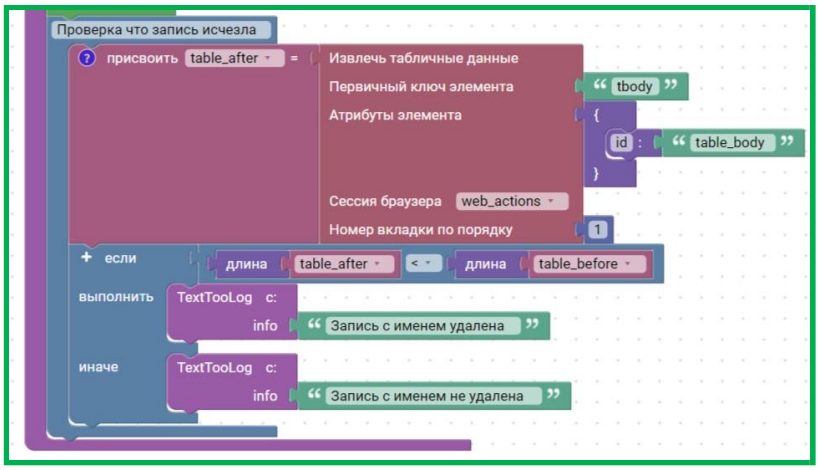
Шаг 11. Блок "Группировка блоков" – начало группы действий по проверке, что запись действительно исчезла.

Шаг 12. Блок "Извлечь табличные данные" – снова извлекаем все данные из таблицы заявок, но теперь после выполнения удаления. Результат сохраняется в переменную `table_after`.

Шаг 13. Блок "Условный оператор" (Если – выполнить) – сравниваем количество строк в таблице после удаления (`table_after`) с количеством строк до удаления (`table_before`). Если `len(table_after) < len(table_before)`, это означает, что количество строк уменьшилось, то есть удаление прошло успешно.

Шаг 14. Если количество строк уменьшилось – вызываем функцию `TextTooLog` с сообщением "Запись с именем удалена". Это подтверждает, что запись исчезла из таблицы.

Шаг 15. Если количество строк не уменьшилось – вызываем функцию TextTooLog с сообщением "Запись с именем не удалена". Это означает, что удаление не произошло, возможно, из-за того, что кнопка была недоступна или произошла ошибка.

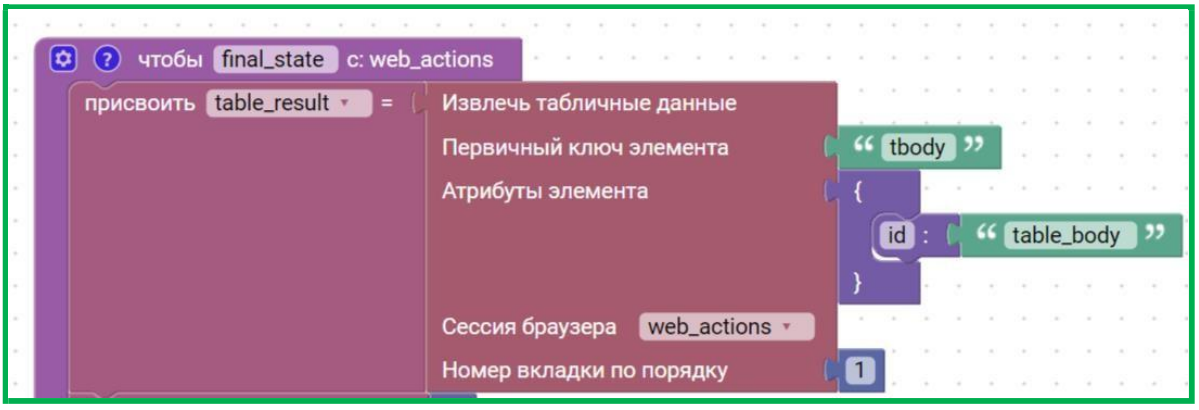


Таким образом, функция `delete_application` последовательно выполняет поиск записи с именем `Anna`, удаляет её, а затем проверяет, что количество строк в таблице уменьшилось. Все действия логируются с помощью функции `TextTooLog`, что позволяет отслеживать успешность выполнения операции. Использование цикла с досрочным выходом (`break`) после нахождения нужной записи делает функцию эффективной, так как она не обрабатывает все строки без необходимости.

Функция `final_state`

Функция `final_state` предназначена для финальной проверки итогового состояния таблицы заявок после выполнения всех операций робота. Она анализирует каждую запись в таблице, проверяет соответствие статусов ожидаемым значениям (`Online` → `Approved`, `Offline` → `Rejected`), подсчитывает количество заявок каждого формата, собирает ошибки в списки и выводит итоговое заключение о корректности данных. На вход функция принимает один параметр: `web_actions` (сессия браузера, через которую осуществляется управление веб-страницей).

Шаг 1. Блок "Извлечь табличные данные" – с помощью переданной сессии `web_actions` извлекаем все данные из итоговой таблицы заявок. Для этого используется блок извлечь табличные данные, который ищет элемент `tbody` с идентификатором `table_body`. Полученные данные сохраняются в переменную `table_result`. Эта переменная будет содержать все записи, которые остались в таблице после добавления, изменения статусов, фильтрации, поиска и удаления.

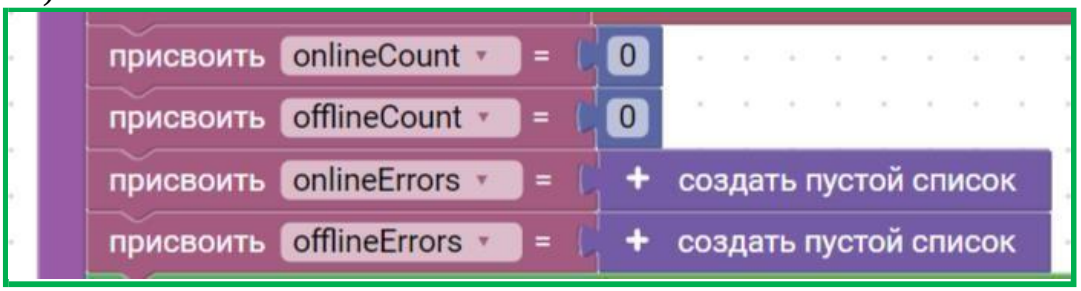


Шаг 2. Блок "Присвоить значение переменной" – создаём переменную `onlineCount` и присваиваем ей начальное значение 0. Эта переменная будет использоваться для подсчёта количества заявок с форматом участия `Online`.

Шаг 3. Блок "Присвоить значение переменной" – создаём переменную `offlineCount` и присваиваем ей начальное значение 0. Эта переменная будет использоваться для подсчёта количества заявок с форматом участия `Offline`.

Шаг 4. Блок "Создать пустой список" – создаём пустой список `onlineErrors`. В этот список будут добавляться сообщения об ошибках для заявок с форматом `Online`, у которых статус не соответствует ожидаемому (не `Approved`).

Шаг 5. Блок "Создать пустой список " – создаём пустой список `offlineErrors`. В этот список будут добавляться сообщения об ошибках для заявок с форматом `Offline`, у которых статус не соответствует ожидаемому (не `Rejected`).



Шаг 6. Блок "Цикл по (от до с шагом)" – организуем цикл для `i` от 1 до количества строк в итоговой таблице включительно. Цикл будет проходить по всем заявкам, которые остались в таблице после всех операций робота.

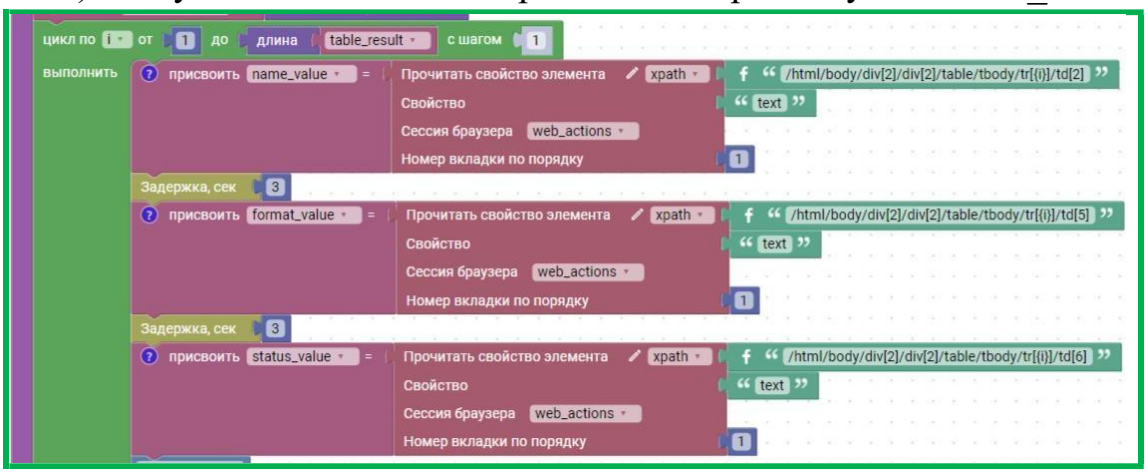
Шаг 7. Блок "Прочитать свойство элемента" – с помощью этого блока считываем имя участника для текущей строки i. XPath для этого элемента – `"/html/body/div[2]/div[2]/table/tbody/tr[{i}]/td[2]"`. Подставляем переменную в наш XPath. Вторая колонка таблицы (`td[2]`) содержит имя участника. Полученное значение сохраняется в переменную `name_value`. Это имя будет использоваться при формировании сообщений об ошибках, чтобы было понятно, по какой именно заявке обнаружена проблема.

Шаг 8. Блок "Задержка" – пауза продолжительностью 3 секунды. Это необходимо для того, чтобы веб-приложение успело стабилизироваться между операциями чтения данных.

Шаг 9. Блок "Прочитать свойство элемента" – с помощью этого блока считываем значение формата участия для текущей строки `i`. XPath для этого элемента – `"/html/body/div[2]/div[2]/table/tbody/tr[{i}]/td[5]"`. Пятая колонка таблицы (`td[5]`) соответствует формату участия (`Online` или `Offline`). Полученное значение сохраняется в переменную `format_value`.

Шаг 10. Блок "Задержка" – пауза продолжительностью 3 секунды для стабилизации перед чтением следующего элемента.

Шаг 11. Блок "Прочитать свойство элемента" – с помощью этого блока считываем значение статуса для текущей строки `i`. XPath для этого элемента – `"/html/body/div[2]/div[2]/table/tbody/tr[{i}]/td[6]"`. Шестая колонка таблицы (`td[6]`) соответствует статусу заявки (`Pending`, `Approved` или `Rejected`). Полученное значение сохраняется в переменную `status_value`.

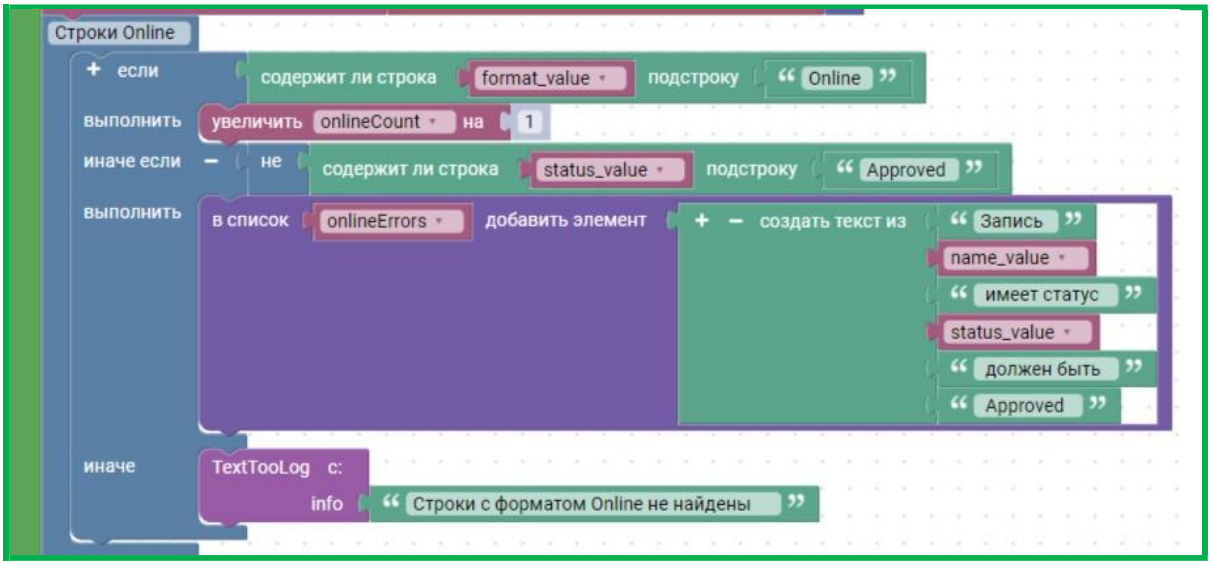


Шаг 12. Блок "Группировка блоков" – начало группы действий по обработке заявок с форматом `Online`.

Шаг 13. Блок "Условный оператор" (Если – выполнить) – проверяем, содержит ли строка `format_value` подстроку `"Online"`. Для этого используется блок содержит ли строка подстроку. Если условие истинно, значит, текущая заявка имеет формат `Online`.

Шаг 14. Если формат равен `Online` – увеличиваем счётчик `onlineCount` на 1.

Шаг 15. Если формат равен `Online`, но статус не равен `Approved` – проверяем, не содержит ли строка `status_value` подстроку `"Approved"`. Если статус не соответствует ожидаемому, формируем сообщение об ошибке: `"Запись [имя] имеет статус [статус] должен быть Approved"`. Это сообщение добавляется в список `onlineErrors` с помощью метода `append`. Таким образом, в списке собираются все проблемные заявки с форматом `Online`.



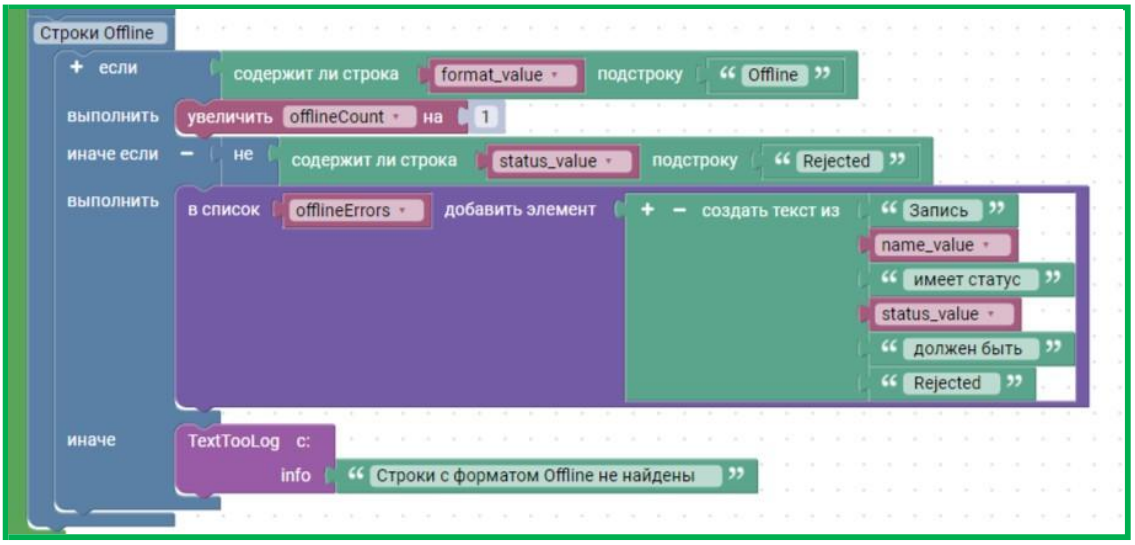
Шаг 16. Если формат не равен `Online` – проверяем следующее условие для формата `Offline`.

Шаг 17. Блок "Группировка блоков" – начало группы действий по обработке заявок с форматом `Offline`.

Шаг 18. Блок "Условный оператор" (Если – выполнить) – проверяем, содержит ли строка format_value подстроку `"Offline"`. Если условие истинно, значит, текущая заявка имеет формат `Offline`.

Шаг 19. Если формат равен `Offline` – увеличиваем счётчик `offlineCount` на 1.

Шаг 20. Если формат равен `Offline`, но статус не равен `Rejected` – проверяем, не содержит ли строка `status_value` подстроку `"Rejected"`. Если статус не соответствует ожидаемому, формируем сообщение об ошибке: `"Запись [имя] имеет статус [статус] должен быть Rejected"`. Это сообщение добавляется в список `offlineErrors`.



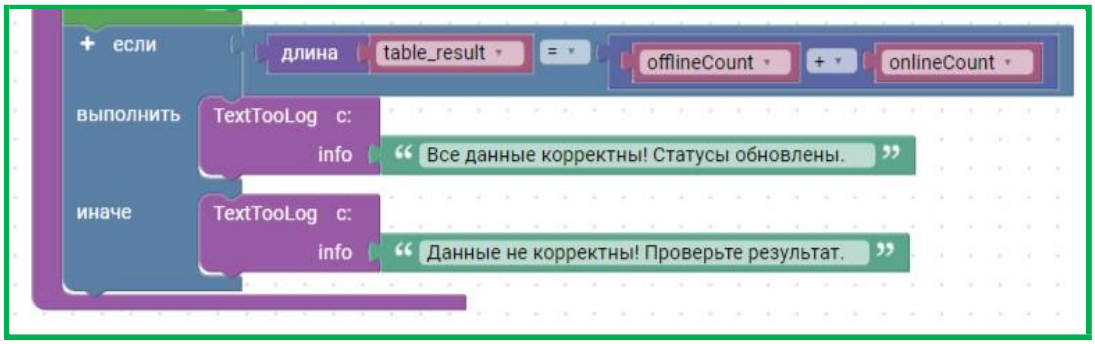
Шаг 21. Завершение цикла – после обработки всех строк таблицы цикл завершается. Теперь переменные `onlineCount` и `offlineCount` содержат общее количество заявок каждого формата, а списки `onlineErrors` и `offlineErrors` содержат описания всех обнаруженных ошибок.

Шаг 22. Блок "Условный оператор" (Если – выполнить) – проверяем, равно ли общее количество строк в таблице `(len(table_result))` сумме `onlineCount` и `offlineCount`. Если условие истинно, это означает, что все строки были классифицированы либо как `Online`, либо как `Offline`, и нет строк с другими форматами или неопределённым статусом.

Шаг 23. Если все строки классифицированы корректно – вызываем функцию `TextTooLog` с сообщением `"Все данные корректны! Статусы обновлены."` Это означает, что проверка прошла успешно, и все заявки имеют правильные статусы.

Шаг 24. Если есть проблемы с классификацией – вызываем функцию `TextTooLog` с сообщением `"Данные не корректны! Проверьте резуль-`

тат." Это означает, что количество строк не соответствует сумме подсчитанных заявок, что может указывать на наличие незаполненных или повреждённых записей.



Таким образом, функция `final_state` выполняет комплексную проверку итогового состояния таблицы заявок. Она подсчитывает количество заявок каждого формата, проверяет соответствие статусов ожидаемым значениям, собирает все ошибки и выводит итоговое заключение. Эта функция является важным элементом контроля качества, так как позволяет автоматически убедиться, что робот выполнил все операции корректно, и в таблице не осталось заявок с неправильными статусами.

Главный процесс Web_2

Главный процесс является оркестратором всего RPA-сценария. Он последовательно вызывает все ранее созданные функции, каждая из которых отвечает за отдельный этап автоматизации. Это позволяет структурировать код, сделать его понятным и удобным для сопровождения. В случае возникновения ошибки на любом этапе, процесс перехватывает исключение, логирует его и корректно завершает работу робота.

Шаг 1. Вызываем функцию `open_browser` с передачей URL веб-приложения `"http://localhost:8000/index.html"`. Функция `open_browser` инициализирует браузер Chrome, открывает указанную страницу и возвращает объект `web_actions`, который содержит все методы для взаимодействия с веб-страницей (клики, ввод текста, ожидание элементов и так далее). Полученный объект сохраняется в переменную `web_actions` для использования на последующих шагах.

Шаг 2. Вызываем функцию `autorisation` с передачей логина `"emsk222"`, пароля `"123456"` и переменной `web_actions`. Функция autorisation выполняет авторизацию в веб-приложении: вводит логин и пароль, нажимает кнопку входа и возвращает результат авторизации. Результат сохраняется в переменную `result_autorisation`. В случае успешной авторизации робот переходит на страницу подачи заявок.

Шаг 3. Вызываем функцию `read_excel` с передачей имени файла `"applications.xlsx"`. Функция `read_excel` читает данные из Excel-файла, который содержит тестовые заявки (имена участников, `email`, названия меропри-

ятий и форматы участия). Прочитанные данные сохраняются в переменную `table` в виде табличной структуры, готовой для дальнейшей обработки.

Шаг 4. Вызываем функцию `new_request` с передачей переменных `table` и `web_actions`. Функция `new_request` организует цикл по всем строкам таблицы, извлекает данные каждой заявки и последовательно заполняет форму на веб-странице: вводит имя участника, `email`, название мероприятия, выбирает формат участия (`Online` или `Offline`), отмечает чекбокс `Accept Terms` и нажимает кнопку `Submit Application`. Таким образом, все тестовые заявки добавляются в систему.

Шаг 5. Вызываем функцию `update_status` с передачей переменной `web_actions`. Функция `update_status` извлекает текущую таблицу заявок, анализирует формат участия каждой заявки (`Online` или `Offline`) и в зависимости от этого нажимает соответствующую кнопку: `Mark` as `Approved` для онлайн-заявок или `Mark` as `Rejected` для офлайн-заявок. Обновлённые данные таблицы сохраняются в переменную `table_data`.

Шаг 6. Вызываем функцию `process_applications` с передачей переменной `web_actions`. Функция `process_applications` проверяет работу фильтрации и поиска: применяет фильтр по статусу `Approved`, выполняет поиск по имени `"ivan"`, проверяет корректность результатов, а затем очищает фильтр и поле поиска. Это необходимо для тестирования функциональности веб-приложения.

Шаг 7. Вызываем функцию `delete_application` с передачей переменных `web_actions` и `table_data`. Функция `delete_application` ищет в таблице заявку с именем, содержащим подстроку `"Anna"`, выделяет её, нажимает кнопку `Delete Application` и проверяет, что запись действительно исчезла из таблицы. Удаление записи имитирует действие пользователя по очистке ошибочных или дублирующихся данных.

Шаг 8. Вызываем функцию `final_state` с передачей переменной `web_actions`. Функция `final_state` выполняет финальную проверку итогового состояния таблицы: подсчитывает количество заявок каждого формата, проверяет соответствие статусов ожидаемым значениям (`Online` → `Approved`, `Offline` → `Rejected`), собирает ошибки и выводит итоговое заключение о корректности данных.

Шаг 9. Вызываем функцию TextTooLog с сообщением `"Робот завершил работу."` Это логирует успешное окончание выполнения сценария. В лог-файл записывается временная метка и сообщение о завершении, что позволяет отследить, когда именно робот закончил свою работу.

Таким образом, главный процесс реализует полный цикл автоматизации: от открытия браузера и авторизации до добавления заявок, изменения статусов, проверки фильтрации и поиска, удаления записи и финальной валидации результатов.

Все этапы логируются, а ошибки обрабатываются централизованно. Такая структура обеспечивает надёжность и прозрачность работы робота.

В рамках данной практической работы была успешно реализована автоматизация процесса обработки заявок на участие в образовательных программах с использованием RPA-робота в среде Puzzle RPA.

В ходе выполнения робота были успешно добавлены все тестовые заявки из Excel-файла. Статусы заявок были корректно обновлены: все заявки с форматом Online получили статус Approved, все заявки с форматом Offline получили статус Rejected. Фильтрация и поиск отработали корректно. Одна запись была успешно удалена. Финальная проверка подтвердила, что все данные корректны и статусы обновлены.

Разработанный RPA-робот полностью заменяет ручной труд при вводе и обработке заявок. Он исключает ошибки, связанные с человеческим фактором, такие как опечатки, пропуски полей или неправильный выбор формата участия. Робот автоматически присваивает статусы в зависимости от формата участия, выполняет фильтрацию и поиск, удаляет ошибочные записи и значительно ускоряет процесс обработки данных. Расширенное логирование позволяет отслеживать все этапы выполнения и быстро выявлять проблемы в случае сбоев.

Практическая часть

Кейс: «Автоматизация расследования и закрытия инцидентов безопасности»

📖 **Ситуация**

После запуска базового портала регистрации инцидентов компания **SecureIT Pro** столкнулась с необходимостью управления жизненным циклом расследований:

- необходимо начинать расследование после классификации инцидента;
- закрывать инциденты после устранения угрозы;
- отслеживать статус выполнения работ.

В связи с этим было принято решение:

- расширить функциональность веб-приложения;
- реализовать более сложный сценарий автоматизации с использованием **Puzzle RPA**.

Задание 1. Расширение функциональности веб-приложения

Необходимо доработать существующее веб-приложение, добавив новые элементы управления и логику обработки инцидентов.

Каждый элемент интерфейса должен иметь уникальный id

⭕ **Новые кнопки управления**

Start Investigation

Назначение: Начало расследования инцидента

Логика:
{ "status": "Investigating" }
Требования:

- доступна только если Status = "Reported"
- при нажатии изменяет статус выбранной записи на "Investigating"

Close Incident
Назначение: Закрытие инцидента после устранения угрозы
Логика:
{ "status": "Resolved" }
Требования:

- доступна только если Status ≠ "Resolved"
- при нажатии изменяет статус выбранной записи на "Resolved"

⚙ Управление состоянием элементов (disabled)

Кнопки Start Investigation, Close Incident
Должны быть:

- disabled = true, если:
  - ни одна строка не выбрана
- disabled = false, если:
  - выбрана строка таблицы

Дополнительные условия

- Start Investigation должна быть неактивна, если:
  { "status": "Investigating" }
  или
  { "status": "Resolved" }
- Close Incident должна быть неактивна, если:
  { "status": "Resolved" }

○ Детализация кнопок управления

Start Investigation
Назначение: Начало расследования после классификации инцидента.
Бизнес-смысл: Специалист по безопасности приступил к анализу инцидента.

Когда используется:

- инцидент прошёл первичную классификацию;
- определён уровень критичности.

Алгоритм работы:

1. Пользователь выбирает строку в таблице.
2. Проверяется текущий статус инцидента:
    - если Status ≠ Reported → операция запрещена;
    - если Status = Reported → операция разрешена.
3. При нажатии кнопки:
    - статус изменяется на:

    { "status": "Investigating" }
4. Таблица обновляется (с задержкой, если реализована асинхронность).

Ограничения:

- нельзя повторно начать уже расследуемый инцидент;
- кнопка должна быть disabled, если:
    - не выбрана строка;
    - статус уже Investigating или Resolved.

Close Incident
Назначение: Закрытие инцидента после устранения угрозы.
Бизнес-смысл: Угроза нейтрализована, инцидент архивируется.

Когда используется:

- проведён полный анализ инцидента;
- приняты меры по устранению уязвимости.

Алгоритм работы:

1. Выбирается строка таблицы.
2. Проверяется статус:
    - допускается любой статус кроме Resolved.
3. При нажатии:
    - статус изменяется на:

    { "status": "Resolved" }
4. Таблица обновляется.

Ограничения:

- нельзя закрыть уже закрытый инцидент;
- кнопка disabled, если:
    - нет выбранной строки;
    - статус уже Resolved.

🔍 **Фильтрация инцидентов**
Необходимо добавить блок фильтрации:
Filter by Severity
Значения:

- All
- Low
- Medium
- High
- Critical

Логика

- при выборе значения отображаются только соответствующие записи
- фильтрация выполняется без перезагрузки страницы

🔍 Детализация фильтрации Filter by Severity

📌 Назначение

Фильтр позволяет отображать инциденты в зависимости от уровня критичности.

📊 Значения критичности и их смысл

Критичность	Значение	Когда назначается
Low	Низкая критичность	при незначительных нарушениях
Medium	Средняя критичность	при стандартных инцидентах
High	Высокая критичность	при серьёзных угрозах
Critical	Критическая	при активных атаках

⚙️ Алгоритм работы фильтра

---

1. Выбор значения

Пользователь выбирает значение в выпадающем списке:
Filter by Severity

---

2. Обработка выбора

Если выбрано:

All

- отображаются все записи без фильтрации

Low

- отображаются только записи:
  { "severity": "Low" }

Medium

- отображаются только:
  { "severity": "Medium" }

High

- отображаются только:
  { "severity": "High" }

Critical

- отображаются только:
  { "severity": "Critical" }

---

3. Применение фильтра

- фильтрация выполняется:
  - без перезагрузки страницы;
  - на стороне клиента (или сервера — опционально);
- обновление таблицы происходит с задержкой (имитация реального API).

---

4. Поведение при отсутствии данных

Если записи отсутствуют:

- отображается сообщение в текстовой метке:
  No incidents found

---

5. Взаимодействие с другими элементами

✓ После изменения статуса

- запись может "исчезнуть" из текущего фильтра (например, была Reported, стала Resolved)

✓ После удаления

- таблица автоматически обновляется с учётом текущего фильтра

🎯 Итоговая логика состояний

Регистрация инцидента → Reported
↓
Investigating
↓
Resolved

🔎 Поиск

Search by Title

Назначение: Поиск инцидента по заголовку

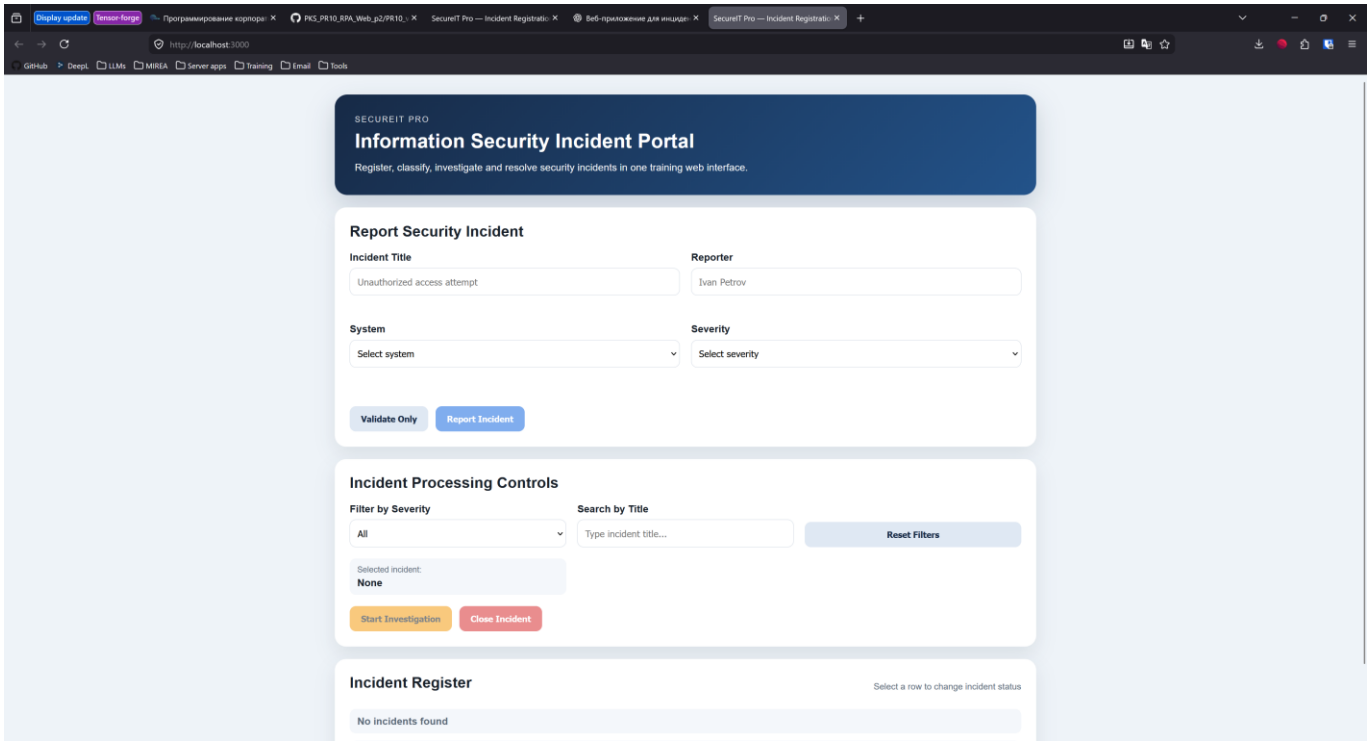
Требования:

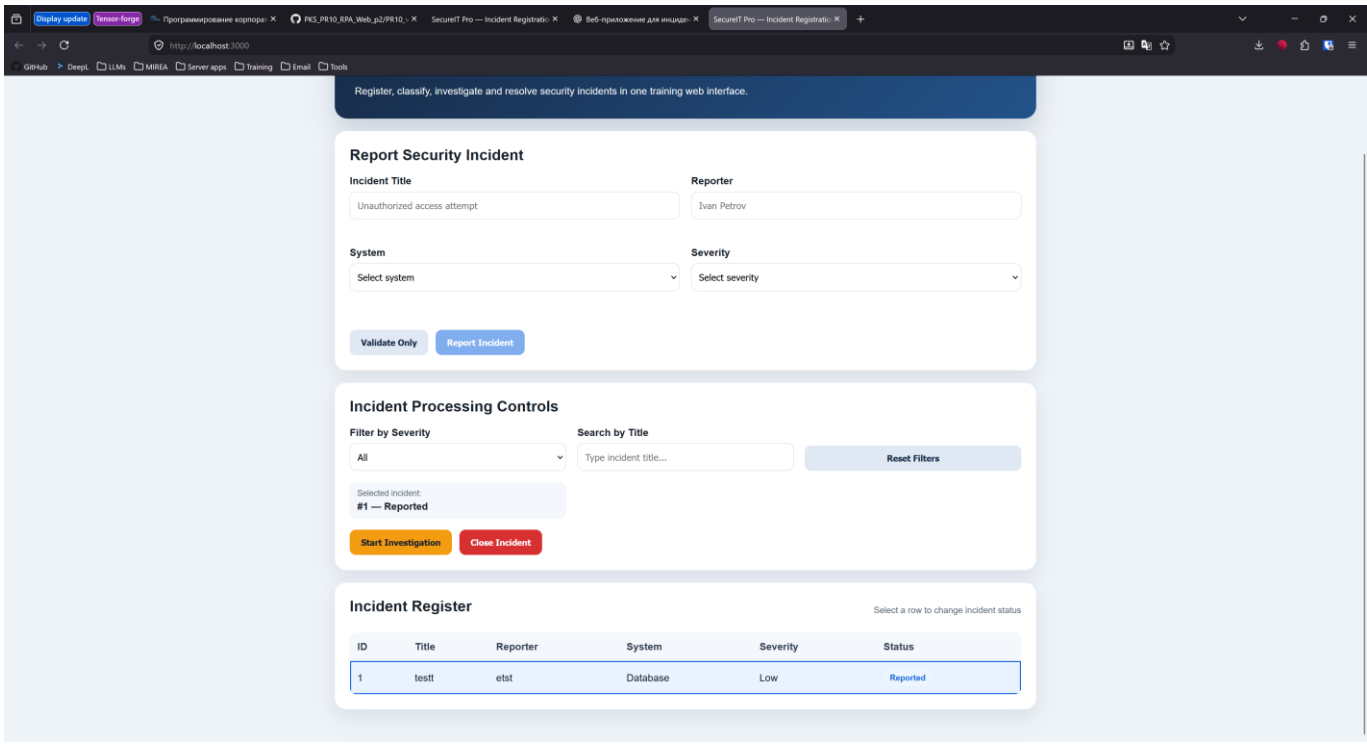
- поиск выполняется в реальном времени
- частичное совпадение допустимо

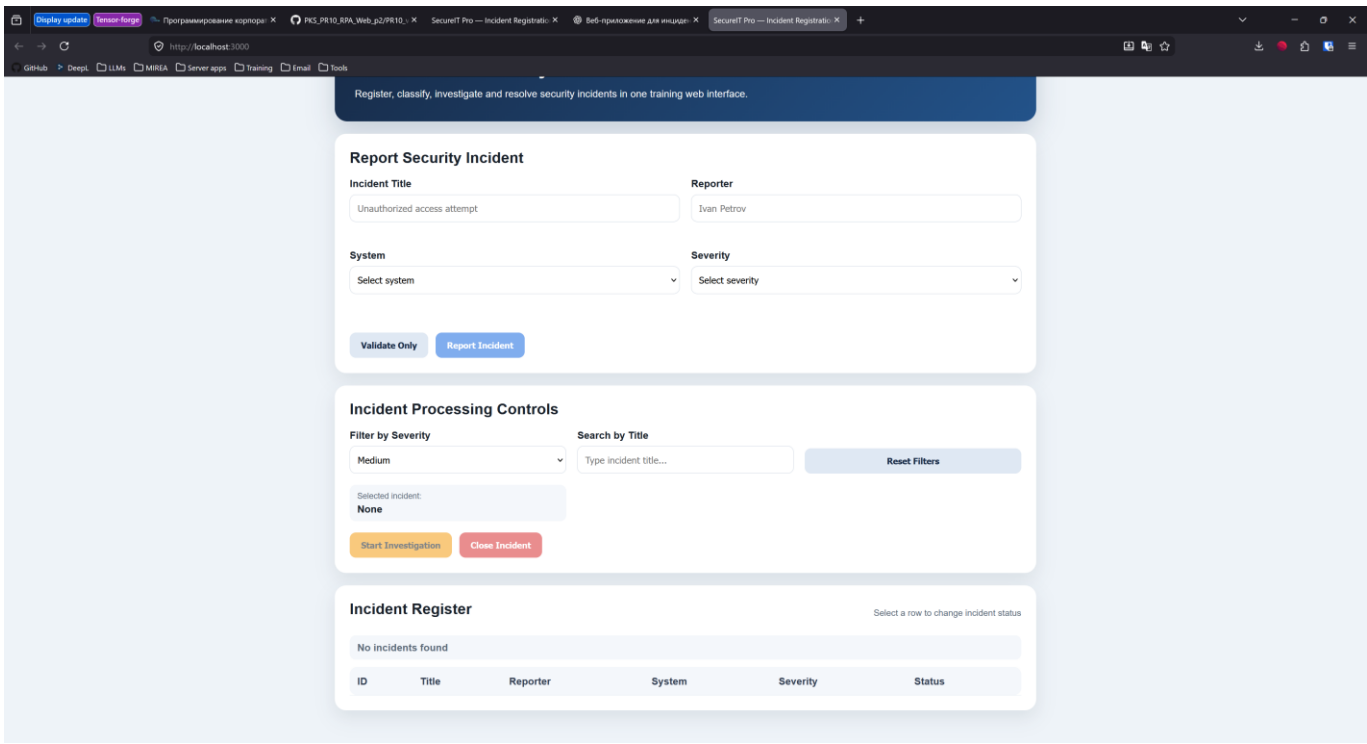
⏳ Дополнительная асинхронность

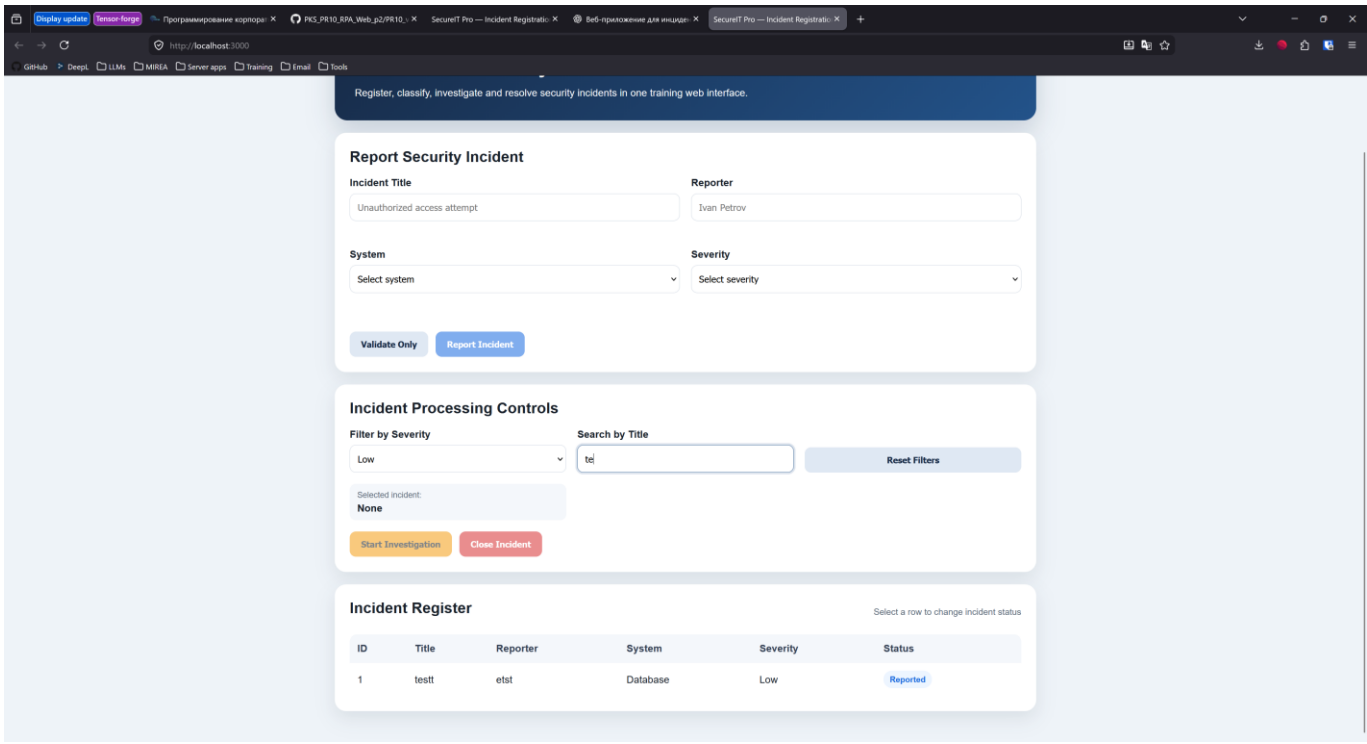
Необходимо реализовать:

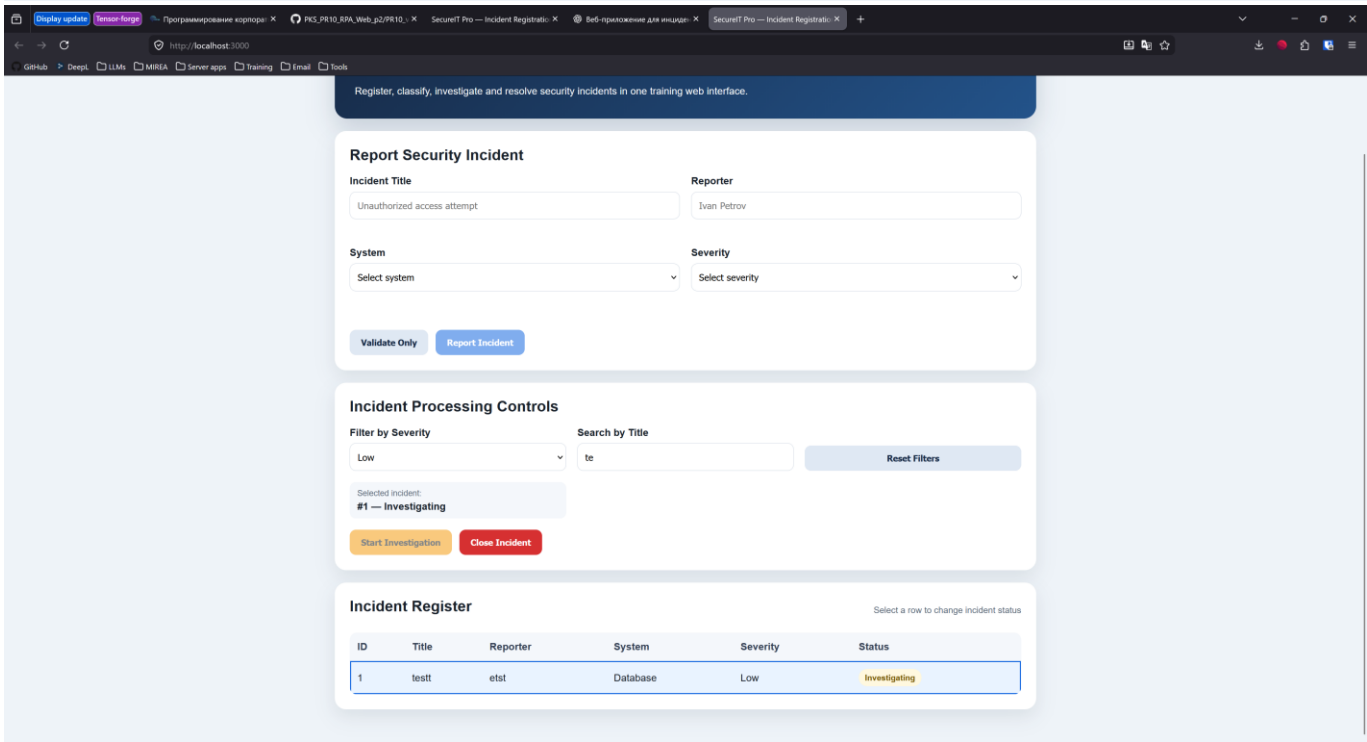
- задержку обновления таблицы после изменения статуса (1–2 сек)
- задержку при закрытии инцидента
- имитацию загрузки (Loading...)




Register, classify, investigate and resolve security incidents in one training web interface.
Report Security Incident
Incident Title
Unauthorized access attempt
Reporter
Ivan Petrov
System
Select system
Severity
Select severity
Validate Only
Report Incident
Incident Processing Controls
Filter by Severity
All
Search by Title
Type incident title...
Reset Filters
Selected incident:
#1 — Reported
Start Investigation
Close Incident
Incident Register
Select a row to change incident status
ID
Title
Reporter
System
Severity
Status
1
testt
etst
Database
Low
Reported


Register, classify, investigate and resolve security incidents in one training web interface.
Report Security Incident
Incident Title
Unauthorized access attempt
Reporter
Ivan Petrov
System
Select system
Severity
Select severity
Validate Only
Report Incident
Incident Processing Controls
Filter by Severity
Medium
Search by Title
Type incident title...
Reset Filters
Selected incident:
None
Start Investigation
Close Incident
Incident Register
Select a row to change incident status
No incidents found
ID
Title
Reporter
System
Severity
Status





Задание 2. Разработка продвинутого RPA-сценария

Необходимо реализовать процесс автоматизации, обрабатывающий инциденты в веб-приложении.

🤖 Алгоритм работы робота

1. Подготовка

- открыть браузер
- перейти на сайт
- дождаться загрузки страницы

---

2. Добавление инцидентов

Для каждой записи:

- заполнить форму
- проверить Report Incident.disabled
- при необходимости исправить данные
- отправить форму
- дождаться сообщения об успехе в текстовой метке

---

3. Начало расследования всех инцидентов

После добавления:

- для каждой записи в таблице:
  - выбрать строку
  - проверить доступность Start Investigation
  - изменить статус на:

{ "status": "Investigating" }

---

4. Закрытие инцидентов с критическим уровнем

- найти инциденты с Severity = Critical
- для каждой найденной записи:
  - выбрать строку
  - проверить доступность Close Incident
  - изменить статус на:

{ "status": "Resolved" }

---

5. Использование фильтра

- выбрать Filter by Severity = Critical
- убедиться, что отображаются только нужные записи

---

6. Поиск

- выполнить поиск по заголовку (например: "Unauthorized")
- проверить корректность результата

---

7. Проверка итогового состояния

- проверить:
  - все инциденты добавлены (10 записей)
  - все записи имеют статус "Investigating" или "Resolved"
  - инциденты с Severity = Critical имеют статус "Resolved"

📊 **Ожидаемый результат**

✓ **После выполнения**

- добавлены все записи
- статусы обновлены:
  Critical → Resolved

📋 **Пример итоговой таблицы**

D

Title	Severity	Status
Unauthorized access attempt	Critical	Resolved
Suspicious database query	High	Investigating
Phishing email detected	Medium	Investigating

🔎 **Требования к корректности выполнения**

✓ **Веб-приложение**

- корректно реализованы:
  - фильтрация
  - поиск
  - управление disabled

✓ **RPA-сценарий**

- корректно работает с динамическими элементами
- использует ожидания (Wait)
- обрабатывает фильтрацию и поиск

✓ **Данные**

- все записи обработаны
- статусы корректны
- отсутствуют дубликаты

📜 **Логирование**

Необходимо реализовать расширенное логирование

📌 **Должно фиксироваться**

- запуск процесса
- добавление записей
- начало расследования
- закрытие инцидентов

- применение фильтра
- поиск
- ошибки

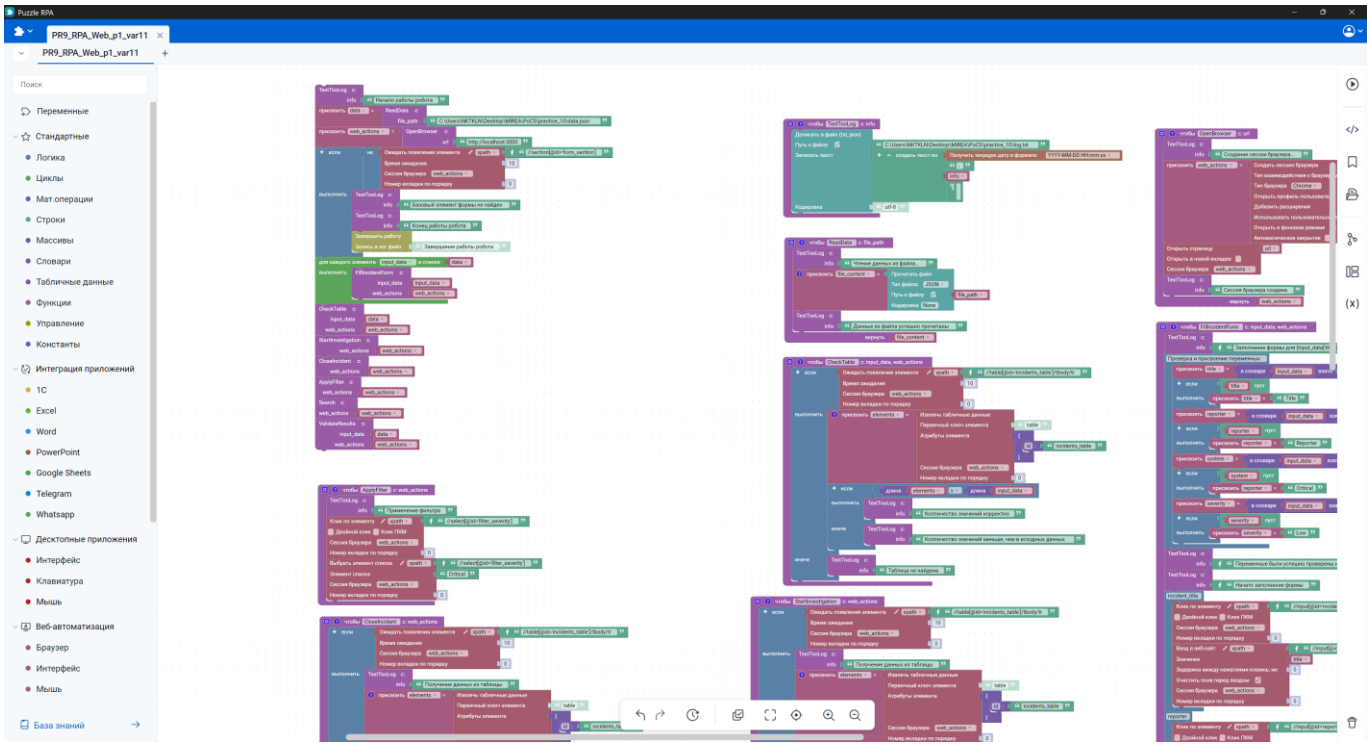
Пример

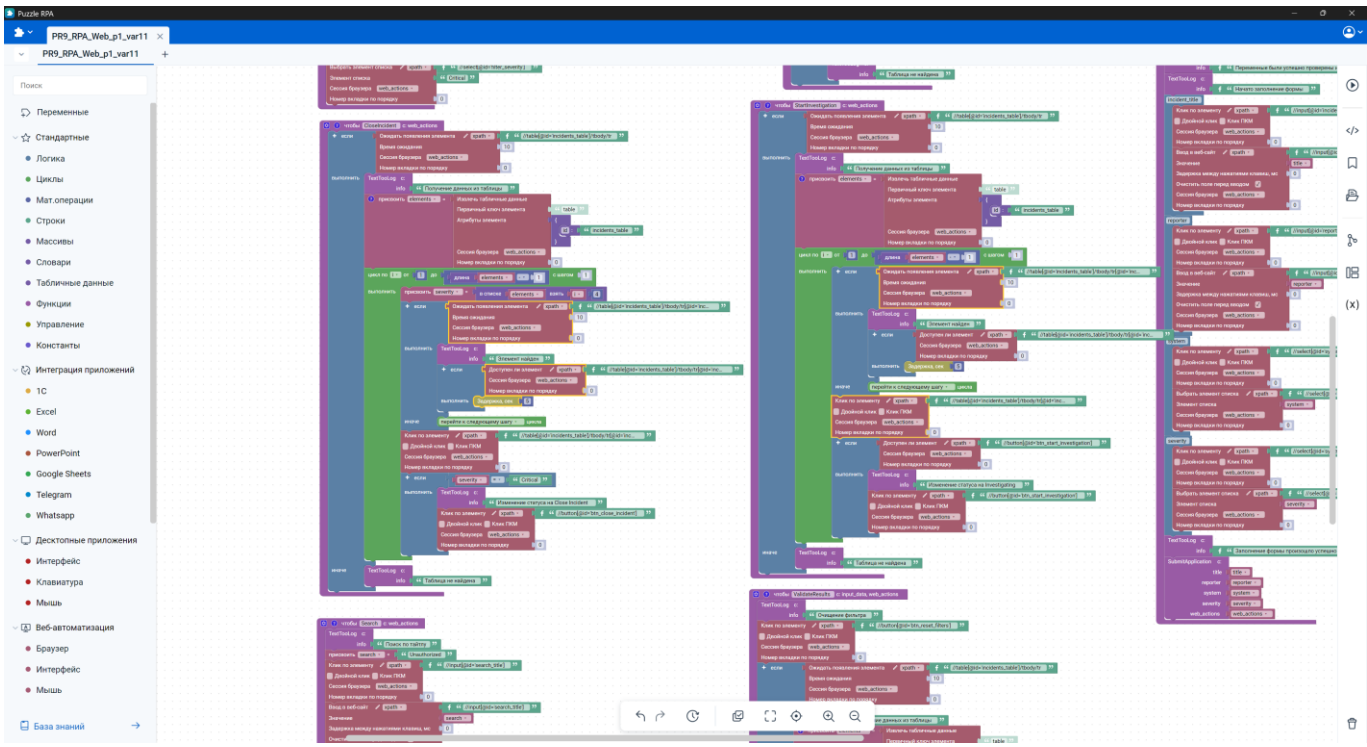

```
{
 "time": "2026-04-03 11:45:02",
 "step": "CloseIncident",
 "title": "Unauthorized access attempt",
 "severity": "Critical",
 "new_status": "Resolved",
 "result": "success"
}
```

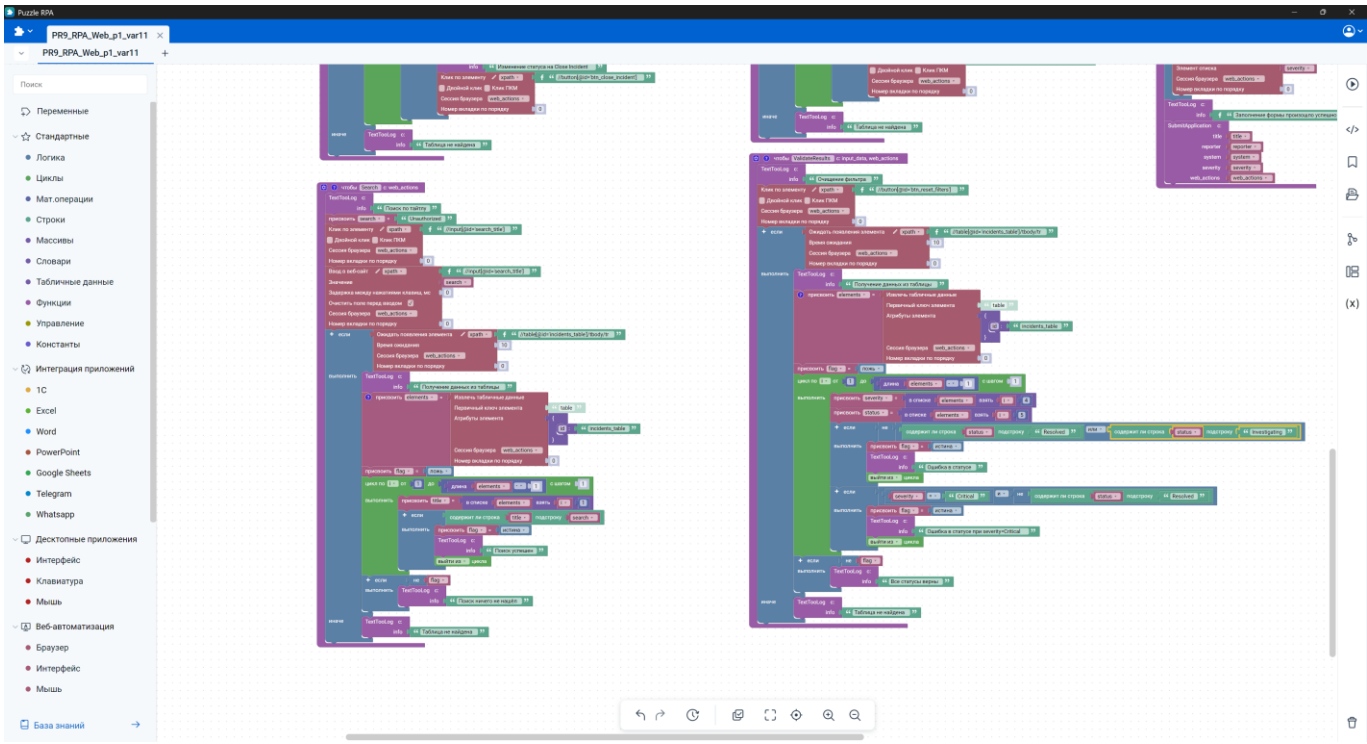
Требования к модульности

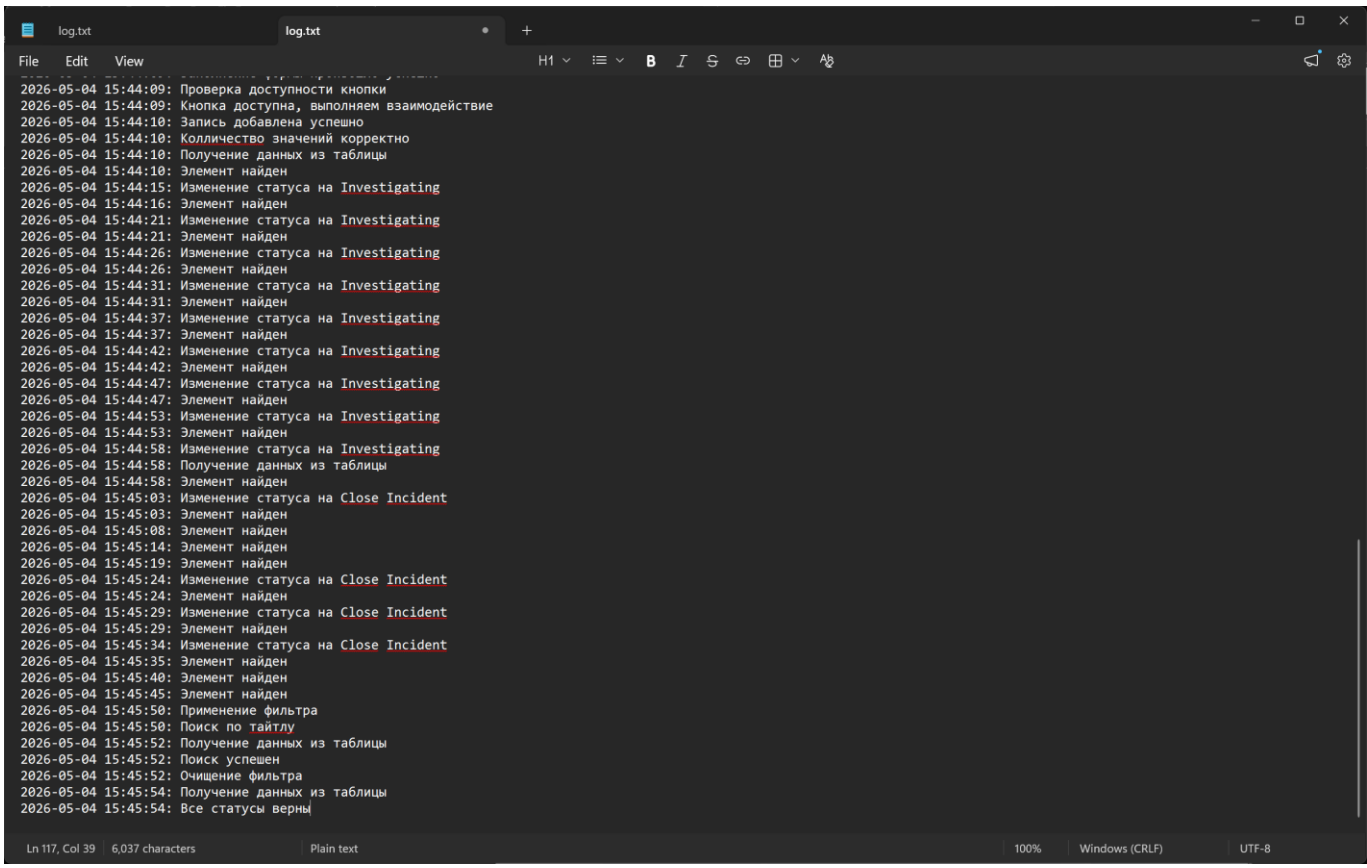
Рекомендуется выделить подпроцессы:

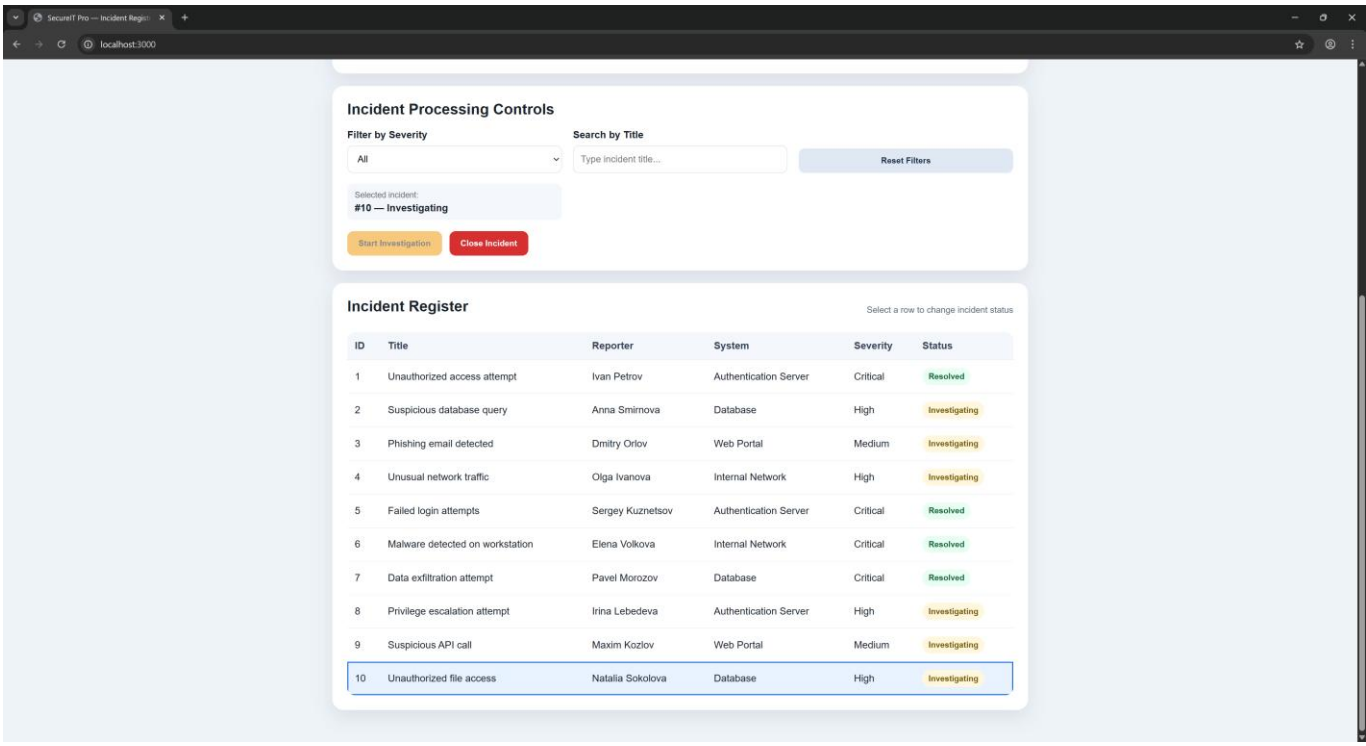
- OpenBrowser
- FillForm
- SubmitApplication
- StartInvestigation
- CloseIncident
- ApplyFilter
- Search
- HandleModal
- ValidateResults


Puzzle RPA
PR9_RPA_Web_p1_var11
Переменные
Стандартные
Логика
Циклы
Мат.операции
Строки
Массивы
Словари
Табличные данные
Функции
Управление
Константы
Интеграция приложений
1С
Excel
Word
PowerPoint
Google Sheets
Telegram
Whatsapp
Десктопные приложения
Интерфейс
Клавиатура
Мышь
Веб-автоматизация
Браузер
Интерфейс
Мышь
База знаний


Puzzle RPA
PR9_RPA_Web_p1_var11
Переменные
Стандартные
Логика
Циклы
Мат.операции
Строки
Массивы
Словари
Табличные данные
Функции
Управление
Константы
Интеграция приложений
1С
Excel
Word
PowerPoint
Google Sheets
Telegram
Whatsapp
Десктопные приложения
Интерфейс
Клавиатура
Мышь
Веб-автоматизация
Браузер
Интерфейс
Мышь
База знаний


Puzzle RPA
PR9_RPA_Web_p1_var11
Переменные
Стандартные
Логика
Циклы
Мат.операции
Строки
Массивы
Словари
Табличные данные
Функции
Управление
Константы
Интеграция приложений
1С
Excel
Word
PowerPoint
Google Sheets
Telegram
Whatsapp
Десктопные приложения
Интерфейс
Клавиатура
Мышь
Веб-автоматизация
Браузер
Интерфейс
Мышь
База знаний


log.txt
log.txt
File
Edit
View
H1
2026-05-04 15:44:09: Проверка доступности кнопки
2026-05-04 15:44:09: Кнопка доступна, выполняем взаимодействие
2026-05-04 15:44:10: Запись добавлена успешно
2026-05-04 15:44:10: Колличество значений корректно
2026-05-04 15:44:10: Получение данных из таблицы
2026-05-04 15:44:10: Элемент найден
2026-05-04 15:44:15: Изменение статуса на Investigating
2026-05-04 15:44:16: Элемент найден
2026-05-04 15:44:21: Изменение статуса на Investigating
2026-05-04 15:44:21: Элемент найден
2026-05-04 15:44:26: Изменение статуса на Investigating
2026-05-04 15:44:26: Элемент найден
2026-05-04 15:44:31: Изменение статуса на Investigating
2026-05-04 15:44:31: Элемент найден
2026-05-04 15:44:37: Изменение статуса на Investigating
2026-05-04 15:44:37: Элемент найден
2026-05-04 15:44:42: Изменение статуса на Investigating
2026-05-04 15:44:42: Элемент найден
2026-05-04 15:44:47: Изменение статуса на Investigating
2026-05-04 15:44:47: Элемент найден
2026-05-04 15:44:53: Изменение статуса на Investigating
2026-05-04 15:44:53: Элемент найден
2026-05-04 15:44:58: Изменение статуса на Investigating
2026-05-04 15:44:58: Получение данных из таблицы
2026-05-04 15:44:58: Элемент найден
2026-05-04 15:45:03: Изменение статуса на Close Incident
2026-05-04 15:45:03: Элемент найден
2026-05-04 15:45:08: Элемент найден
2026-05-04 15:45:14: Элемент найден
2026-05-04 15:45:19: Элемент найден
2026-05-04 15:45:24: Изменение статуса на Close Incident
2026-05-04 15:45:24: Элемент найден
2026-05-04 15:45:29: Изменение статуса на Close Incident
2026-05-04 15:45:29: Элемент найден
2026-05-04 15:45:34: Изменение статуса на Close Incident
2026-05-04 15:45:35: Элемент найден
2026-05-04 15:45:40: Элемент найден
2026-05-04 15:45:45: Элемент найден
2026-05-04 15:45:50: Применение фильтра
2026-05-04 15:45:50: Поиск по тайтлу
2026-05-04 15:45:52: Получение данных из таблицы
2026-05-04 15:45:52: Поиск успешен
2026-05-04 15:45:52: Очищение фильтра
2026-05-04 15:45:54: Получение данных из таблицы
2026-05-04 15:45:54: Все статусы верны
Ln 117, Col 39
6,037 characters
Plain text
100%
Windows (CRLF)
UTF-8



DeepSeek

https://chat.deepseek.com/share/jtxxml03gzajbrawzr

GitHub https://github.com/NKTKLN/mirea-puzzle-rpa/tree/main/practice_10